Algoritmi e strutture dati

Alessandro Carella

Indice

1 Algoritmi e programmi **5**
- 1.1 Ciclo di sviluppo del software . . . 5
  - 1.1.1 Qualità esterne . . . 6
  - 1.1.2 Qualità interne . . . 6
- 1.2 Astrazione nei sistemi software . . . 7
- 1.3 Tecniche di progettazione . . . 8
- 1.4 Classificazione dei linguaggi di programmazione . . . 9

2 Algebre di dati **10**
- 2.1 Astrazione funzionale . . . 10
- 2.2 Astrazione di dati . . . 12
  - 2.2.1 Requisiti dell'astrazione di dati . . . 16
  - 2.2.2 Specifica e realizzazione . . . 16
- 2.3 Struttura di dati . . . 18
  - 2.3.1 Tecniche di specifica . . . 20

3 Introduzione alla programmazione object oriented **24**
- 3.1 Oggetti e classi . . . 24
- 3.2 Linguaggi Object Oriented . . . 25
  - 3.2.1 Ereditarietà . . . 25

4 Liste **27**
- 4.1 Strutture lineari . . . 27
  - 4.1.1 Modalità di accesso . . . 27
  - 4.1.2 Modalità di agire . . . 28
- 4.2 Lista . . . 28
  - 4.2.1 Specifica sintattica . . . 29
  - 4.2.2 Specifica semantica . . . 29

4.2.3 Realizzazione . . . 32
Rappresentazione sequenziale . . . 32
Rappresentazione collegata . . . 32
4.2.4 Realizzazione con cursori . . . 33
4.2.5 Realizzazione con puntatori . . . 37
4.2.6 Realizzazione a doppi puntatori . . . 39

5 Pile **40**
5.1 Specifiche . . . 40
5.2 Realizzazione con vettore . . . 41
5.3 Pila e procedure ricorsive . . . 43

6 Code **44**
6.1 Specifiche . . . 44
6.2 Realizzazione di una coda con vettore circolare . . . 45
6.3 Realizzazione con puntatori . . . 46

7 Insiemi **47**
7.1 Specifica . . . 48
7.1.1 Specifica semantica . . . 48
7.2 Realizzazione con vettore booleano . . . 49
7.3 Realizzazione con liste non ordinate . . . 51
7.4 Realizzazioni con liste ordinate . . . 51

8 Dizionari **53**
8.1 Operazioni . . . 53
8.2 Specifica sintattica . . . 54
8.3 Specifica semantica . . . 54
8.4 Rappresentazione con vettore ordinato . . . 55
8.4.1 Ricerca binaria . . . 55
8.5 Rappresentazione con tabella hash . . . 56
8.5.1 Collisioni . . . 57
8.5.2 Metodi di scansione . . . 60

9 Alberi **62**
9.1 Specifica . . . 64
9.2 Visite . . . 66
9.3 Realizzazioni . . . 67

9.3.1 Realizzazione con vettore dei padri . . . 67
9.3.2 Realizzazione con liste di figli . . . 68
9.3.3 Realizzazione con puntatori padre/primofiglio/fratello . 69
9.4 Alberi binari . . . 70

10 Code con priorità **75**
10.1 Specifica . . . 75
10.2 Realizzazioni . . . 76
10.2.1 Realizzazione con heap . . . 76

11 Alberi binari di ricerca **80**
11.1 Specifica . . . 81
11.2 Albero rosso-neri . . . 84
11.3 AVL Tree . . . 85
11.4 Treaps . . . 85

12 Grafi **87**
12.1 Specifica . . . 88
12.2 Realizzazioni con matrici . . . 89
12.2.1 Matrici di incidenza . . . 89
12.2.2 Matrici di adiacenza . . . 90
12.3 Realizzazione con vettore di adiacenza . . . 91
12.4 Rappresentazione con lista di adiancenza . . . 91
12.5 Rappresentazione con struttura a puntatori . . . 92
12.6 Esplorazione di un grafo . . . 92

13 Complessità degli algoritmi e efficienza dei programmi **95**
13.1 Risorse di interesse . . . 96
13.1.1 Dimensione dell'input . . . 97
13.2 Complessità in spazio . . . 97
13.3 Complessità in tempo e asintotica . . . 98
13.3.1 Modello di costo . . . 98
13.4 Notazioni . . . 102
13.5 Valutazione della complessità . . . 104
13.5.1 Istruzione dominante . . . 105
13.6 Classi di complessità . . . 106
13.7 Costi per ADT . . . 108

14 Tencniche algoritmiche **111**
14.1 Classificazione dei problemi 111
14.1.1 Problemi di ricerca 113
14.1.2 Problemi di decisione 114
14.1.3 Problemi di ottimizzazione 116
14.2 Spazio di ricerca 119
14.3 Teniche algoritmiche 124
14.3.1 Tecnica enumerativa 125
14.3.2 La tecnica del backtracking 128
14.4 La tecnica greedy 134
14.5 La tecnica divide-et-impera 140

Capitolo 1

Algoritmi e programmi

1.1 Ciclo di sviluppo del software

Il ciclo di sbiluppo del software, generalmente, è composto dalle seguenti attività.

- **Studio di fattibilità**. Questa fase ha lo scopo di valutare costi e benefici del sistema da costruire, quindi, vengono analizzati tempi e modi di attuazione, vengono valutate alternative possibili per lo sviluppo del sistema e delle risorse necessarie.
- **Raccolta e analisi dei requisiti**. In questa fase viene definito il problema e si specifica l'ambiente di sviluppo sia hardware che software. Alla fine viene prodotto un *documento di analisi* dove viene specificato l'hardware e il software scelto, la struttura concettuale dei dati da elaborare e le caratteristiche generali delle operazioni da fare.
- **Progettazione**. Nella fase di progettazione viene individuata la soluzione informatica del problema, si definisce l'architettura del programma specificandone la struttura, le funzioni delle componenti individuate. Inoltre, si sceglie le strutture di rappresentazione degli oggetti e si scrive il programma secondo il linguaggio e l'ambiente operativo scelto ottendo il **codice**.
- **Verifica**. In questa fase si analizza sia la documentazione prodotta che il programma realizzato, vengono, quindi, eseguite prove di correttezza sintattiche e logiche.

- **Manutenzione**. Durante il periodo di esercizio, si verifica che il programma produca i risultati attesi e se necessario, aggiornare il programma.

La fase di progettazione deve fornire un'analisi delle qualità che il programma deve possedere:

- **Esterne**. Si riferiscono a caratteristiche evidenziabili dal solo funzionamento del programma durante la fase di esercizio.
- **Interne**. Si riferiscono alle caratteristiche analizzabili e valutabili da esperti, attraverso uno studio delle scelte tecniche adottate.

1.1.1 Qualità esterne

Le qualità esterne del programma sono:

- **Correttezza**. La correttezza è la capacità di eseguire precisamente i compiti individuati durante l'analisi dei requisiti.
- **Efficienza**. L'efficienza è la capacità di usare in modo razionale ed economico le risorse di calcolo.
- **Robustezza**. La robustezza è la capacità di funzionare in modo soddisfacente in condizioni limite o anomale rispetto a quelle previste in fase di analisi dei requisiti.
- **Usabilità**. L'usabilità è la capacità di consentire un'interazione semplice, naturale ed efficace con l'utente finale.

1.1.2 Qualità interne

Le qualità interne del programma sono:

- **Riusabilità**. La riusabilità è la capacita di essere riutilizzato, in tutto o in parte, per applicazioni diverse rispetto a quella per la quale è stato prodotto.
- **Modularità**. La modularità è il grado di organizzazione interna del programma, quindi la strutturazione delle singole parti, della funzionalità e del modo in cui cooperano per l'obiettivo generale.
- **Estensibilità**. L'estensibilità è la capacità di adattarsi facilmente a modifiche nei requisiti.

- **Portabilità e compatibilità**. Essa è la facilità di trasferire il software prodotto in ambiti diversi.
- **Leggibilità**. Essa è la capacità del codice di essere autoesplicante.
- **Bontà della documentazione**. Essa è la completezza ed efficacia dei documenti annessi.

1.2 Astrazione nei sistemi software

Una descrizione specifica di un sistema pone enfasi su alcuni dettagli o proprietà e ne elimina altri. Il processo di astrazione prevede il decidere:

- quali caratteristiche sono rilevanti;
- quali parametri andrebbero inclusi;
- quale formalismo descrittivo andrebbe adottato;
- come validare il modello.

Come in altri campi, ci sono gerarchie di modelli e i livelli piu' bassi forniscono descrizioni piu' esaustive rispetto ai livelli superiori.
In programmazione ci si riferisce alla descrizione astratta fornita da un modello come alla specifica e al livello piu' bassi nella gerarichi di modelli come alla realizzazione.
Le astrazioni usate per il software tendono ad evidenziare gli aspetti funzionali, quindi cosa è calcolato piuttosto che come è condotto il procedimento di calcolo.
L'astrazione fornisce una distinzione tra il livello di *concettualizzazione* e quello di *realizzazione*. Nella fase di *concettualizzazione* viene individuata la soluzione al problema, vegono specificati gli oggetti da trattare e vengono ideate le scelte algoritmiche.
La fase di *realizzazione* si riferisce a quel processo che, partendo dallo schema concettuale, permette di ottenere uno specifico programma che realizza gli elementi dello schema e questo impone la scelta dei moduli software e della loro realizzazione.
L'astrazione viene usata per decomporre, rappresentare e generalizzare. La scomposizione di un problema in sottoproblemi è un metodo semplice per affrontare problemi complessi e l'astrazione è usata per cogliere gli aspetti essenzziali di

una situazione del mondo reale in modo da definire le opportune strutture di rappresentazione che ne evidenziano alcuni aspetti e ne trascurino altri.
Ci sono due tipi di astrazione:

- **Astrazione sui dati**. Consente di far riferimento a strtture algebrico-matematiche, caratterizzate da valori e da operazioni su tali valori. Si prescinde dal modo in cui queste strutture sono organizzate e realizzata mediante i costrutti di un linguaggio di programmazione.
- **Astrazione sulle funzioni**. Consetne di concentrare l'attenzione su cosa fa una particolare operazione piuttosto che sul come è fatta. Si astrae dalle modalità di realizzazione, ci si concentra sul compito o sulle funzionalità di un segmento di programma.

sui dati e sulle funzioni.

1.3 Tecniche di progettazione

Le tecniche di specifiche consentono di esprimere gli elementi essenziali dello schema concettuale mediante formalismi grafici o un linguaggio basato sulla logica e sull'algebra per descrivere gli aspetti concettuali dei tipi astratti di dato.
Le tecniche di programmazione riguardano i metodi per la strutturazione e per la stesura dei programmi a partire dallo schema concettuale e possono essere:

- programmazione strutturata, con schemi iterativi e ricorsivi;
- programmazione orientata agli oggetti;
- programmazione logica e funzionale.

La *modularizzazione* consente di razionalizzare lo sviluppo del software costruendo programmi costituiti da parti indipendenti e interagenti. Un modulo è caratterizzato da una struttura interna, è definito con un determinato scopo, offre all'esterno un insieme di funzionalità utilizzabili da altri moduli e ha precise relazioni con altri moduli. La modularizzazione facilita il processo di sviluppo e le attività di analisi e di verificae f avorisce la riusabilità. La qualità della modularizzazione aumenta:

- all'aumentare della **coesione** di un modulo in quanto incapsula un insieme di caratteristiche omogenee, indipendenti da altri moduli;

- all'aumentare dell'uso di**information hiding**, i dettagli interni al modulo non devono giocare alcun ruolo nell'utilizzo del modulo da parte di moduli esterni;
- al diminuire dell'**accopiamento** tra moduli. Non si creano dipendenze non volute o non necessarie.
- con l'utilizzo dell'**interfacciamento esplicito** che suggerisce di rappresentare mediante parametri tutti i dati che vengono scambiati tra due sottoprogrammi.

Le *tecniche di progettazione di algoritmi e strutture dati* riguardano la definizione degli algoritmi e la individuazione delle strutture dati piu' appropriate per un determinato problema, come tecniche enumerative, backtracking, divide et impera, ecc.

1.4 Classificazione dei linguaggi di programmazione

- **Linguaggi imperativi**. Il programma è un insieme di istruzioni che corrispondono a precisi comando impartiti ad una macchina che li esegue sequenzialmente.
- **Linguaggi di programmazione funzionale**. Il programma è la specifica di una funzione che, in base a un insieme di dati di ingresso, calcola il risultato secondo una legge specificabile in modo matematico.
- **Linguaggi di programmazione logica**. Il programma è la specifica di una relazione che sussiste tra un insieme di dati e la specifica è costruita mediante un sistema formale basato sulla logica.
- **Linguaggi di programmazione O.O.** Il programma è la specifica di un insieme di oggetti che rappresentano gli elementi della situazione in gioco in un certo problema, e ciascun oggetto è specificato in termini di una struttura e di un insieme di operazioni tramite le quali si ottiene il comportamento voluto per risolvere il problema.

Capitolo 2

Algebre di dati

2.1 Astrazione funzionale

Un programma corrisponde alla tripla $\{D, A, R\}$ ove:

- D corrisponde ai dati in input al programma;
- A corrisponde all'implementazione del programma;
- R corrisponde ai risultati ottenuti dal programma.

Si puo' dire che, poichè trasforma i dati iniziali in risultati, definisce un nuovo *operatore* sui dati e il repertorio di operatori disponibili sui dati puo' essere ampliato scrivendo programmi.
L'*astrazione funzionale* è la tecnica che permette di potenziare il linguaggio disponibile introducendo nuovi operatori:

- questi sono definiti attraverso sotto-programmi in cui si fa degli operatori di base già disponibili;
- l'intestazione del sotto-programma introduce il nome e gli argomenti dell'operatore, mentre il corpo ne descrive le regole di comportamento.

Esempio : Funzione per il calcolo del coefficiente binomiale
Consideriamo la formula per il calcolo del coefficiente binomiale:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

che fornisce il numero di possibili combinazioni di n oggetti presi a gruppi di k.
Il calcolo è immediato se si suppone di avere un operatore $fatt(i)$ che calcola il fattoriale di un valore i.

```
int binomial(int n, int k)
  return fatt(n) / fatt(k) * fatt(n-k);
```

Se $fatt(i)$ non è disponibile è necessario scrivere un sotto-programma per ampliare il repertorio degli operatori e l'astrazione funzionale ci consente questo.

I linguaggi di programmazione ad alto livello permettono l'uso di astrazione funzionale, cioè consentono di:

- creare delle unità di programma dando un nome ad un gruppo di istruzioni e stabiliscono le modalità di comunicazione tra l'unità di programma creata ed il resto del programma in cui essa si inserisce;
- al momento della attivazione (su chiamata) della unità del programma viene sopsesa l'esecuzione del programma chiamante: il controllo passa all'unità attivata e, all'atto del completamento della sua esecuzione, l'attivazione termina e il controllo torna al programma chiamante.

I costrutti linguistici per realizzare l'astrazione funzionale consentono di definire una specifica e una realizzazione:

- la specifica definisce "cosa" ci si aspetta dalla funzione, cioè definirà la relazione che caratterizza il legame tra dati di ingresso e risultati.
  Nel caso del fattoriale la specifica dirà che l'intestazione è "int fatt(int k)"; l'argomento k è una variabile di tipo intero e il risultato è ancora di tipo intero. La funzione da calcolare è definita come: fatt(k) = k(k-1)$\cdot \cdot \cdot 2 \cdot 1$
- La realizzazione definisce come il risultato viene ottenuto. Alla stessa specifica potranno corrispondere scelte di realizzazione diverse. La funzione "fatt" potrebbe essere realizzata con un programma iterativo:

```
fatt(int k)
  f = 1
  for i = 2 to k do
    f = f * i
  return f
```

  ma anche come un programma ricorsivo:

```
fatt(int k)
  if k = 1 then f = 1
  else f = k * fatt(k - 1)
  return f
```

2.2 Astrazione di dati

La astrazione di dati ricalca ed estende il concetto di astrazione funzionale. Cosi' come l'astrazione funzionale permette di ampliare l'insieme di modi di operare sui dati, la astrazione di dati permette di ampliare i tipi di dati disponibili attraverso l'introduzione sia di nuovi tipi di dati che di nuovi operatori.
L'astrazione funzioanle stimola gli sforzi per evidenziare operazioni ricorrenti o ben caratterizzate all'interno della soluzione di un problema.
L'astrazione di dati sollecita ad individuare le organizzazioni dei dati piu' adatte alla soluzione del problema.
Un'astrazione di dati consente una estensione dell'algebra dei dati disponibile in un linguaggio di programmazione. Un'*algebra* è un sistema matematico costituito da un dominio e da un insieme di funzioni applicabili su tali valori: il numero, il tipo e le proprietà delle funzioni sono gli elementi essenziali di un algebra.
La corrispondenza tra tipo astratto e algebra si basa sul parallelo tra gli insiemi di valori di un algebra e i domini di definizione di un tipo astratto, e tra le funzioni di un'algebra e le operazioni associate a un tipo astratto.
Molti linguaggi di programmazione forniscono i mezzi per definire e creare nuovi tipi di dati, ma non tutti consentono di fare astrazione dati o crare tipi di dati astratti. Un tipo di dati si dice astratto se le operazioni applicabili sugli oggetti che li rappresentano sono isolate dai dettagli usati per realizzare il tipo.
Un'algebra di dati puo' essere definita come costituita da:

- A) una famiglia di insiemi, cioè l'insieme dei dati;
- B) una famiglia di operatori sui dati;
- C) un repertorio di simboli o nomi per indicare l'insieme dei dati;
- D) un repertorio di simboli o nomi per indicare gli operatori;
- E) un repertorio di simboli, detti *costanti*, per indicare elementi singoli degli insiemi dei dati.

Esempio : Esempio di un'algebra di dati

- A) La famiglia di **insiemi** comprende: i numeri interi, i booleani, le stringhe;
- B) la famiglia di **operatori** comprende:
    - operatori aritmetici come somma, sottrazione, moltiplicazione e divisione;
    - operatori di congiunzione, disgiunzione e negazione per i booleani;
    - operatori di confronto tra interi come maggiore, minore, uguale ecc.;
    - operatori di concatenazione per le sequenze di caratteri e di selezione di sottostringa
- C) il repertorio dei simboli o nomi per l'insieme dei dati potrebbe essere: integer, boolean, string;
- D) il repertorio dei simboli o nomi per gli operatori potrebbe essere:
    - +, -, *, / per gli operatori aritmetici;
    - and, or, not per gli operatori booleani;
    - $<$, $>$, = per i confronti tra interi;
    - substr e & per la selezione e concatenazione di stringhe.
- E) le costanti per gli insiemi di dati potrebbero essere:
    - ogni intero viene indicato da una sequenza di cifre decimali, eventualmente precedata da + o -, interpretata secondo la notazione decimale;
    - ogni sequenza di caratteri viene indicata dalla sequenza racchiusa tra apici;
    - i valori booleani sono indicati da true e false.

Gli operatori possono essere combinati per ottenere delle espressioni. Le caratteristiche di un'algebra dipendono dalla famiglia di insiemi e dalla famiglia di operatori.
Nei linguaggi di programmazione il binomio "dati-operatori" è inscindibile. Dun-

que, è necessario definire un tipo di dati come un insieme di dati piu' gli operatori che ad esso si riferiscono, nel contesto di una definita algebra.
Anche un'astrazione di dati è costituita da:

- una **specifica**: ha il compito di descrivere sintenticamente il tipo dei dati e gli operatori che li caratterizzano;
- una **realizzazione**: stabilisce come i dati e gli operatori vengono ricondotti ai tipi di dati e agli operatori già disponibili.

La scelta delle strutture dati è il primo passo per un buon risultato nell'attività di programmazione.

Esempio : Un esempio di astrazione dati

Supponiamo di voler effettuare un'astrazione di dati aggiungendo ai tipi di dati stnadard i numeri complessi.
I numeri complessi hanno forma: $r_1 + i \cdot r_2$ ove r_1 e r_2 sono numeri reali e $i = \sqrt{-1}$.
Per denotarli matematicamente basta la coppia (r_1, r_2) dove r_1 è il coefficiente della parte reale e r_2 quello della parte immaginaria.
è necessario definire operatori di selezione e costruzione che operano su complessi e su reali, operatori di somma e prodotto che operano esclusivamente sui complessi.
Il calcolo di somma e prodotto su numeri complessi non è difficile se ricordiamo che:

- la somma di due numeri complessi c_1 e c_2 dà luogo ad un nuovo numero complesso c_3 la cui parte reale è la somma delle parti reali e la cui parte immaginria è somma delle parti immaginiarie:
  - parteReale($c3$) = parteReale(c_1) + parteReale(c_2)
  - parteImmg(c_3) = parteImmg(c_1) + parteImmg(c_2)
- il prodtto di due numeri complessi c_1 e c_2 dà luogo ad un nuovo numero complesso c_3, la cui parte reale è la differenza tra il prodotto della parte reale del primo numero per la parte reale del secondo numero e il prodotto della parte immaginaria del primo numero per la parte immaginaria del secondo numero:
  - parteReale(c_3) = parteReale(c_1) * parteReale(c_2) - parteImmg(c_1) * parteReale(c_2)

e la cui parte immaginaria è somma del prodtto della parte reale del primo numero per la parte immaginaria del secondo numero con il prodotto della parte immaginaria del primo numero con la parte reale del secondo numero:

  - parteImmg(c_3) = parteReale(c_1) * parteImmg(c_2) + parteReale(c_2) * parteImmg(c_1)

Passando agli operatori necessari, abbiamo bisogno di definire, attraverso procedure e funzioni:

- cxsum(c1, c2) = c3 cioè la somma di complessi;
- cxprod(c1, c2) = c3 cioè il prodotto di complessi;
- realpart(c) = r che seleziona la parte reale del complesso c;
- immgpart(c) = i che seleziona la parte immaginaria del complesso c;
- conscx(r1, r2) = c che è la combinazione di due reali, il primo come parte reale, il secondo come immaginaria del complesso c.

Per poter fare astrazione dati, bisogna disporre di un lingaggio di programmazione a tipizzazione forte come il pascal o il C.
In Pascal, infatti, si puo' dichiarare:

```
TYPE
  COMPLEX = ARRAY[2] OF REAL;
  VAR
    C1, C2, C3, C4: COMPLEX;
  PROCEDURE SOMMACOMPLESSI(
    C1, C2: COMPLEX;
    VAR C3: COMPLEX);
```

oppure:

```
TYPE
  COMPLEX = RECORD OF
    PARTEREALE: REAL;
    PARTEIMMAGINARIA: REAL
  END;
```

Disponendo di un linguaggio con tipi, possiamo utilizzare dati che definiamo e dichiaramo direttamente come complessi, prescindendo dalla effettiva realizza-

zione, e fare in modo che alle procedure costruite per i nostri dati abbiano accesso esclusivamente a quei dati.
Queste caratteristiche necessarie a una buona astrazione di dati sono note come *i requisiti dell'astrazione di dati.*

2.2.1 Requisiti dell'astrazione di dati

- Il **requisito di astrazione** è verificato quando i programmi che usano un'astrazione possono essere scritti in modo da non dipendere dalle scelte di realizzazione.
  La mancanza del requisito di astrazione si manifesta con l'impossibilità di dichiarare direttamente variabili con il nuovo tipo definito, quando non posso fare riferimento ai nuovi dati creati prescindendo dalla rapparesentazione usata e dalla realizzazione;

- Il **requisito di protezione** è verificato se sui nuovi dati si puo' lavorare esclusivamente con gli operatori definiti all'atto della specifica.
  La mancanza del requisito di protezione si manifesta con la possibilità di lavorare con gli operatori definiti per i nuovi dati anche su dati con rappresentazioni simili, ma non omogenei per tipo.

Il requisito di astrazione, in accordo con i principi dell'astrazione di dati, impone che i programmi siano sensibili solo a variazioni della specifica dei nuovi tipi. Il requisito di protezione è particolarmente importante, se vogliamo che l'esecutore del linguaggio assicuri i controlli di consistenza tra i tipi di dati e gli operatori, non solo ai dati primitivi, ma anche ai dati ottenuti per astrazione.

2.2.2 Specifica e realizzazione

Si è detto che un'astrazione di dati è costituita da una **specifica** e una **realizzazione**.
Una specifica, che ha il compito di descrivere sinteticamente il tipo dei dati e gli operatori che li caratterizzano, si distingue in:

- **specifica sintattica** che fornisce:
  - l'elenco dei nomi dei tipi di dato utilizzati per definire la struttura, delle operazioni specifiche della struttura e delle costanti;

  - i domini di partenza e di arrivo, cioè i tipi degli operandi e del risultato per ogni nome di operatore.

- **specifica sementica** che definisce il significato dei nomi introdotti con la specifica sintattica. Associa:

  - un insieme ad ogni nome di tipo introdotto nella specifica sintattica;
  - un valore ad ogni costante;
  - una funzione ad ogni nome di operatore esplicitando le seguenti condizioni sui domini di partenza e di arrivo:
    * **precondizione** che definisce quando l'operatore è applicabile;
    * **postcondizione** che stabilisce come il risultato sia vincolato agli argomenti dell'operatore.

La realizzazione sfrutta al meglio le possibilità offerte dall'ambiente di programmazione. La definizione di un formalismo per le specifiche sintattiche non pone particolari problemi. L'individuazione di formalismi per le specifiche semantiche è piu' complicato, in genere le specifiche semantiche sono date con frasi estratte dal linguaggio naturale o da quello matematico. Vedere dalle slide l'esempio di specifica.
La realizzazione riconduece la specifica ai tipi di dato primitivi e agli operatori già disponibili, in particolare gli operatori sono simulati con sottoprogrammi.
Alla stessa specifica possono corrispondere diverse realizzazioni piu' o meno efficienti. Nel valutare l'efficienza di sottoprogrammi che utilizzano tipi di dato primitivi, si precinde dalla caratteristiche della macchina, assumendo di far riferimento ad una organizzazione molto semplice. Si fa riferimento a:

- dati contenuti in memoria;
- memoria organizzata in celle (parola);
- celle accessibili tramite indirizzo;
- indirizzi che sono interi consecutivi;
- dati elementari che non decidono la capacità di una cella;
- variabili contenute in celle di indirizzo noto il cui accesso richiede tempo costante.

Un costrutto di programmazione propriamente adatto alla realizzazione dati dovrà:

- dichiarare esplicitamente quali sono i nuovi tipi di dati e quali sono i nuovi operatori;
- definire la rappresentazione dei nuovi dati in termini di dati pre-esistenti;
- contenere un insieme di sottoprogrammi che realizzino gli operatori definiti sui nuovi dati.

2.3 Struttura di dati

Il tipo di dato è un modello matematico che consiste nella definizione di una collezione di valori sui quali sono ammesse certe operazioni.
Una struttura du datu è un particolare tipo di dato, caratterizzato piu' dalla organizzazione imposta agli elementi che la compongono che dal tipo degli elementi componenti.
Le strutture solitamente disponibili nei linguaggi di programmazione sono le tabelle monodimensionali, come vettori o array, i cui elementi sono contenuti in memoria centrale in locazioni contigue. In generale, distinguiamo strutture:

- **lineari**: dati disposti in sequenza;
- **non lineari**: in cui non è individuata una sequenza;
- **a dimensione fissa (statiche)**: in cui il numero di elementi non puo' variare nel tempo;
- **a dimensione variabile (dinamiche)**: in cui il numero di elementi puo' variare nel tempo;
- **omogenee**: dati dello stesso tipo;
- **non omogenee**: dati non dello stesso tipo.

Esempio : Esempio di struttura dati

La classica struttura vettore (array) è a dimensione fissa. è una tabella monodimensionale di elementi omogenei, su cui possono effettuarsi operazioni di:

- lettura o selezione: reperimento del valore di un elemento;

- scrittura o sostituzione di un valore di un elemento con un nuovo valore.

Un vettore di n componenti è inteso come sequena di ogetti in realazione d'ordine tra loro e l'organizzazione in memoria consente l'accesso diretto al singolo componente attraverso l'indice.
Se volessimo definire le specifich edi un vettore:

- Specifica sintattica:

```
Tipi:
  vettore, intero, tipoelem
Operatori:
  creavettore: () -> vettore
  leggivettore: (vettore, intero) -> tipoelem
  scrivivettore: (vettore, intero, tipoelem) -> vettore
```

- Specifica semantica:

```
Tipi:
  intero: l'insieme dei numeri interi
  vettore: l'insieme delle sequenza di n elementi di tipo
  tipoelem
Operatori:
  creavettore = v
    post: per ogni i, 1 <= i <= n, l'i-esimo elemento
    del vettore v(i) e' uguale ad un definito elemento
    di tipo tipoelem
  leggivettore(v, i) = e
    pre: 1 <= i <= n
    post: e = v(i)
  scrivivettore(v, i, e) = v'
    pre: 1 <= i <= n
    post: v'(i) = e, v'(j) = v(j)
    per ogni j tale che 1 <= j <= n e j != i
```

- Realizzazione:
  In molti linguaggi il vettore è il tipo di dato primitivo:

  - in Fortran creavettore corrisponde a dimension v(n), leggivettore(v, i) corrisponde a v(i) e scrivivettore(v, i, e) corrisponde a v(i) = e
  - in Pascal corrisponde all'array, creavettore corrisponde a var v: array[1..n] of type tipoelem, leggivettore(v, i) corrisponde a v[i] e scrivivettore(v, i, e) corrisponde a v[i] := e

Vedere esempio della matrice dalle slide.

2.3.1 Tecniche di specifica

Due tecniche comuni per la scrittura di specifiche sono:

- specifiche assiomatiche o algebriche;
- modelli astratti (approccio costruttivo).

In entrambe le tecniche la sintassi di ogni operatore viene specificata in termini del nome dell'operatore e dei tipi di parametri. La differenza fra le due sta nel come viene definita la semantica degli operatori.
Le specifiche assiomatiche sono self-contained e specificano ogni oggetto come composizione di funzioni.
I modelli astratti definiscono la semantica delle operazioni in termini di un altro ben definito tipo di dato. La chiave di queste e di altre tecniche di specifica sta nel fatto che la descrizione della semantica non fa riferimento alla realizzazione.
Possiamo definire formalmente una specifica di un tipo di dato astratto come una tripla (D, F, A):

- D: l'insieme di tutti i domini per la definizione del dato.
  Il tipo di dato che stiamo defininendo, indicato con il simbolo d, è chiamato **domionio designato**; tutti gli altri domini che fanno parte del dato sono detti **domini ausiliari**.
  Nel caso dello Stack, D potrebbe essere Stack, ItemType, Boolean ove $d =$Stack e i domini ausiliari sono ItemType, Boolean
- F è l'insieme degli operatori;
- A è l'insieme degli assiomi o regole che descrivono la semantica degli operatori, nel caso dello Stack dovrebbero definire la proprietà LIFO dello stack.

Tipi di operatori
Gli operatori si suddividono in:

- **Costruttori** (Constructors): costruiscono/inizializzano una nuova istanza del dato;

- **Modificatori** (Transformers/Modifiers): operatori che cambiano la struttura in qualche modo;
- **Osservatori** (Observers/Accessors): permettono di osservare lo stato del dato senza modficarlo
- **Distruttori** (Destructors): puliscono l'occupazione di memoria del dato e la rilasciano per il riuso;
- **Iteratori** (Iterators): sono usati per permettere all'utente di processare l'intera struttura, componente per componente.

Esempio : Esempio di specifica assiomatica

Le specifiche assiomatiche definiscono il comportamento di un tipo di dato astratto dando gli **assiomi** che legano tra loro gli operatori.
Vediamo un esempio di una specifica algebrica per il tipo Stack:

struttura Stack(di ItemType)

interfaccia

Create	→ Stack	d = Stack(di ItemType)
Push(Stack, ItemType)	→ Stack	
Pop(Stack)	→ Stack	
Top(Stack)	→ ItemType	
IsEmpty(Stack)	→ Boolean	

D = Stack, ItemType, Boolean
F = Create, Push, Pop, Top, IsEmpty
L'interfaccia elenca le funzioni, il tipo dei parametri e tipi per i risultati. Non c'è nessuna indicazione su cosa faccia ogni funzione, la semantica degli operatori viene dato con gli assiomi.
Ogni istanza del dato astratto Stack puo' essere denotata come una sequenza di applicazioni di operatori detta **espressione**, ad esempio: Push(Push(Push(Create, 1), 2), 3)
Gli operatori observer non fanno parte di una espressione, in quanto vengono applicati ad una espressione perchè non modificano la struttura.
Gli assiomi costituiscono la terza componente della nostra tripla:

1. Gli assiomi per gli observer sono scritti in termini dei costruttori;
2. I costruttori che rimuovono un elemento dalla struttura sono definiti in termini dei costruttori che aggiungono un elemento alla struttura;
3. I costruttori che creano la struttura o aggiungono un elemento sono definiti esplicitamente.

Per la struttura Stack avremo, **per ogni S in Stack, i in ItemType**:

- IsEmpty(Create) = True
- IsEmpty(Push(S, i)) = False
- Top(Create) = Error
- Top(Push(S, i)) = i
- Pop(Create) = Error
- Pop(Push(S, i)) = S

Esempio : Esempio di specifica costruttiva o modello astratto

Specifica sintattica:

struttura	Stack(di ItemType)		
interfaccia	Create	→ Stack	d = Stack(di ItemType), l'in-
	Push(Stack, ItemType)	→ Stack	
	Pop(Stack)	→ Stack	
	Top(Stack)	→ ItemType	
	IsEmpty(Stack)	→ Boolean	

sieme di tutte le istanze di Stack.
ItemType è l'insieme di tutti gli item che possonoe essere presenti in uno stack.
Boolean è l'insieme dei valori di verità true e false.

Specifica semantica:

In questo approccio diamo la semantica di ogni operatore associando ad ognuno una precondiziona e una postcondizione. La precondizione è un'asserzione logica che specifica le caratteristiche richieste dei valori dei parametri, la postcondizione è un'asserzione lgoica che specifica le caratteristiche del risultato rispetto agli argomenti.

- Push(S, I) → S':
  - PRE: -
  - POST: S' è lo stack S con I al top
- Pop(S) → S':
  - PRE: not empty(S)
  - POST: S' è lo stack S con l'elemento top eliminato

- Top(S) $\rightarrow$ I:
  - PRE: not empty(S)
  - POST: I è l'elemento al top dello stack S

Capitolo 3

Introduzione alla programmazione object oriented

Nella programmazione imperativa è possibile accedere alle variabili globali da qualsiasi parte del programma ma nei programmi molto grandi questa tecnica diventa ingestibile.
Pertando è stato pensato di incapsulare ogni variabile globale in un modulo insieme ad un gruppo di operatori che possono accedere a tali variabilie e gli altri moduli possono accedere indirettamente alle variabili solo per mezzo di questi operatori.
La programmazione Object-Oriented è una disciplina che si basa su oggetti per imporre una struttura modulare sui programmi.

3.1 Oggetti e classi

Un oggetto è definito come un gruppo di procedure che condividono uno stato. Quindi un oggetto è una collezione di dati che rappresetano gli *stati dell'oggetto* e procedure in grado di alterare lo stato, il *comportamento*.
Agli oggetti sono associati operazioni e valori.

> **Esempio : Coda**
>
> La Coda è la classe di tutti gli oggetti coda, q è un oggetto della classe Coda e sono definite le seguenti operazioni: creaCoda, aggiungi(q, i), primo(q), rimuovi(q) e CodaVuota(q).
> Nei linguaggi orientati agli oggetti, creaCoda è detto **costruttore** e crea

l'oggetto q. Anche il **distruttore**, che distrugge un oggetto, fa parte delle operazioni applicabili su un oggetto.

3.2 Linguaggi Object Oriented

Un linguaggio object-based supporta:

- **information hiding (encapsulation)**;
- **Data abstraction**;
- **Message passing (polymorphism)**.

Un linguaggio object-oriented implementa anche l'**ereditarietà**, incluso il dynamic binding, dove gli oggetti vengono organizzati secondo una gerarchia di classi e un oggetto puo' ereditare le proprietà della sua classe genitore.
Un oggetto è un'entità dinamica che potrebbe cambiare ma resta sempre lo stesso oggetto. Un programmatore object oriented affronta un problema dividendolo in oggetti che fanno qualcosa ed interagiscono fra loro.
Gli oggetti sono costituiti da dati privati e da operazioni permesse su tali dati e gli oggetti comunicano fra loro con il passaggio di messaggi che non sono altro che chiamate a delle procedure, dette *metodi*, che appartiene all'oggetto e potrebbe essere nascosta all'utente.
Una classe è una descrizione di oggetti da istanziare e nei linguaggi object oriented una classe è una collezione di oggetti, in cui ogni oggetto della classe include gli stessi metodi e le stesse variabili ma puo' includere diversi valori per i dati.

3.2.1 Ereditarietà

I linguaggi object-oriented, come detto precedentemente, supportano l'**ereditarietà** ovvero gli oggetti vengono organizzati secondo una gerarchi di classi e un oggetto puo' ereditare le proprietà della classe genitore. Gli attributi di una classe ereditata possono essere:

- **ridefiniti**: un attributo che ha lo stesso nome come in un superclasse, ma è definito in una sottoclasse;
- **specifici**: un attributi definito solo in una sottoclasse;
- **ereditati**: un oggetto ha un attributo definito solo in una delle sue superclassi.

Alcuni oggetti potrebbero ereditare piu' classi e un problema che si potrebbe verificare è il **name clashes** fra i metodi, ovvero se c'è un metodo con lo stesso nome nelle classi ereditate. In questo caso, in alcuni linguaggi object-oriented come il C++, ci sono operatori di scopre resolution, :: in C++, per decidere quale metodo utilizzare.

Capitolo 4

Liste

4.1 Strutture lineari

Consideriamo strutture di dati che si sviluppano in una dimensione possono essere considerate come sequenze di oggetti, cioè *strutture lineari di dati*. In esse è importante l'esistenza di una relazione d'ordine tra gli oggetti che ci aiuta a individuarli e a selezionarli. Per distinguere le diverse strutture, bisogna considerare:

- le modalità di accesso ai dati;
- i modi di agire nella posizione individuata.

4.1.1 Modalità di accesso

Le modalità di accesso ai dati all'interno di una struttura possono essere di diverso tipo.

- Accesso diretto.
- Accesso per scansione.
- Accesso agli estremi.

Il vettore, l'array, è tipico esempio di accesso diretto in quanto consente di accedere al singolo componentne mediante il nome e il meccanismo dell'indice.
La lista è un esempio di accesso per scansione in quanto consente l'accesso all'elemento generico solo dopo aver scandito gli elementi che lo precedono.

La pila e la coda sono esempi di accesso agli estremi dove la pila ha una filosofia LIFO e la coda ha una filosofia FIFO.

4.1.2 Modalità di agire

Relativamente ai modi di agire nelle posizioni individuate possiamo elencare i seguenti tipi di operazione.

- Lettura del valore di un componente, detto *ispezione*;
- Aggiornamento del valore di un componente, detto *cambio di valore*;
- Inserimento di un nuovo componente, dettto *scrittura*;
- Rimozione di un componente, detto *cancellazione*.

Sono possibili diverse combinazioni tra i modi di accedere e i modi d'operare, l'unico problema riguarda la combinazione tra accesso diretto a tutta la struttura e possibilità di scrittura e cancellazione di componenti.
Generalmente queste operazioni alternano la numerazione dei componenti e questo è in contrasto col principio che i componenti siano individuati dal numero d'ordine della loro posizione.
Tutte le strutture dati lineari, escluso il vettore, sono dotate di operatori di inserimento e rimozione di componenti.

4.2 Lista

Una lista è una sequenza finita, anche vuota, di elementi dello stesso tipo in cui è possibile aggiungere o togliere elementi. Per fare questo, occorre specificare la posizione relativa all'interno della sequenza nella quale il nuovo elemento va aggiunto o dalla quale il vecchio elemento va tolto. La lista è una *struttura dati dinamica*, cioè a dimensione variabile e su puo' accedere direttamente solo ad un ristretto sottoinsieme di elementi, di solito il primo e per accedere ad un generico elemento occore scandire sequenzialmente gli elementi della lista.
è bene notare che in una lista, a differenza di un insieme, uno stesso elemento puo' apparire piu' volte e gli elementi della lista, ai quali sono associati delle informazioni, sono definiti *atomi* o *nodi*.
Indichiamo la lista con la notazione:

$$l = < a_1, a_2, \ldots, a_n > \quad n \geq 0$$

La lunghezza di una lista è il numero dei suoi elementi. Se il numero di elementi è zero la lista si dice vuota. Inoltre, la lunghezza conta le posizioni, non i simboli distinti, cosi un simbolo che compare k volte contribuisce con k unità alla lunghezza della lista.
Se $l = < a_1, a_2, \ldots, a_n >$ è una lista allora $\forall i, j \mid 1 \leq i \leq j \leq n$ anche $< a_i, \ldots, a_j >$ è una sottolista di l, ottenuta partendo dalla posizione i e prendendo tutti gli elementi fino alla posizione j. Inoltre, la lista vuota $<>$ è sottolista di qualsiasi lista.

4.2.1 Specifica sintattica

La specifica sintattica per il tipo di dato lista è la seguente.

- Tipi: lista, posizione, boolean, tipoelem
- Operatori:
  - `crealista` : () → lista
  - `listavuota` : (lista) → boolean
  - `leggilista` : (posizione, lista) → tipoelem
  - `scrivilista` : (tipoelem, posizione, lista) → lista
  - `primolista` : (lista) → posizione
  - `finelista` : (posizione, lista) → boolean
  - `succlista` : (posizione, lista) → posizione
  - `predlista` : (posizione, lista) → posizione
  - `inslista` : (tipoelem, posizione, lista) → lista
  - `canclista` : (posizione, lista) → lista

4.2.2 Specifica semantica

La specifica semantica per la struttura dati lista è la seguente.

- Tipi:
  - Lista: insieme delle sequenza $l = < a_1, a_2, \ldots, a_n >$ con $n \geq 0$, di elementi di tipoelem dove l'elemento i-esimo ha valore a(i) e posizione pos(i);

- boolean: insieme dei valori di verità.

- Operatori:
  - `crealista` = l'
    - POST: $l' =<>$ lista vuota
  - `listavuota(l)` = b
    - POST: $\begin{cases} b = true & l = <> \\ b = false & l \neq <> \end{cases}$
  - `leggilista(p, l)` = a
    - PRE: $p = pos(i) \quad 1 \leq i \leq n$
    - POST: $a = a(i)$
  - `scrivilista(a, p, l)` = l'
    - PRE: $p = pos(i) \quad 1 \leq i \leq n$
    - POST: $l' =< a_1, a_2, \ldots, a_{i-1}, a, a_{i+1}, \ldots, a_n >$
  - `primolista(l)` = p
    - POST: $p = pos(1)$
  - `finelista(p, l)` = b
    - PRE: $p = pos(i) \quad 1 \leq i \leq n+1$
    - POST: $\begin{cases} b = true & p = pos(n+1) \\ b = false & altrimenti \end{cases}$
  - `succlista(p, l)` = q
    - PRE: $p = pos(i) \quad 1 \leq i \leq n$
    - POST: $q = pos(i+1)$
  - `predlista(p, l)` = q
    - PRE: $p = pos(i) \quad 2 \leq i \leq n$
    - POST: $q = pos(i-1)$
  - `inslista(a, p, l)` = l'
    - PRE: $p = pos(i) \quad 1 \leq i \leq n+1$
    - POST: $\begin{cases} l' =< a_1, \ldots, a_{i-1}, a, a_{i+1}, \ldots, a_n > & 1 \leq i \leq n \\ l' =< a_1, a_2, \ldots, a_n, a > & i = n+1 \\ l' =< a > & i = 0 \; e \; l = <> \end{cases}$

- `canclista(p, l)` = l'
  - PRE: $p = pos(i) \quad 1 \leq i \leq n$
  - POST: $l' = < a_1, a_2, \ldots, a_{i-1}, a_{i+1}, \ldots, a_n >$

Dalla specifica semantica emerge che per accedere a un elemento occorre conoscerne la posizione. Queseto nelle liste è possibile solo per il primo elemento, L'unico operatore che da; per risultato la posizione è `primolista`. Per gli altri la posizione si ottiene conoscendo a priori la posizione dell'elemento precedente o seguente e applicando l'operatore `predlista` o `succlista`.
Dunque per accedere ad un generico elemento occorre scandire la lista a partire dal primo elemento.
L'operatore `listavuota` è ridondante visto che puo' essere sostituito da:

```
finelista(primolista(l), l)
```

Talvolta, l'operatore `primolista(l)` puo' avere implementazioni differenti, a volte ritorna l'elemento primo e non la posizione.

Esempio : Eliminazione di duplicati

Questo è un esempio di come sia possibile, avendo a disposizione gli operatori definiti nell'algebra per il trattamento di una lista, risolvere un generico problema, senza sapere come la lista sia stata implementata.
Utilizziamo come linguaggio di programmazione il pascal e supponiamo di avere a disposizione degli operatori sul tipo lista.
Quello che vogliamo fare è eliminare gli elementi duplicati in una lista.

```
elimina_duplicati(lista l)
  p = primolista(l)
  while not finelista(p, l) do
    q = succlista(p, l)
    while not finelista(q, l) do
      if leggilista(p, l) == leggilista(q, l) then
        canclista(q, l)
      q = succlista(q, l)
    p = succlista(p, l)

```

Quindi abbiamo fatto riferimento alla struttura lista senza sapere i dettagli realizzativi.

4.2.3 Realizzazione

Per realizzare una lista distinguiamo due rappresentazioni fondamentali:

1. rappresentazione sequenziale;
2. rappresentazione collegata.

Rappresentazione sequenziale

Una lista puo' essere rappresentata usando un vettore e visto che il numero di elementi che compongono la lista puo' variare si utilizza una variabile *primo* per il valore dell'indice della componente del vettore in cui è memorizzato il primo elemento della lista.
Inoltre, si utilizza un'altra variabile *lunghezza* per indicare il numero di elementi di cui è composta la lista rappresentata.
Ad esempio data, una lista l:

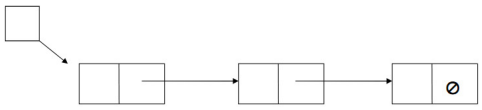
$$l = < 4, 5, 1, 21, 45 >$$

la variabile *primo* ha valore 1 e la variabile *lunghezza* ha valore 5.
Questa rappresentazione consente di realizzare molto semplicemente alcune delle operazioni definite per la lista ma crea problemi per l'inserzione e la rimozione di componenti, infatti se vogliamo aggiungere l'elemento 11 nella posizione 4, $l = < 4, 5, 1, 11, 21, 45 >$, dovremmo spostare verso il basso il quarto e il quinto elemento.

Rappresentazione collegata

L'idea fondamentale della rappresentazione collegata di una lista è quella di memorizzare i suoi elementi associando ad ognuno di essi una particolare informazione, il *riferimento*, che permetta di individuare la locazione in cui è memorizzato l'elemento successivo. Per visualizzare tale rappresentazione si usa una notazione grafica in cui:

- gli elementi sono rappresentati mediante nodi;
- i riferimenti mediante archi che collegano nodi.



Si puo' notare che si usa un riferimento al primo elemento della lista, detto *riferimento iniziale* e un simbolo speciale $\emptyset$ come riferimento associato all'ultimo nodo, nel caso in cui la lista sia vuota, tale simbolo compare direttamente al riferimento iniziale.

4.2.4 Realizzazione con cursori

Nei linguaggi che non hanno i puntatori come tipi di dato primitivi è possibile simulare i puntatori con i cursori, cioè con variabili intere il cui valore è interpretato come un indice di vettore. Per fare questo si definisce un vettore spazio che:

- contiene tutte le liste, ognuna individuata da un proprio cursore iniziale;
- contiene tutte le celle libere, organizzate anch'esse in una lista, detta *lista libera*.

Tramite la realizzazione con cursori, si riesce a risolvere il problema dell'aggiornamento attraverso i riferimenti.

Esempio : Lista tramite cursori

Consideriamo tre liste:

$$l =< 7, 2 > \quad m =< 4, 9, 13 > \quad s =< 13, 4, 9, 13 >$$

e possiamo utilizzare un vettore spazio per rappresentare le tre liste. La componente di spazio ha due campi:

1. il campo *elemento* dove è memorizzato il contenuto del nodo;
2. il campo *successivo* che contiene il riferimento al nodo successivo.

La sequenza degli elementi che formano la lista è ricostruibile iniziando dalla componente dell'array corrispondente al valore di inzio, nel cui campo *elemento* è memorizzato il valore del primo elemento. Seguendo i cursori si trovano gli elementi successivi della lista. Infatti, si può definire la posizione $pos(i)$ dell'i-esimo elemento della lista l uguale al valore del cursore alla cella

del vettore spazio che contiene l'elemento $(i-1)-$esimo, se $2 \leq i \leq n+1$ uguale a 0 se $i = 1$.
Analogamente si definisce $pos(n+1)$ uguale al cursore alla cella di spazio che contiene l'elemento $n-$esimo se $n \geq 1$ o uguale a 0 altrimenti. La lista vuota si indica con $l = \emptyset$.

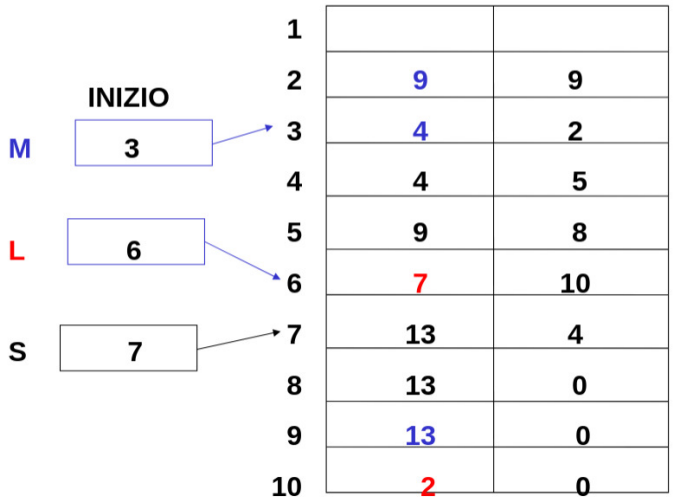


Figura 4.1: Rappresentazione di una lista mediante cursori

Per poter aggiornare una lista così realizzata sorge il problema di individuare la posizione di una componente libera nell'array. Per questo si usa la *listalibera* memorizzata sempre nel vettore spazio per raccogliere in modo collegato le componenti libere nell'array.

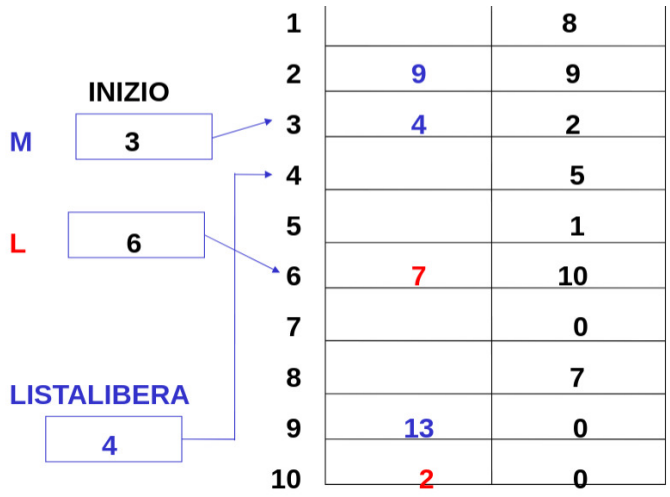


Figura 4.2: Realizzazione della lista libera

Quando si deve inserire un nuovo elemento nella lista, si preleva una nuova componente nella *listalibera* e la si utilizza per memorizzare il nuovo elemento, collegandolo in modo opportuno agli altri elementi della lista.
La *listalibera* rappresenta un serbatoio da cui prelevare componenti libere dell'array e in cui riversare le componenti dell'array che non sono più utilizzate per la lista.

Una possibile dichiarativa è la seguente:

- definizione di costanti:
  - maxlung = . . . /* costante intera positiva */
- definizione di tipi:
  - posizione intero in $0 \ldots$ maxlung
  - lista di tipo posizione
  - tipoeleme di tipo intero
  - componente tipo strutturato con componenti:
    - elemento di tipoelem;
    - successivo di tipo posizione.

- definizione di variabili:
  - listalibera: variabile di tipo lista;
  - spazio: array di maxlung elementi di tipo componente.

Le operazioni eseguibili sulla lista libera sono:

- inizializzazione che serve a collagare le varia componenti dell'array che inizialmente fanno parte di listalibera;
- il prelievo di una componente della listalibera se esiste;
- la restituzione di una componente alla listalibera.

```
1   inizializza_lista_libera()
2     listalibera = 1
3     for i = 1 to maxlung - 1 do
4       spazio[i].successivo = i + 1
5       spazio[maxlung].successivo = 0
```

È immediato verificare, prima di definire le operazioni di `inslista` e `canclista`, che con questa rappresentazione l'inserimento e la cancellazione di un elemento non richiedono lo spostamento di altri elementi nella lista grazie alla *listalibera* ma rimangono i problemi connessi all'uso dell'array cioè all'esigenza di definirne una dimensione.
Quindi, rispetto alla rappresentazione sequenziale l'aggiormaneto della lista non richiede lo spostamento di altri elementi, la complicazione della gestione della *listalibera* è compensata dal fatto che l'aggiornamento della lista richiede un'operazione limitato di operazioni e rispetto alla rappresentazione sequenziale c'è un'altra occupazione di memoria vista la necessità di memorizzare i riferimenti.

Spostamento È fondamentale disporre della procedura `sposta(h, k)` che trasferisce la cella puntata da h spostandola prima della cella puntata da k. La procedura aggiorna il cursore h in modo che punti alla cella che seguiva quella spostata e il cursore k in modo che punti alla cella spostata.

```
1   sposta(riferimento h, riferimento k)
2     temp = k
3     k = h
4     h = spazio[h].successivo
5     spazio[k].successivo = temp
6
```

Inserimento

```
inslista(tipoelem a, posizione p, lista l)
  if listalibera = 0 then error
  elseif p = 1 then
    sposta(listalibera, l)
    spazio[l].elemento = a
  else
    sposta(listalibera, spazio[p].successivo)
    spazio[spazio[p].successivo].elemento = a

```

Eliminazione

```
canclista(posizione p, lista l)
  if p = 1 then
    sposta(l, listalibera)
  else
    sposta(spazio[p].successivo, listalibera)

```

È evidente che la realizzazione con cursori è implementabile in tutti i linguaggi che dispongono di vettori e consentono l'astrazione funzionale mediante procedure o funzioni.

4.2.5 Realizzazione con puntatori

Un ulterioro modo per realizzare una lista è quella mediante l'uso congiunto del tipo puntatore e del tipo record e questa è la più efficace realizzazione della rappresentazione collegata. Una possibile realizzazione è quella di una lista monodirezionale semplificata. Vi èuna struttura di n elementi o celle, tale che l'i-esima cella contiene l'i-esimo elemento della lista e l'indirizzo della cella che contiene l'elemento successivo.

- La prima cella è indirizzata da una variabile l di tipo puntatore;
- l'ultima cella punta ad un valore convenzionale nil;
- gli indirizzi sono noti alla macchina ma non al programmatore;
- la posizione $pos(i)$ è uguale al valore del puntatore che contiene l'i-esimo elemento ove $1 \leq i \leq n$



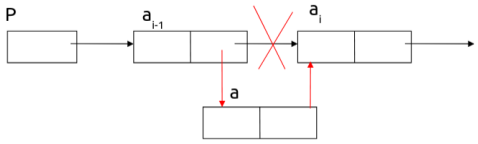
Figura 4.3: Realizzazione di una lista mediante puntatori

Per $n > 1$, $pos(i)$ è il puntatore alla cella contente l'i-esimo elemento, per $n = 0$ allora $L = NULL$. Una possibile dichiariazione per lista realizzata mediante puntatori è la seguente:

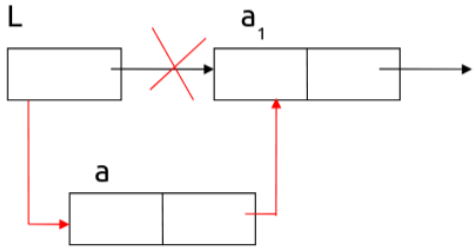
```
posizione: tipo puntatore a cella
cella: tipo strutturato con componenti
  elemento di tipo tipoelem
  successivo di tipo posizione
lista: alias per il tipo posizione

```

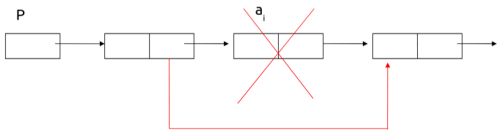
Aggiornamento dei puntatori
Vediamo la rappresentazione quando c'è un inserimento, ovvero quando `inslista` in $P = pos(i)$ con $i > 1$:



In $P = pos(1)$, invece:



Per `Canclista` in $P = pos(i)$:



Da queste figure notiamo una limitazione, non è possibile sapere la cella che precede la cella i-esima, per questo c'è un'altra realizzazione, cioè quella a *doppi puntatori* o *simmetrica*

4.2.6 Realizzazione a doppi puntatori

In questa realizzazione ogni elemento contiene, oltre al riferimento al nodo successivo, anche il riferimento al precedente. Eventualmente, la lista può essere espansa con una cella in più per ottenere una lista circolare:

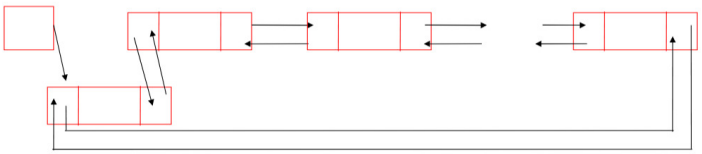


Figura 4.4: Realizzazione di una lista a doppi puntatori circolare

Con questa rappresentazione si ha il vantaggio di:

- poter scandire la lista in entrambe le direzioni;
- poter individuare facilmente l'elemento che precede;
- poter realizzare le operazioni di inserimento senza dover usare variabili aggiuntive.

Capitolo 5

Pile

Uno stack o pila è una sequenza di elementi di un certo tipo in cui è possibile aggiungere o togliere elementi soltanto da un estremo, la testa, della sequenza. Può essere vista come un caso speciale di lista in cui l'ultimo elemento inserito è anche il primo ad essere rimosso e non è possibile accedere ad alcun elemento che non sia quello in testa. Questo meccanismo di accesso è detto **LIFO**, Last In First Out.

5.1 Specifiche

Le operazioni per il tipo di dato stack comprendono l'operazione di creazione, CREAPILA, per inizializzare la pila alla sequenza vuota, quella di interrogazione PILAVUOTA, per verificare che la pila sia vuota oppure no, quella di lettura LEGGIPILA, per leggere il valore dell'elemento in testa alla pila, e quelle di modifica FUORIPILA, per eliminare l'elemento che si trova in testa alla pila, e INPILA, per aggiungere un elemento in testa alla pila. La specifica sintattica del tipo di dato è la seguente:

$$
\begin{aligned}
\text{CREAPILA:} &\quad () \to \text{pila} \\
\text{PILAVUOTA:} &\quad (\text{pila}) \to \text{booleano} \\
\text{LEGGIPILA:} &\quad (\text{pila}) \to \text{tipoelem} \\
\text{FUORIPILA:} &\quad (\text{pila}) \to \text{pila} \\
\text{INPILA:} &\quad (\text{tipoelem}, \text{pila}) \to \text{pila}
\end{aligned}
$$

Indicando col nome *pila* l'insieme delle sequenze di elementi di tipo *tipoelem* e con P una generica sequenza $a_1, a_2, \ldots, a_n$, la semantica della pila può essere specificata formalmente nel seguente modo:

- `CREAPILA() = P'`
  Post: P' = Λ, la sequenza vuota

- `PILAVUOTA(P) = b`
  Post: $\begin{cases} b = vero & P = \Lambda \\ b = falso & altrimenti \end{cases}$

- `LEGGIPILA(P) = a`
  Pre: $P = a_1, a_2, \ldots, a_n$ e $n \geq 1$
  Post: $a = a_1$

- `FUORIPILA(P) = P'`
  Pre: $P = a_1, a_2, \ldots, a_n$ e $n \geq 1$
  Post: $\begin{cases} P' = a_2, \ldots, a_n & n > 1 \\ P = \Lambda & n = 1 \end{cases}$

- `INPILA(a, P) = P'`
  Pre: $P = a_1, a_2, \ldots, a_n$ e $n \geq 0$
  Post: $\begin{cases} P' = a, a_1, a_2, \ldots, a_n & n > 0 \\ P' = a & n = 0 \end{cases}$

Poichè la pila è un caso particolare di lista, ogni realizzazione descritta per le liste funziona anche per le pile e per le code. Ad esempio, le operazioni per le pile possono essere simulate con quelle delle liste:

```
CREAPILA = CREALISTA
PILAVUOTA(P) = LISTAVUOTA(P)
LEGGIPILA(P) = LEGGILISTA(PRIMOLISTA(P), P)
FUORILISTA(P) = CANCLISTA(PRIMOLISTA(P), P)
INPILA(a, P) = INSLISTA(a, PRIMOLISTA(P), P)
```

5.2 Realizzazione con vettore

Una realizzazione dello stack con il tipo di dato pila consiste nel memorizzare gli *n* elementi della pila, in ordine inverso, nelle prime *n* posizioni di un vettore

di *maxlung* elementi, mantenendo un curosre alla testa della pila. La pila vuota viene individuata dal valore 0 contenuto nel cursore, mentra la pila satura, è individuata dal valore *maxlung*. FUORIPILA viene realizzata decrementando il valore del cursore, mentre INPILA è realizzata incrementando il valore del cursore ed inserendo il nuovo elemento nella posizione del vettore individuata dal cursore. La realizzazione dettagliata con procedure Pascal è la seguente:

```
type
  pila = record
    testa: 0..maxlung;
    elementi: array[1..maxlung] of tipoelem;
  end;

procedure CREAPILA(var P: pila);
  begin
    P.testa := 0
  end;

function PILAVUOTA(var P: pila): boolean;
  begin
    PILAVUOTA := (P.testa = 0)
  end;

function LEGGIPILA(var P: pila): tipoelem;
  begin
    if not PILAVUOTA(P) then
      LEGGIPILA := P.elementi[P.testa]
  end;

procedure FUORIPILA(var P: pila);
  begin
    if not PILAVUOTA(P) then
      P.testa := P.testa - 1
  end;

procedure INPILA(a: tipoelem, var P: pila);
  begin
    if P.testa = maxlung then
      { messaggio di errore: pila satura }
    else begin
      P.testa := P.testa + 1;
      P.elementi[P.testa] := a
    end
  end;

```

5.3 Pila e procedure ricorsive

Una applicazione importante delle pile consiste nella gestione dell'esecuzione di procedure ricorsive. Infatti, l'esecuzione di una procedura ricorsiva prevede il "salvataggio" dei dato sui quali lavora la procedure al momento di una nuova chiamata ricorsiva interna. Tali dati sono successivamente riesumati quando la computazione interna termina e viene ripresa la computazione che era stata sospesa.
Grazie alle pile è sempre possibile, dato un programma ricorsivo, trasformarlo in uno iterativo. I programmi ricorsivi corrispondono a metodi solutivi particolarmente adatti a problemi per i quali:

- la soluzione del problema di rango *n* è definibile in termini della soluzione del problema di rango inferiore a *n*;
- è definibile una soluzione per assioma sul problema di rango minimo (1 o 0).

Ad esempio, la successione di fibonacci è definita come una successione il cui $k-$esimo elemento è uguale alla somma dei due che lo precedono:

$$fib(1) = 1 \quad fib(2) = 1 \quad \text{livello assiomatico}$$
$$fib(k) = fib(k-1) + fib(k-2) \quad k > 2$$

La ricorsione può essere usata per formalizzare un'ampia classe di strutture dati che mostrano caratteristiche ricorsive nella struttura.
Vedere l'esempio delle torre di Hanoi.

Capitolo 6

Code

Una coda è una sequenza di elementi di un certo tipo, in cui è possibile aggiungere elementi ad un estremo, il fondo, e togliere elementi dall'altro estremo, la testa. Può essere considerata come una particolare lista in cui il primo elemento inserito è anche il primo ad essere rimosso e non è possibile accedere ad alcun elemento che non sia quello in testa o quello in fondo. Tale meccanismo di accesso è detto **FIFO**, First In First Out.

6.1 Specifiche

Le operazioni per il tipo di dato stack comprendono l'operazione di creazione, CREACODA, CODAVUOTA, LEGGICODA, FUORICODA, con la stessa specifica delle rispettiva operazioni della pila, e INCODA, per aggiungere un elemento in fondo alla coda. La specifica semantica di INCODA è ovviamente diversa da quella di INPILA.

INCODA(a, Q) = Q'

$$Pre : Q = a_1, a_2, \ldots, a_n \quad n \geq 0$$

$$Post : \begin{cases} Q' = a_1, a_2, \ldots, a_n & n > 0 \\ Q' = a_1, a_2, \ldots, a_n, a \end{cases}$$

Per simulare INCODA con operazioni delle liste, bisogna distinguere il caso di inserzione in una coda vuota da quello di inserzione in una coda non vuota. Nel primo caso la simulazione è identica a quella di INPILA, mentre nel secondo caso occorre inserire l'elemento nella posizione che segue quella dell'ultimo elemento

della lista:

```
INCODA(a, Q) = INSLISTA(a, SUCCLISTA(ULTIMOLISTA(Q), Q), Q)
```

6.2 Realizzazione di una coda con vettore circolare

Supponiamo di avere un array di *maxlung* elementi, indicati da 0 a *maxlung* - *1*, in cui l'elemento di indice 0 è considerato come successore di quello di indice *maxlung* - *1*. La coda è contenuta in *n* posizioni consecutive del vettore e i suoi elementi sono individuati da due cursori. Per cancellare o inserire un elemento, viene incrementato di uno (modulo *maxlung*) il rispettivo cursore di testa o di fondo. In questo modo la coda "migra" lungo il vettore circolare e, ad ogni istante, si trova da qualche parte nel vettore, sempre in posizioni consecutive secondo il verso orario. Per non utilizzare due cursori, per individuare la coda è sufficiente conoscere l'indice del primo elemento e la lunghezza della coda stessa. Infatti, se *i* è la posizione dell'elemento in testa alla coda, allora quello dell'ultimo elemento della coda è $(i + n - 1) \mod maxlung$.

```
type
  coda = record
    elementi: array[0..maxlung - 1] of tipoelem;
    testa: 0..maxlung - 1;
    lung: 0..maxlung;
  end;

procedure CREACODA(var Q: coda);
  begin
    Q.testa := 0;
    Q.lung := 0
  end;

function CODAVUOTA(var Q: coda): boolean;
  begin
    CODAVUOTA := (Q.lung = 0)
  end;

function LEGGICODA(var Q: coda): tipoelem;
  begin
    if not CODAVUOTA(Q) then
      LEGGICODA := Q.elementi[Q.testa]
```

```
23   end;
24
25 procedure FUORICODA(var Q: coda);
26   begin
27     if not CODAVUOTA(Q) then begin
28       Q.testa := (Q.testa + 1) mod maxlung;
29       Q.lung := Q.lung - 1
30     end
31   end;
32
33 procedure INCODA(a: tipoelem, var Q: coda);
34   begin
35     if Q.lung = maxlung then
36       { messaggio di errore: coda satura }
37     else begin
38       Q.elementi[(Q.testa + Q.lung) mod maxlung] := a;
39       Q.lung := Q.lung + 1
40   end;
```

6.3 Realizzazione con puntatori

Nella realizzazione con puntatori la coda è realizzata con n celle, la prima delle quali è indirizzata da un puntatore "testa" e l'ultima da un puntatore "fondo". La coda vuota è individuata dal valore nullo del puntatore di testa.

Capitolo 7

Insiemi

Un insieme è una collezione di elementi distinti, detti anche componenti o membri, dello stesso tipo. L'insieme è la struttura matematica fondamentale, e può essere descritto elencandone tutti gli elementi o stabilendo una proprietà che ne caratterizzi gli elementi. A differenza delle liste, gli elementi non sono caratterizzati da una posizione, nè sono ammesse più copie dello stesso elemento nel medesimo insieme.
Il numero di elementi che compongono un insieme A è detto cardinalità di A e si indica con $|A|$ oppure con $\#A$. Se tale numero è finito, allora l'insieme ha cardinalità finita, altrimenti ha cardinalità infinita. Un insieme che non contiene alcun elemento è detto vuoto, e si indica con $\emptyset$.
La relazione fondamentale per gli insiemi è l'appartenenza di un elemento x ad un insieme A, indicata formalmente con $x \in A$. Da questa si deriva l'inclusione tra due insiemi A e B, indicata con $A \subseteq B$, significa che ogni elemento x appartiene sia ad A che a B.
Se $A \subseteq B$ e $B \subseteq A$ allora $A = B$.
Le operazioni principali tra due insiemi sono l'unione, l'intersezione e la differenza:

- L'unione tra due insiemi, $A \cup B$, denota l'insieme contenente tutti gli elementi di A e tutti gli elementi di B;
- L'intersezione tra due insiemi, $A \cap B$, denota l'insieme contente tutti gli elementi che appartengono sia ad A che a B, quindi gli elementi in comune tra i due insiemi;

- La differenza tra due insiemi, $A - B$, contiene gli elementi che appartengono ad A ma non appartengono a B.

7.1 Specifica

Una specifica per il tipo di dato insieme è la seguente:

CREAINSIEME:	$() \rightarrow$ insieme
INSIEMEVUOTO:	$(\text{insieme}) \rightarrow$ booleano
APPARTIENE:	$(\text{tipoelem}, \text{insieme}) \rightarrow$ booleano
INSERISCI:	$(\text{tipoelem}, \text{insieme}) \rightarrow$ insieme
CANCELLA:	$(\text{tipoelem}, \text{insieme}) \rightarrow$ insieme
UNIONE:	$(\text{insieme}, \text{insieme}) \rightarrow$ insieme
INTERSEZIONE:	$(\text{insieme}, \text{insieme}) \rightarrow$ insieme
DIFFERENZA:	$(\text{insieme}, \text{insieme}) \rightarrow$ insieme

7.1.1 Specifica semantica

- `CREAINSIEME()` = A
  Post: $A = \emptyset$
- `INSIEMEVUOTO`(A) = `b`
  Post: $\begin{cases} b = vero & A = \\ b = falso & altrimenti \end{cases}$
- `APPARTIENE`(x, A) = `b`
  Post: $\begin{cases} b = vero & x \in A \\ b = falso & altrimenti \end{cases}$
- `INSERISCI`(x, A) = A'
  Post: $A' = A \cup \{x\}$
- `CANCELLA`(x, A) = A'
  Post: $A' = A - \{x\}$

- UNIONE(A, B) = C
  Post: $C = A \cup B$
- INTERSEZIONE(A, B) = C
  Post: $C = A \cap B$
- DIFFERENZA(A, B) = C
  Post: $C = A - B$

Gli operatori INSERISCI e CANCELLA, pur essendo rindondanti, in quanto ottenibili con gli operatori di UNIONE e DIFFERENZA, sono molto utilizzati e di solito compaiono in tutte le specifiche degli insiemi.

7.2 Realizzazione con vettore booleano

Si supponga di voler realizzare un insieme i cui elementi possono essere interi compresi tra 1 e N. Un insieme A può essere rappresentato con un vettore booleano di N bit, in cui il k-esimo bit è vero se $k \in A$ ed è falso se $k \notin A$. In questo modo è semplice verificare se un elemento k appartiene all'insieme, oppure inserire o cancellare un elemento di k: basta accedere direttamente al k-esimo bit del vettore. Anche le operazioni di unione, intersezione e differenza di due insiemi si realizzano facilmente con operazioni logiche. Per l'unione basta porre a vero tutti i valori che sono veri in almeno uno dei due vettori, per l'intersezione tutti quelli veri in entrami i vettori e per la differenza tutti quelli veri nel primo vettore e falsi nel secondo.

```
1   const N = ... { valore opportuno };
2   type tipoelem = 1..N;
3     insieme = packed array[1..N] of boolean;
4
5   procedure CREAINSIEME(var A: insieme);
6     var i: integer;
7     begin
8       for i := 1 to N do A[i] := false
9     end;
10
11  function INSIEMEVUOTO(var A: insieme): boolean;
12    var i: integer;
13    begin
14      i := 1;
15      while (i < N) and not A[i] do i := i + 1;
```

```
16        INSIEMEVUOTO = not A[i];
17      end;
18
19    function APPARTIENE(i: tipoelem, var A: insieme): boolean;
20      begin
21        APPARTIENE := A[i]
22      end;
23
24    procedure INSERISCI(i: tipoelem, var A: insieme);
25      begin
26        A[i] := true;
27      end;
28
29    procedure CANCELLA(i: tipoelem, var A: insieme);
30      begin
31        A[i] := false;
32      end;
33
34    procedure UNIONE(var A, B, C: insieme);
35      var i: integer;
36      begin
37        for i := 1 to N do C[i] := A[i] or B[i]
38      end;
39
40    procedure INTERSEZIONE(var A, B, C: insieme);
41      var i: integer;
42      begin
43        for i := 1 to N do C[i] := A[i] and B[i]
44      end;
45
46    procedure DIFFERENZA(var A, B, C: insieme);
47      var i: integer;
48      begin
49        for i := 1 to N do C[i] := A[i] and not B[i]
50      end;
51
```

Questa realizzazione, pur essendo estremamente semplice, presenta molti svantaggi. Innanzitutto vi è un notevole spreco di memoria, perchè ogni insieme richiede sempre $O(N)$ spazio per la sua memorizzazione, indipendentemente dal numero di elementi presenti nell'insieme. Inoltre, alcune operazioni sono troppo lente perchè richiedono $O(N)$ tempo, anche per insiemi con pochi elementi. Inoltre, la cardinalità non può superare la costante N.

7.3 Realizzazione con liste non ordinate

Si può rappresentare un insieme con una lista i cui elementi sono quelli dell'insieme. In questo modo, l'insieme non deve necessariemente essere composto da interi da 1 ad N. Inoltre, l'occupazione della memoria cresce linearmente col numero degli elementi effettivamente presenti nell'insieme.
Quindi, le operazioni di inserimento, cancellazione, presenza di un elemento all'interno della lista si riducono alle funzioni della lista già descritte in precedenza.
Le funzioni di UNIONE, INTERSEZIONE e DIFFERENZA sono abbastanza simili tra loro. Occorrono tre variabili di tipo posizione per scandire le liste che rappresentano i tre insiemi A, B e C. Per l'unione si inseriscono dapprima tutti gli elementi di B in C, mentre per l íntersezione e la differenza si parte con C vuoto. Successivamente, per ogni elemento di A, si verifica tramite la APPARTIENE se tale elemento compare anche in B oppure no e lo si inserisce in C nel primo caso (INTERSEZIONE) oppure no (UNIONE e DIFFERENZA).
Di seguito l'implementazione della procedura DIFFERENZA:

```
1   procedure DIFFERENZA(var A, B, C: insieme);
2     var p, q, r: posizione;
3     begin
4       CREAINSIEME(C);
5       p := PRIMOLISTA(A); q := PRIMOLISTA(B); r := PRIMOLISTA(C);
6       while not FINELISTA(p, A) do begin
7         if not APPARTIENE(LEGGILISTA(p, A), B)
8           then INSLISTA(LEGGILISTA(p, A), r, C);
9         p := SUCCLISTA(p, A);
10      end;
11    end;
12
```

7.4 Realizzazioni con liste ordinate

Se sugli elementi dell'insieme è definita una relazione "$\leq$" di ordinamento totale, l'insieme stesso può essere rappresentato con una lista ordinata per valori crescenti degli elementi.
Ad esempio, l'operatore di intersezione può essere scritto nel modo seguente. Si utilizzando ancora tre variabili di tipo posizione per scandire A, B e C ma grazie all'ordinamento si può evitare l'utilizzo di appartiene. Infatti, considerando il primo elemento di A e il primo di B, se sono uguali si inserisce in C uno dei due, altrimenti se sono diversi si inserisce in C il minore e ci si posizione sull'elemento

successivo solo nella lista del minore. Il procedimento continua finchè non viene esaurita almeno una delle due liste. Per mantenere C ordinata, l'inserimento in C va sempre effettuata in fondo.

```
procedure INTERSEZIONE(var A, B, C: insieme);
  var p, q, r: posizione;
  begin
    CREAINSIEME(C);
    p := PRIMOLISTA(A); q := PRIMOLISTA(B); r := PRIMOLISTA(C);
    while not FINELISTA(p, A) and not FINELISTA(q, B) do
      if LEGGILISTA(p, A) = LEGGILISTA(q, B) then
        begin
          INSLISTA(LEGGILISTA(p, A), r, C);
          p := SUCCLISTA(p, A);
          q := SUCCLISTA(q, B);
          r := SUCCLISTA(r, C);
        end
      else
        if LEGGILISTA(p, A) < LEGGILISTA(q, B)
          then p := SUCCLISTA(p, A)
        else
          q := SUCCLISTA(q, B)
  end;

```

Capitolo 8

Dizionari

Esistono delle applicazioni che, pur richiedendo una struttura dati del tipo insieme, non richiedono tutte le operazioni definite sull'insieme. Questi sottotipi del tipo insieme sono i *dizionari*.
Gli elementi sono generalmente tipi strutturati ai quali si accede per mezzo di un riferimento a un campo chiave. Gli elementi assumono la forma di una coppia costituita da (chiave, valore).
La caratteristica della chiave è legata all'applicazione, ad esempio nei dizionari degli elaboratori di testi la chiave individua una parola mentre in un dizionario di magazzino la chiave può un essere un codice pezzo.
Il valore associato, invece, rappresenta l'informazione associata per scopi di gestione o manutenzione. Ad esempio nelle tabelle di simboli, utili ai compilatori e ai linker, la chiave viene usata per l'identificatore, mentre nel valore vengono memorizzate le informazioni di gestione, come indirizzo della locazione, dimensione, posizioni nelle quali l'identificatore è usato nel programma ecc.

8.1 Operazioni

Le operazioni applicate ad un dizionario devono consentire la verifica dell'esistenza di una definita chiave e deve essere possibile l'inserimento di nuove coppie (chiave, valore), come pure la cancellazione.
Può essere utile anche il recupero delle informazioni presenti nell'attributo oppure la loro eventuale modifica.
Dato che i dizionari sono casi particolari di insieme, la specifica per i dizionari è identica a quella del tipo di dato insieme. Le operazioni ammesse sono quelle di

creazione, appartenenza, inserimento e cancellazione.
In alcuni casi troviamo anche operazioni come quello di recupero e aggiornamento.

8.2 Specifica sintattica

I tipi di dati, nella specifica sintattica, sono:

- dizionario, boolean, chiave, valore

Gli operatori sono:

$$
\begin{aligned}
\text{creadizionario:} &\quad () \rightarrow \text{dizionario} \\
\text{dizionariovuoto:} &\quad (\text{dizionario}) \rightarrow \text{booleano} \\
\text{appartiene:} &\quad (\text{chiave}, \text{dizionario}) \rightarrow \text{booleano} \\
\text{inserisci:} &\quad (\text{<chiave, valore>}, \text{dizionario}) \rightarrow \text{dizionario} \\
\text{cancella:} &\quad (\text{chiave}, \text{dizionario}) \rightarrow \text{dizionario} \\
\text{recupera:} &\quad (\text{chiave}, \text{dizionario}) \rightarrow \text{valore}
\end{aligned}
$$

8.3 Specifica semantica

I tipi di dati, nella specifica semantica, sono:

- dizionario = famiglia di dizionari costituita da coppie di tipo <chiave, valore>
- boolean= insieme di valori di verità

Gli operatori sono:

- `creadizionario() =` D
  Post: $D = \{\}$
- `dizionariovuoto(`D`) = b`
  Post: $\begin{cases} b = vero & D = \{\} \\ b = falso & altrimenti \end{cases}$

- `appartiene`(k, D) = b
  Post: $\begin{cases} b = vero & \exists < k', v > \in D \mid k' = k \\ b = falso & altrimenti \end{cases}$
- `inserisci`$(< k, v >, D) = D'$
  Post: $D' = D \cup \{< k, v >\} \iff \nexists < k', v > \mid k' = k$
- `cancella`$(k, D) = D'$
  Pre: $\exists < k', v > \in D \mid k' = k$ *Post:* $D' = D - \{< k', v >\}$
- `recupara`$(k, D) = v$
  Pre: $\exists < k', v' > \in D \mid k' = k$ *Post:* $v = v'$

Oltre alle realizzazioni viste per l'insieme, che si rifanno alla rappresentazione con vettore booleano e alla rappresentazione mediante una lista, ci sono realizzazioni più efficienti mediante vettori ordinati e tabelle hash.

8.4 Rappresentazione con vettore ordinato

Nella rappresentazione con vettore ordinato si utilizza un vettore con un cursore all'ultima posizione occupata. Avendo definito una relazione di ordinamento totale $\leq$ sulle chiavi, queste si memorizzano nel vettore in posizioni continue e in ordine crescente a partire dalla prima posizione. Per verificare l'appartenenza di un elemento o chiave k, si utilizza la ricerca binaria (dicotomica, logaritmica) ove si confronta il valore da ricercare k con il valore v che occupa la posizione centrale del vettore e si stabilisce in quale metà continuare la ricerca.

8.4.1 Ricerca binaria

Definiamo i seguenti tipi:

- dizionario: tipo strutturato con componenti
  - elementi: array di maxlung elementi di tipo tipoelem;
  - ultimo: intero in [0..maxlung]

```
bool appartiene(k: tipoelem, D: dizionario)
  return ricbin(D.elementi, k, 1, D.ultimo)

bool ricbin(V: vettore, k: tipoelem, i: integer, j: integer)
  if i > j then
    ricbin = false
  else
    m = (i + j) div 2
    if k = V[m] then
      ricbin = true
    else
      if k < V[m] then
        ricbin = ricbin(V, k, i, m - 1)
      else
        ricbin = ricbin(V, k, m + 1, j)

```

8.5 Rappresentazione con tabella hash

Esiste una tecnica denominata "hash", che si appoggia su di una struttura di dati tabellare, che si presta ad essere usata per realizzare dizionari. Con questa struttura, le operazioni di ricerca e di modifica di un dizionario possono operare in tempi costanti e indipendenti sia dalla dimensione del dizionario che dall'insieme dei valori che verranno gestiti.
Nella rappresentazione con tabella hash, l'idea di basi è ricavare la posizione che la chiave occupa in un vettore dal valore della chiave. Esistono diverse varianti che si possono far risalire ad una forma statica e ad una forma dinamica o estensibile.
La prima fa uso di strutture o tabelle di dimensione prefissata, mentre l'hash dinamico è in grado di modificare dinamicamente le dimensioni della tabella hash sulla base del numero di elementi che vengono via via inseriti o eliminati.
L'hash, con rappresentazione statica, può assumere a sua volta due forme diverse denominate rispettivamente:

- **hash chiuso**: consente di inserire un insieme limitato di valori in uno spazio di dimensione fissa;
- **hash aperto**: consente di memorizzare un insieme di valori di dimensione qualsiasi in uno spazio potenzialmente illimitato.

Entrambe queste varianti però utilizzano una sottostante tabella hash a dimensione fissa costituita da una struttura allocata sequenzialmente in memoria e che assume la forma di un array.
Nel caso di hash chiuso la struttura sarà composta da un certo numero, *maxbucket*, di contenitori di uguale dimensione denominati *bucket*.
Ognuno di questi contenitori può mantenere al proprio interno al massimo un numero $nb = 1$ di elementi che comprenderanno la chiave e il corrispondente valore (nel cao $nb = 1$ ogni bucket avrà una sola coppia <chiave, valore>).
Nel caso di hash aperto la struttura sarà composta da un certo numero indeterminato di contenitori bucket.
In entrambi i casi viene usata una funzione aritmetica allo scopo di calcolare, partendo dalla chiave, la posizione in tabella delle informazioni contenute nell'attributo collegato alla chiave. Questa è un'alternativa efficace all'indirizzamento diretto in un vettore perchè la dimensione è proporzionale al numero di chiavi attese.
Se k è l'insieme di tutte le possibili chiavi distinte e v è il vettore di dimensione m in cui si memorizza il dizionario, la soluzione ideale è la funzione di accesso $h : K \to \{1, \ldots, m\}$ che permette di ricavare la posizione $h(k)$ della chiave k nel vettore v così che, se $k_1 \in K$ e $k_2 \in K$ con $k_1 \neq k_2$ si ha $h(k_1) \neq h(k_2)$.
Utilizzando $m = |K|$ si ha garanzia di biunivocità e di poter accedere direttamente alla posizione contenente la chiave. Se $|K|$ è grande, si ha un enorme spreco di memoria e la dimensione, m, del vettore delle chiavi va scelta in base al numero delle chiavi attese.

8.5.1 Collisioni

Consideriamo un esempio di hash chiuso:

```
albinoni        h(albinoni) = 1
offenbach       h(offenbach) = 15
palestrina      h(palestrina) = 16
puccini         h(puccini) = 16
prokofev        h(prokofev) = 16
rossini         h(rossini) = 18
```

ALBINONI	**1**
...	
OFFENBACH	**15**
PALESTRINA	**16**
PUCCINI	**17**
PROKOFEV	**18**
ROSSINI	**19**
...	
	26

Ci sono collisioni per i cognomi che iniziano con la stessa lettera.
Una collisione si verifica quando chiavi diverse producono lo stesso risultato della funzione. Esistono funzioni hash più o meno buone anche se le collisioni non si protranno mai evitare del tutto.
Nell'esempio visto si è adottata una semplice strategia per la risoluzione delle collisioni, chiamata **scansione lineare**: ovvero se $h(k)$ per qualunque chiave k indica una posizione già occupata, si ispeziona la posizione successiva nel vettore. Se la posizione è piena si prova con la seguente e così via fino a trovare una posizione libera o trovare che la tabella è completamente piena.
Una posizione libera può venire facilmente segnalata in fase di realizzazione da una chiave fittiza, chiamata *libero*. Per la cancellazione è più semplice sostituire l'oggetto cancellato con una chiave fittizia *cancellato* che dovrebbe essere facilmente distinguibile dalle altre chiavi reali e dall'altra chiave fittizia *libero*.
La strategia lineare può produrre nel tempo il casuale addensamento di informazioni in certi tratti della tabella, i cossidetti **agglomerati**, piuttosto che la loro dispersione.
Indipendentemente dalla funzione hash adottata, deve essere prevista una strategia per gestire il problema degli agglomerati e delle collisioni. In definitiva:

- occorre una funzione hash, calcolabile velocemente e che distribuisca le chiavi uniformemente in *v*, in modo da ridurre le collisioni;
- occorre un metodo di scansione per la soluzione delle collisioni utile a reperire chiavi che hanno trovato la posizione occupata e che non provochi la formazione di agglomerati di chiavi;
- la dimensione *m* del vettore *v* deve essere una sovrastima del numero delle chiavi attese, per evitare di riempire *v* completamente.

Per definire funzioni hash, è conveniente considerare la rappresentazione binaria $bin(k)$ della chiave *k*. Nel caso di chiavi non numeriche si può considerare la rappresentazione binaria dei caratteri e $bin(k)$ è data dalla loro concatenazione.
Di seguito sono elencati 4 buoni metodi di generazione hash. Denotiamo con $int(b)$ il numero intero rappresentato da una stringa binaria b e indichiamo da 0 a $m-1$ gli elementi di v. I primi tre pressunpono $m = 2^p$, per qualche p, mentre per il quarto si presuppone m dispari (meglio se primo).

1. $h(k) = int(b)$, dove b è un sottoinsieme di p bit di $bin(k)$, solitamente estratti nelle posizioni centrali;
2. $h(k) = int(b)$, dove b è data dalla somma modulo 3, effettuata bit a bit, di diversi sottoinsiemi di p bit di $bin(k)$;
3. $h(k) = int(b)$, dove b è un sottoinsieme di p bit estratti dalle posizioni centrali di $bin(int(bin(k))2)$;
4. $h(k)$ è uguale al resto della divisione $^{int(bin(k))}/_{m}$. Se $m = 2^p$, due numeri con gli stessi p bit finali darebbero sempre luogo ad una collisione.

L'ultima funzione hash definita è la migliore dal punto di vista probabilistico e fornisce un'eccellente distribuzione degli indirizzi $h(k)$ nell'intervallo $[0, m-1]$.

Esempio : Funzione hash

Si supponga che le chiavi siano di 6 caratteri alfanumerici, le chiavi più lunghe vengono troncate mentre quelle più corte sono espanse a destra con spazi. Si assuma $ord(A) = 1, ord(B) = 2, \ldots, ord(Z) = 26$ e $ord(\underline{b}) = 32$, dove b indica lo spazio, con rappresentazione di ogni ordinale su 6 bit.
Per le chiavi *weber* e *webern* si ottiene:

- $bin(weberb) = 010111\ 000101\ 000010\ 000101\ 010010\ 100000\ 0000$
- $bin(webern) = 010111\ 000101\ 000010\ 000101\ 010010\ 001110\ 0000$

dove si sono evidenziati per chiarezza i gruppi di 6 bit che rappresentano ciascun carattere. Calcoliamo gli indirizzi hash ottenuti con le funzioni definite in precendeza, la 1, la 2 e la 4:

- sia $m = 256 = 2^8$. Estraendo gli 8 bit dalla posizione 15 alla 22 di $bin(k), h(weber) = h(webern) = int(00100001) = 33$, le due chiavi danno luogo ad una collisione;
- sia ancora $m = 256 = 2^8$ e si calcoli b raggruppando $bin(k)$ in cinque gruppi di 8 bit. $h(weber) = int(11000011) = 195$, poichè:

  $11000011 = 01011100 \oplus 01010000 \oplus 10000101 \oplus 01001010 \oplus 00000000$

  dove + indicala somma bit a bit modulo 2. Analogamente, $h(webern) = int(00100001) = 33$, poichè:

  $00100001 = 01011100 \oplus 01010000 \oplus 10000101 \oplus 01001000 \oplus 11100000$

  La collisione è stata, quindi, eliminata.

- Sia $m = 383$, utilizzando 6 bit, $int(bin(k))$ è un numero in base $2^6 = 64$.
  $int(bin(webern)) = 23 * 645 + 5 * 644 + 2 * 643 + 5 * 642 + 18 * 641 + 14 * 640 = 64(64(64(64(23 * 64) + 5) + 2) + 5) + 18) + 14$
  $h(webern)$ e $h(weber)$ sono ottenuti prendendo il resto della divisione $int(bin(\dots))/383$.

Hash aperto Una tecnica che evita la formazione di agglomerati è quella dell'hash aperto che richiede che la tabella hash mantenga la lista degli elementi la cui chiavi producono lo stesso valore di funzione (trasformata). La tabella hash viene realizzata definendo un array di liste di bucket, le **liste di trabocco**.
La funzione hash viene utilizzata per determinare quale lista potrebbe contenere l'elemento che possiede una determinata chiave in modo da poter attivare una successiva operazione di ricerca nella lista corrispondente e da restituire la posizione del bucket che contiene la chiave.
Con le liste di trabocco, il vettore v contiene in ogni posizione un puntatore ad una lista:

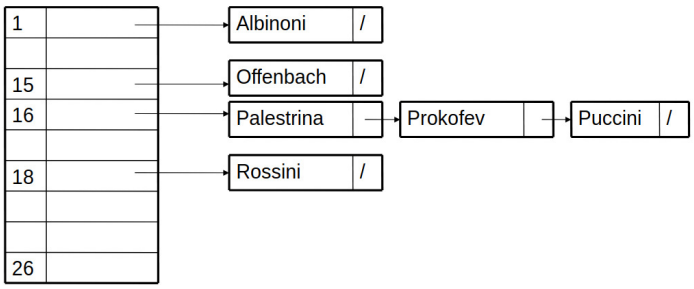


Figura 8.1: Lista di trabocco

8.5.2 Metodi di scansione

Nella realizzazione con hash e liste di trabocco si è usato un metodo di scansione esterno contrapposto alla scansione lineare che è un metodo di scansione interno. Altri metodi interni, o chiusi, sono:

- scansione quadratica;
- scansione pseudocasuale;
- hashing doppio.

Scansione interna Sia fi la funzione che viene utilizzata l'i-esima volta che si trova occupata una posizione del vettore *v*, $i \geq 0$. Per $i = 0, f0 = h$. fi va scelta in modo da toccare, una sola volta, tutte le posizioni di *v*.

Scansione lineare Nella scansione lineare la funzione fi è definita come $fi = (h(k) + h \cdot i) \mod m$ ove h è un intero positivo primo con *m* e *h* rappresenta la distanza tra due posizioni successive esaminate nella scansione. Dato che *m* e *h* sono primi tra loro, vengono esaminate tutte le posizioni di *v* prima di riconsiderare le posizioni già esaminate. Lo **svantaggio** è che non riduce la formazione di agglomerati.

Scansione quadratica Nella scansione quadratica la funzione fi è definita come $fi = (h(k) + h \cdot i + i(i-1)/2) \mod m$ ove m è primo. La distanza tra due posizioni successive nella sequenza è variabile, quindi la possibilità di agglomerati è ridotta. Lo **svantaggio** è che la sequenza di scansione non include tutte le posizioni di *v*, che è un vantaggio trascurabile per m non troppo piccolo.

Scansione pseudocasuale Nella scansione pseudocasuale la funzione fi è definita come $fi = (h(k) + pi) \mod m$ ove pi è l'i-esimo numero generato da un generatore di numeri pseudocasuali, che genera gli interi tra 1 e m una sola volta in un ordine qualunque.

Hashing doppio Nell'hashing doppio la funzione fi è definita come $fi = (h(k) + i \cdot f(k)) \mod m$ ove m è un'altra funzione hash diversa da h.

Usando metodi di scansione interna e potendo cancellare chiavi, non si è mai sicuri che, raggiunta una posizione vuota nella ricerca di k, tale chiave non si trovi in un'altra posizione di v, poichè la posizione ora vuota era occupata quando k è stata inserita.
Bisogna dunque scandire anche le posizioni in cui si è cancellato e fermarsi o sulla posizione mai riempita o essere tornati su una posizione già scandita e questo determina un aumento del tempo di ricerca.
Se sono previste molte cancellazioni è bene utilizzare il metodo di scansione esterna.

Capitolo 9

Alberi

Un albero ordinato è formato da un insieme finito di elementi, detti nodi. Se tale insieme non è vuoto, allora un particolare nodo è designato come radice, ed i rimanenti nodi, se esistono, sono a loro volta partizionati in insiemi ordinati disgiunti, ciascuno dei quali è un albero ordinato. L'albero ordinato, comunemente chiamato "albero", è una struttura informativa fondamentale, che si presta a rappresentare svariate situazioni, quali:

1. partizioni successive di un insieme in sottinsiemi disgiunti;
2. organizzazioni gerarchiche di dati;
3. procedimenti enumerativi o decisionali.

Sia T un albero ordinato di n nodi di radice r. Siano $T_1, \ldots, T_k$ gli insiemi disgiunti non vuoti in cui sono partizionati i rimanenti $n-1$ nodi di T, aventi come radici, rispettivamente, i nodi $r_1, \ldots, r_k$. Ciascun T_i è detto sottoalbero di T radicato in r_i, mentra ciascun r_i è detto figlio di r. I nodi $r_1, \ldots, r_k$ sono tra loro fratelli, ed r è il loro padre. Un discendente di un nodo u è un figlio di u oppure un figlio di un discendente di u; analogamente, un antenato di u è suo padre oppure il padre di un suo antenato. Un nodo senza figli è detto fogia, mentre la radice dell'albero è l'unico nodo senza padre. In pratica, in un albero ordinato è stabilito un doppio ordinamento tra i nodi. In primo luogo c'è un ordinamento "verticale" per livelli, cioè in base alla distanza dei nodi dalla radice. La radice r è a livello 0, tutti i figli di r stanno al livello 1, i figli dei figli di r stanno al livello 2, e così via fino all'esaurimento dei nodi. In secondo luogo, è stabilito un ordinamento "orizzontale" tra i nodi che stanno allo stesso livello, distinguendo

così tra il primo figlio di un nodo, il suo secondo figlio, e così via fino all'ultimo.
Un **cammino** di un albero è la sequenza di linee che unisce due nodi, uno dei quali è antenato dell'altro; la **lunghezza** di un cammino è costituita dal numero di linee che lo compongono.
L'**altezza** di un nodo è la lunghezza del cammino più lungo da quel nodo ad una foglia.
La **profondità** di un nodo è la lunghezza del cammino dalla radice a quel nodo.
In un albero valgono le seguenti proprietà:

- Un albero è un **grafo aciclico**, in cui per ogni nodo c'è solo un arco entrante, tranne che per la radice che non ne ha nessuno.
- Un albero è un **grafo debolmente connesso**.
- Se esiste un cammino che va da un nodo u ad un altro nodo v, tale cammino è unico.
- In un albero esiste un solo cammino che va dala radice a qualunque altro nodo.
- Tutti i nodi di un albero T, tranne r, possono essere ripartiti in insiemi disgiunti ciascuno dei quali individua un albero.

Un albero è un particolare tipo di grafo defintio matematicamente con una coppia:

$$T = (N, A)$$

dove N è un insieme finito di noid ed A è un insieme di coppie non ordinate tali che:

- il numero di archi è uguale al numero di nodi meno 1 $|A| = |N| - 1$
- T è connesso, ovvero per ogni coppia di nodi u e v in N, esiste una sequenza di nodi distinti $u_0, u_1, \ldots, u_k$ tali che $u = u_0$ e $v = u_k$ e a coppia $< u_i, u_{i+1} >$ è un arco di A, per $i = 0, 1, \ldots, k - 1$.

Un albero radicato è ottenuto da un albero libero designando aribitriaramente un nodo r come radice e e ordinando i nodi per livelli.

9.1 Specifica

L'albero ordinato può essere interpretato come una generalizzazione di una sequenza lineare, in cui ciascun elemento ha ancora un solo predecessore (il padre), ma in generale può avere più di un successore (i figli). Infatti, una lista è un particolare albero in cui ogni nodo ha al più un figlio e c'è una sola foglia. Analogamente alle operazioni sulle liste, quelle definite sugli alberi ordinati prevedono la possibilità di spostarsi da un nodo ad un suo parente, come figlio, fratello o padre, e inoltre di inserire o cancellare interi sottoalberi.
Un tipico insieme di operazioni per gli alberi ordinati comprende l'operazione di creazione CREAALBERO, quelle di selezione RADICE, per accedere direttamente alla radice, e PADRE, PRIMOFIGLIO e SUCCFRATELLO, per accedere, rispettivamente, al padre, al primo figlio, o al fratello sucessivo di un nodo, quelle di interrogazione ALBEROVUOTO, per verificare che l'albero sia vuoto, FOGLIA e FINEFRATELLI, per controllare che un nodo non abbia figli o che siano stati considerati tutti i fratelli, e quelle di modifica INSRADICE, per inserire la radice in un albero vuoto, CANCSOTTOALBERO e INSSOTTOALBERO, per cancellare o inserire un sottoalbero.

CREAALBERO:	$() \rightarrow$ albero
ALBEROVUOTO:	$(albero) \rightarrow$ booleano
INSRADICE:	$(nodo, albero) \rightarrow$ albero
RADICE:	$(albero) \rightarrow$ nodo
PADRE:	$(nodo, albero) \rightarrow$ nodo
FOGLIA:	$(nodo, albero) \rightarrow$ booelano
PRIMOFIGLIO:	$(nodo, albero) \rightarrow$ nodo
FINEFRATELLI:	$(nodo, albero) \rightarrow$ booleano
SUCCFRATELLO:	$(nodo, albero) \rightarrow$ nodo
INSSOTTOALBERO:	$(nodo, albero, albero) \rightarrow$ albero
INSPRIMOSOTTOALBERO:	$(nodo, albero, albero) \rightarrow$ albero
CANCSOTTOALBERO:	$(nodo, albero) \rightarrow$ albero

Indicando per semplicità con il termine "albero" l'insieme di tutti gli alberi ordinati, come definiti precedentemente, e con T un generico albero ordinato, la specifica semantica è la seguente.

- CREAALBERO() = T'
  Post: $T' = \Lambda$

- ALBEROVUOTO(T) = b
  Post: $\begin{cases} b = vero & T = \Lambda \\ b = falso & altrimenti \end{cases}$

- INSRADICE(u, T) = T'
  Pre: $T = \Lambda$
  Post: T' è formato dalla sola radice u

- RADICE(T) = u
  Pre: $T \neq \Lambda$ *Post:* u è la radice di T

- FOGLIA(u, T) = b
  Pre: $T \neq \Lambda$, u è un nodo di T
  Post: $\begin{cases} b = vero & \text{se non esiste alcun figlio di } u \\ b = falso & \text{altrimenti} \end{cases}$

- PRIMOFIGLIO(u, T) = v
  Pre: $T \neq \Lambda$, u è un nodo di T, FOGLIA$(u, T) = falso$
  Post: v è il primo figlio di u secondo la relazione precedente stabilita tra i figli di u.

- FINEFRATELLI(u, T) = b
  Pre: $T \neq \Lambda$, u è un nodo di T o un nodo "sentinella" s che segue l'ultimo fratello di ciascun nodo
  Post: $\begin{cases} b = vero & u = s \\ b = falso & \text{altrimenti} \end{cases}$

- SUCCFRATELLO(u, T) = v
  Pre: $T \neq \Lambda$, u è un nodo di T
  Post: v è il fratello di u che lo segue immediatamente u nella relazione di precedenza (e quindi v è la sentinella se u è l'ultimo fratello)

- INSOTTOALBERO(u, v, T, U) = T'
  Pre: $T \neq \Lambda, U \neq \Lambda$, u è un nodo di T non è radice di T. *Post*: T' è ottenuto da T aggiungendovi U come sottoalbero: la radice z di U diventa il nuovo fratello che segue immediatamente u nella relazione di precedenza.

- INSPRIMOSOTTOALBERO(u, T, U) = T'
  Pre: $T \neq \Lambda, U \neq \Lambda$, u è un nodo di T. *Post*: T' è ottenuto da T aggiungendovi U come sottoalbero la cui radice r' è il nuovo primofiglio di u.

- CANCSOTTOALBERO(u, T) = T'
  Pre: $T \neq \Lambda$, u è un nodo di T
  Post: T' è ottenuto da T togliendovi il sottoalbero di radice u, cioè u stesso e tutti i discendenti di u (e quindi $T' = \Lambda$ se u è la radice di T)

9.2 Visite

La visita di un albero ordinato consiste nel seguire una "rotta" di viaggio che consenta di esaminare ogni nodo dell'albero esattamente una volta. La visita può essere eseguita in vari modi, detti "ordini", tra cui i 3 più importanti sono: ordine anticipato o *previsita*, differito o *postivisita*, e simmetrico o *invisita*.
Sia T un albero non vuoto di radice r. Se r non è una foglia ma ha $k > 0$ figli, indichiamo con $T_1, \ldots, T_k$ i k sottoalberi di T radicati nei k figli di r. I predetti ordini di visita sono definiti ricorsivamente come segue:

1. la previsita di T consiste nell'esaminare r e poi nell'effettuare, nell'ordine, la previsita di $T_1, \ldots, T_k$;

2. la postvisita di T consiste nell'effettuare, nell'ordine, la postvisita di $T_1, \ldots, T_k$ e poi nell'esaminare r;

3. Fissato $i \geq 1$, la invisita di T consiste nell'effettuare, nell'ordine, la invisita di $T_1, \ldots, T_1$, nell'esaminare r, e poi nell'effettuare, nell'ordine, la invisita di $T_{i+1}, \ldots, T_k$.

Il seguente schema di proceudra Pascal serve per effettuare sia la visita anticipata che quella differita.

```
1     procedure VISITA(u: nodo; var T: albero);
2       var v: nodo;
3       begin
4 (1)       {esame ''anticipato'' del nodo u};
5         if not (FOGLIA(u, T)) then begin
6           v := PRIMOFIGLIO(u, T);
7           while not (FINEFRATELLI(v, T)) do begin
8             PREVISITA(v, T);
```

```
9             v := SUCCFRATELLO(v, T)
10          end
11        end;
12 (2)     {esame ‘‘differito’’ del nodo u};
13      end;
14
```

La chiamata nel programma principale è:

```
1     if not ALBEROVUOTO(T) then VISITA(RADICE(T), T)
2
```

Se l'esame del nodo u viene effettuato solo nella riga (1), ignorando la riga (2), si ottiene la procedura PREVISITA, mentre se viene effettuato solo nella riga (2), ignorando la riga (1), allora si ottiene la procedura POSTVISITA. La procedura INVISITA invece segue uno schema diverso. Per $i = 1$ lo schema è il seguente:

```
1     procedure INVISITA(u: nodo; var T: albero);
2       var v: nodo;
3       begin
4         if FOGLIA(u, T) then
5           {esamina il nodo u}
6         else begin
7           v := PRIMOFIGLIO(u, T);
8           INVISITA(v, T);
9           {esame ‘‘simmetrico’’ del nodo u};
10          v := SUCCFRATELLO(v, T);
11          while not (FINEFRATELLI(v, T)) do begin
12            INVISITA(v, T);
13            v := SUCCFRATELLO(v, T)
14          end
15        end
16      end;
17
```

9.3 Realizzazioni

9.3.1 Realizzazione con vettore dei padri

Si supponga che i nodi dell'albero T siano numerati da 1 ad n. La più semplice realizzazione fa uso di un vettore contentenente, per ogni nodo i, $1 \leq i \leq n$, il cursore al padre: $T[i] = j$, se j è il padre di i; $T[i] = 0$, se i è la radice.
Con questa realizzazione è facile visitare i nodi lungo percorsi, che vadano dal basso verso l'alto, cioè dalle foglie verso la radice. Viceversa, è costoso passare

da un nodo ai suoi figli, individuare il livello di un nodo, e inserire e cancellare sottoalberi. Infine non è chiara la relazione di precedenza tra fratelli, a meno che non se ne assuma una implicitamente, per esempio imponendo che un figlio abbia sempre un numero maggiore del padre e che nodi fratelli siano numerati in modo crescente da sinistra verso destra.

9.3.2 Realizzazione con liste di figli

Si supponga ancora che i nodi dell'albero T siano numerati da 1 ad n. Una realizzazione talvolta usata consiste nel mantenere un vettore contenente, per nodo i, $1 \leq i \leq n$, l'identificatore di una lista di tutti i figli del nodo i. Trascurando la realizzazione del tipo di dato lista, si hanno le seguenti dichiarazioni:

```
type
  nodo = integer;
  albero = record
    testa: array[1..maxlung] of lista;
    radice: nodo
  end;

```

Gli operatori degli alberi possono essere realizzati in termini degli operatori delle liste. Per esempio,

```
PRIMOFIGLIO = LEGGILISTA(PRIMOLISTA(T.testa[u]), T.testa[u])
```

Ovviamente, la realizzazione con liste dei figli presenta il vantaggio di poter scorrere velocemetne tutti i figli di un dato nodo, ma non permette di realizzare efficientemente altri operatori come PADRE o SUCCFRATELLO. Infatti, per poter individuare il padre o il fratello successivo di un dato nodo u, occorre scandire nel caso pessimo tutte le n liste alla ricerca dell'unica lista nella quale compare il nodo u.

```
  function PADRE(u: nodo; var T: albero): nodo;
    var i: nodo;
    begin
      i := 1;
      while (not APPARTIENE(u, T.testa[u]) and (i <= n)) do i := i + 1
      PADRE := i
    end;

  function APPARTIENE(u: nodo; var L: lista): boolean;
    var p: posizione; trovato: boolean;
```

```
11        begin
12          p := PRIMOLISTA(L); trovato := false;
13          while (not trovato) and (not FINELISTA(L)) do
14            if LEGGILISTA(p, L) = u THEN trovato := true;
15          APPARTIENE := trovato
16        end;
17
```

Per realizzare SUCCFRATELLO, occorre essenzialmente ripetere le stesse operazioni effettuate per PADRE, modificando la funzione APPARTIENE in modo da resstituire il nodo che segue u nella lista L.

9.3.3 Realizzazione con puntatori padre/primofiglio/fratello

La realizzazione con liste dei figli non si presta bene ad eseguire operazioni come PADRE, SUCCFRATELLO o INSSOTTOALBERO. Per realizzare PADRE e SUCCFRATELLO si potrebbero introdurre due campi con questi valori in ciascuna delle celle liste, oppure usare, oltre al vettore testa, anche un vettore dei padri e un vettore dei fratelli successivi. Ciò comunque non renderebbe efficiente la INSSOTTOALBERO. Per effettuare un'inserzione efficientemente, occorre che tutti gli alberi condividano un'area comune, così come le liste condividono la memoria indirizzabile dei puntatori nella realizzazione con puntatori o il vettore SPAZIO nella realizzazione con cursori. Inoltre, è necessario che tutte le celle dell'area comune abbiano la stessa struttura dove siano immediatamente reperibili le informazioni per raggiungere il padre, il primo figlio, e il fratello successivo di un nodo. Quest scopo è raggiunto imponendo che ogni cella contenga almeno tre puntatori: uno al padre, uno al primo figlio e uno al fratello successivo. In tal caso, il nodo diventa un puntatore nell'area comune e non può più essere identificato col valore del nodo stesso, e in ogni cella occorre un campo apposito per il valore del nodo.
A tal fine, si introducono due ulteriori operazioni, una di lettura LEGGINODO, per leggere il valore contenuto in un nodo, e una di modifica SCRIVINODO, per cambiare il valore memorizzato in un nodo, la cui specifica è la seguente.

$$\begin{aligned} \text{LEGGINODO}: &\quad (nodo, albero) \rightarrow \text{tipoelem} \\ \text{SCRIVINODO}: &\quad (tipoelem, nodo, albero) \rightarrow \text{albero} \end{aligned}$$

- LEGGINODO(u, T) = a
  Pre: $T \neq \Lambda$, u è un nodo di T
  Post: a è il valore memorizzato nel nodo u

- SCRIVINODO(a, u, T) = T'
  Pre: $T \neq \Lambda$, u è un nodo di T
  Post: T' è ottenuto scrivendo il valore di a nel nodo u

Nella realizzazione con puntatori, le dichiarazioni sono:

```
albero = ↑cella;
nodo = albero;
cella = record
  valore: tipoelem;
  genitore, figlio, fratello: nodo;

```

9.4 Alberi binari

Un albero binario è un particolare albero ordinato in cui ogni nodo ha al più due figli e si fa distinzione tra il figlio sinistro e il figlio destro di un nodo. Questa proprietà è sottile, poichè impone che due alberi T e U aventi gli stessi nodi, gli stessi figli per ogni nodo e la stessa radice, siano distinti qualora un nodo u sia designato come figlio sinistro di un nodo v in T e come figlio destro del medesimo nodo in U.
L'importanza degli alberi binari risiede sia sulla semplicità della loro struttura che sulle innumerevoli applicazioni in cui possono essere impiegati. Una specifica ragionevole per gli alberi binari può includere gli operatori CREABINALBERO, BINALBEROVUOTO, BINRADICE, BINPADRE, CANCSOTTOALBERO, LEGGINODO, SCRIVINODO che sono analoghi ai rispettivi operatori per gli alberi ordinati ed i nuovi operatori:

FIGLIOSINISTRO :	(nodo, binalbero) $\rightarrow$ nodo
FIGLIODESTRO:	(nodo, binalbero) $\rightarrow$ nodo
SINISTROVUOTO:	(nodo, binalbero) $\rightarrow$ booleano
DESTROVUOTO:	(nodo, binalbero) $\rightarrow$ booleano
COSTRBINALBERO:	(binalbero, binalbero) $\rightarrow$ binalbero
CANCSOTTOBINALBERO:	(nodo, binalbero) $\rightarrow$ binalbero

la cui specifica semantica è:

- FIGLIOSINISTRO(u, T) = v
  Pre: $T \neq \Lambda$, u è in T, u ha un figlio sinistro
  Post: v è il figlio sinistro di u in T.

- FIGLIODESTRO(u, T) = v
  Pre: $T \neq \Lambda$, u è in T, u ha un figlio destro
  Post: v è il figlio destro di u in T.

- SINISTROVUOTO(u, T) = b
  Pre: $T \neq \Lambda$, u è in T
  Post: $\begin{cases} b = vero & \text{se } u \text{ non ha figlio sinistro} \\ b = falso & altrimenti \end{cases}$

- DESTROVUOTO(u, T) = b
  Pre: $T \neq \Lambda$, u è in T
  Post: $\begin{cases} b = vero & \text{se } u \text{ non ha figlio destro} \\ b = falso & altrimenti \end{cases}$

- COSTRBINALBERO(T, U) = T'
  Post: T' è ottenuto introducendo un nuovo nodo u, che è la radice di T', che ha come sottoalbero sinistro T e come sottoalbero destro U (se $T = \Lambda$, u non ha figlio sinistro; se $U = \Lambda$, u non ha figlio destro)

- CANCSOTTOBINALBERO(u, T) = T'
  Pre: $T \neq \Lambda$, u è un nodo di T.
  Post: T' è ottenuto da T eliminando il sottoalbero di radice u, con tutti i suoi discendenti.

Si noti che l'operatore FOGLIA è ridondante, poichè FOGLIA$((u, T))$ = vero se e solo se SINISTROVUOTO(u, T) = DESTROVUOTO(u, T) = vero.
L'algebra appena presentata rappresenta una scelta precisa di progetto, si è scelto di enfatizzare la natura ricorsiva degli alberi e di costruire l'albero binario dal basso verso l'alto, cioè dal livello delle foglie verso la radice.
Non sempre questa scelta è opportuna: soprattutto se l'albero è usato per rappresentare un processo decisionale è preferibile un'algebra che preveda di costruire l'albero dall'alto verso il basso, inserendo prima la radice e poi i nodi figli via via. In tal caso, mentre rimangono validi tutti gli operatori definiti, andrebbe sostituito l'operatore di costruzione con tre operatori nuovi, uno dedicato all'inserimento della radice e gli altri due dedicati all'inserimento del figlio sinistro e del

figlio destro.

INSBINRADICE :	(nodo, alberobin) $\rightarrow$ alberobin
INSFIGLIOSINISTRO :	(nodo, alberobin) $\rightarrow$ alberobin
INSFIGLIODESTRO :	(nodo, alberobin) $\rightarrow$ alberobin

Con la relativa specifica semantica:

- INSBINRADICE(u, T) = T'
  Pre: $T = \Lambda$
  Post: T' è l'albero costituito dalla sola radice u.

- INSFIGLIOSINISTRO(u, T) = T'
  Pre: $T = \Lambda$, SINISTROVUOTO$(u, T) = vero$
  Post: T' è l'albero ottenuto da T aggiungendo un nodo v come figlio sinistro di u.

- INSFIGLIODESTRO(u, T) = T'
  Pre: $T = \Lambda$, DESTROVUOTO$(u, T) = vero$
  Post: T' è l'albero ottenuto da T aggiungendo un nodo v come figlio destro di u.

Utilizzando gli operatori introdotti, la visita di un albero binario può essere effettuata molto semplicemente nel modo seguente:

```
1      begin
2  (1)   {esame ‘‘anticipato’’ del nodo u};
3        if not SINISTROVUOTO(um T) then
4          BINVISITA(FIGLIOSINISTRO(um T), T);
5  (2)   {esame ‘‘simmetrico’’ del nodo u};
6      if not DESTROVUOTO(u, T) then
7        BINVISITA(FIGLIODESTRO(u, T), T);
8  (3)   {esame ‘‘differito’’ del nodo u}
9     end;
10
```

La previsita si ottiene esaminando il nodo u soltanto nella riga (1) e ignorando le righe (2) e (3). Analogamente, la invisita o la postvisita si hanno esaminando il nodo u solo nella riga (2) o (3), rispettivamente.‘
Una realizzazione ragionevole fa uso di un'area comune AREALIBERA, come nel caso degli alberi ordinati, in cui i campi "figlio" e "fratello" sono chiamate, invece,

"sinistro" e "destro".
Ovviamente è possibile utilizzare puntatori anzichè cursori. Indicando la mancanza di un figlio col valore **nil** contenuto nell'apposito campo si ha:

```
type
  binalbero = ↑bincella;
  nodo = binalbero;
  bincella = record
    valore: tipoelem;
    sinistro, destro, genitore: nodo;
  end;
procedure CREABINALBERO(var T: binalbero);
  begin
    T := nil
  end;
function BINALBEROVUOTO(var T: binalbero): boolean;
  begin
    BINALBEROVUOTO := (T = nil)
  end;
function BINRADICE(var T: binalbero): nodo;
  begin
    BINRADICE := T
  end;
function BINPADRE(u: nodo; var T: binalbero): nodo;
  begin
    BINPADRE := u↑.genitore
  end;
function FIGLIOSINISTRO(u: nodo; var T: binalbero): nodo;
  begin
    FIGLIOSINISTRO := u↑.sinistro
  end;
function FIGLIODESTRO(u: nodo; var T: binalbero): nodo;
  begin
    FIGLIODESTRO := u↑.destro
  end;
function SINISTROVUOTO(u: nodo; var T: binalbero): boolean;
  begin
    SINISTROVUOTO := (u↑.sinistro = nil)
  end;
function DESTROVUOTO(u: nodo; var T: binalbero): boolean;
  begin
    DESTROVUOTO := (u↑.destro = nil)
  end;
procedure COSTRBINALBERO(var T, U: binalbero);
  var u: nodo;
```

```
42          begin
43            new(u);
44            u↑.sinistro := T; u↑.destro L := U; u↑.genitore := nil;
45            if T ≠ nil then T↑.genitore := u;
46            if U ≠ nil then U↑.genitore := u;
47            T := u
48          end;
49
```

Capitolo 10

Code con priorità

Una coda con priorità è un particolare insieme, sugli elementi del quale è definita una relazione "$\leq$" di ordinamento totale, in cui è possibile inserire un nuovo elemento o estrarre l'elemento "minimo". La specifica è simile a quella del tipo di dato "insieme", in cui sono amesse, oltre alla creazione, le operazioni `INSERISCI`, `MIN` e la nuova operazione `CANCELLAMIN`, che rimuove l'elemento minimo. Il nome deriva dall'interpretazione in cui A è una coda di elementi che devono essere serviti rispettandone la priorità. Per semplicità si fa coincidere le priorità con gli elementi stessi e si assumono priorità distinte in accordo alla specifica del tipo di dato insieme.

10.1 Specifica

La specifica sintattica è la seguente:

`CREAPRIORCODA:`	$() \to$ priorcoda
`INSERISCI:`	(tipoelem, priorcoda) $\to$ priorcoda
`MIN:`	(priorcoda) $\to$ tipoelem
`CANCELLAMIN:`	(priorcoda) $\to$ priorcoda

La specfica semantica è la seguente:

- `CREAPRIORCODA()` = A
  Post: $A = \emptyset$

- INSERISCI(x, A) = A'
  Post: $A' = A \cup \{x\}$

- MIN(A) = x
  Pre: $A \neq \emptyset$
  Post: $x \in A$ e $x < y \quad \forall y \in A \; x \neq y$

- CANCELLAMIN(A) = A'
  Pre: $A \neq \emptyset$
  Post: $A' = A - \{x\}$ ove $x = \text{MIN}(A)$

10.2 Realizzazioni

Si può rappresentare una coda con priorità di n elementi utilizzando strutture sequenziali come liste ordinate e non ordinate.
Tuttavia, poichè la coda con priorità è costituito da un insieme di atomi linearmente ordinati ma senza alcuna relazione strutturale sull'insieme delle posizioni, la sua raprpesentazione è quasi sempre associata ad un unico modello concettuale: quello di albero binario.

10.2.1 Realizzazione con heap

Gli elementi della coda con priorità C possono essere disposti in un vettore H, chiamato *heap*, che può essere interpretato come un albero binario B. Ciascun nodo di B corrisponde ad una posizione di H che memorizza un elemento di C. L'albero B deve verificare le seguenti proprietà:

1. se h è il livello massimo delle foglie, allora ci sono esattamente $2^h - 1$ nodi di livello minore di h;

2. tutte le foglie di livello h sono addossate a sinistra;

3. ogni nodo contiene un elemento di C che è maggiore di quello contenuto nel padre.

Grazie alle prime due proprietà, ciascun livello k (tranne l'ultimo) di B contiene tutti i possibili 2^k nodi, $0 \leq\leq h$, mentre tutti i nodi di livello massimo h stanno a sinistra. B è "quasi pieno", cioè non ha nodi mancanti tranne le foglie a destra di livello massimo. Pertanto gli n nodi di B corrispondono alle posizioni $1 \ldots n$ del

vettore H nell'ordine in cui si incontrano i nodi visitando B per livelli crescenti ed esaminando da sinistra verso destra i nodi dello stesso livello. Poichè ad ogni livello il numero di nodi raddoppia, si ha che:

- la radice di B corrisponde alla posizione 1 di H;
- il figlio sinistro e il figlio destro di i corrispondono alle posizioni $2i$ e $2i+1$ di H.

Il figlio sinistro (destro) di i non esiste se e solo se $2i > n$ ($2i + 1 > n$). Inoltre, per $1 \leq i \leq n$, il padre di i è $\frac{i}{2}$. Ovviamente in $H[i]$ è memorizzato l'elemento di C contenuto nel corrispondente nodo di B. Così, per la terza proprietà, se il figlio sinistro (destro) di i, allora $H[2i] > H[i]$ ($H[2i + 1] > H[i]$).
La realizzazione degli operatori MIN, CANCELLAMIN e INSERISCI è molto semplice se si immagina di operare sull'albero B anzichè sul vettore H. Per eseguire MIN basta leggere l'elemento contenuto della radice, mentre per eseguire CANCELLAMIN si cancella la foglia di livello massimo più a destra, così da mantenere verificate le prime due proprietà, se ne copia l'elemento nella radice, e si fa "scendere" tale elemento lungo un percorso radice-foglia, scambiandolo col minimo degli elementi contenuti nei figli, fino a verificare la terza proprietà.
Vediamo le procedure Pascal che realizzano la coda con priorità usando il vettore heap. Gli n elementi sono memorizzati in $heap[1 \ldots n]$ dove $heap[1]$ contiene l'elemento minimo. Il cursore *ultimo* registrea n, cioè l'ultima posizione occupata, che corrisponde alla foglia di B che sta più a destra tra quelle di livello massimo. Al solito, il valore 0 in *ultimo* indica una coda con priorità vuota.

```
1   type
2     priorcoda = record
3       heap: array[1..maxlung] of tipoelem;
4       ultimo: 0..maxlung;
5     end;
6
7   procedure CREAPRIORCODA(var C: priorcoda);
8     begin
9       C.ultimo := 0
10    end;
11
12  function MIN(var C: prioricoda): tipoelem;
13    begin
14      if C.ultimo = 0
15        then {errore: priorcoda vuota}
16        else
```

```
            MIN := C.heap[1]
    end;

```

Nella INSERISCI, si incrementa il valore del cursore *ultimo*, così da far spazio per il nuovo elemento x. Questo corrisponde ad aggiungere una foglia al livello massimo più a sinistra in B. Si usano due cursori i e k per risalire l'heap. Inizialmente i indica la foglia che è stata aggiunta. Il cursore $k = \lfloor i/2 \rfloor$ indica il padre del nodo i, che esiste solo se $i > 1$, cioè se i non è la radice. Se non è verificata la terza proprietà tra $heap[i]$ e $heap[k]$, allora avviene lo scambio e si continua a risalire, con i che assume il vecchio valore di k. Nel caso pessimo, la risalita ha termine quando è raggiunta la radice, cioè quando $i = 1$.

```
    procedure INSERISCI(x: tipoelem; var C: priorcoda);
      var i, k: integer; temp: tipoelem;
      begin
        if C.ultimo = maxlung then
          {errore: priorcoda piena}
        else
          C.ultimo := C.ultimo + 1
          C.heap[C.ultimo] := x;
          i := C.ultimo;
          if i > i then k := i div 2;
          while (i > 1) and (C.heap[i] < C.heap[k]) do begin
            temp := C.heap[i];
            C.heap[i] = C.heap[k];
            C.heap[k] = temp;
            i := k;
            if i > 1 then k := i div 2;
          end
      end
    end;

```

Nella CANCELLAMIN, si decrementa il valore del cursore *ultimo*, così da rimuovere la foglia al livello massimo più a destra in B, il cui valore è copiato nella radice. Si usano ancora due cursori i e k per scendere nell'heap, ma stavolta i inizia dalla radice. Il cursore k indica il figlio sinistro $2i$ o il figlio destro $2i + 1$, se esistono, a seconda di chi contiene l'elemento minimo. Se non è verificata la terza prorprietà tra $heap[i]$ e $heap[k]$, allora avviene lo scambio e si continua a scendere con i che assume il vecchio valore di k. Nel caso peggiore, la discesa ha termine quando è raggiunta una foglia, cioè quando $i > \lfloor n/2 \rfloor$ dove n è il valore contenuto in ultimo.

```
1   procedure CANCELLAMIN(var C: priorcoda);
2     var i, k: integer; temp: tipoelem; scambio: boolean;
3     begin
4       if C.ultimo = 0 then
5         {errore: priorcoda vuota}
6       else
7         C.heap[1] := C.heap[C.ultimo]
8         C.ultimo := C.ultimo - 1;
9         i := 1;
10        scambio = true;
11        while (i < C.ultimo div 2) and (scambio) do begin
12          k := 2 * i;
13          if (k < C.ultimo) then
14            if C.heap[k] > C.heap[k + 1] then
15              k := k + 1;
16          if C.heap[k] < C.heap[i] then
17            temp := C.heap[i];
18            C.heap[i] := C.heap[k];
19            C.heap[k] := temp;
20            i := k
21          else
22            scambio := false;
23        end
24    end
25  end;
26
```

Capitolo 11

Alberi binari di ricerca

Gli alberi di ricerca sono strutture dati che supportano molte operazioni sugli insiemi dinamici:

- SEARCH(S, k)
  Query che dato un insieme S e un valore chiave k restituisce un elemento x di S.

- MINIMUM(S) e MAXIMUM(S)
  Query su insieme S totalmente ordinato che restituiscono un elemento di S con la chiave più piccola, rispettivamente con la chiave più grande.

- SUCCESSOR(S, x) e PREDECESSOR(S, x)
  Query che dato un elemento x, la cui chiave appartiene ad S, restituisce il prossimo elemento più grande, rispettivamente il più piccolo.

- INSERT(S, x) e DELETE(S, x)
  Operazioni di modifica che inseriscono, rispettivamente rimuovo, elementi da S.

Gli alberi di ricerca possono essere utilizzati come dizionari o come code di priorità.
Un albero binario di ricerca è organizzato in un albero binario e valgono le segueneti proprietà:

1. Per ogni nodo u, tutti gli elementi contenuti nel sottoalbero radicato nel figlio sinistro di u sono minori dell'elemento contenuto in u;

2. Per ogni nodo u, tutti gli elementi contenuti nel sottoalbero radicato nel figlio destro di u sono maggiori dell'elemento contenuto in u.

Queste due proprietà hanno lo scopo di agevolare la ricerca di un elemento.
Le chiavi vengono memorizzate in modo tale da soddisfare le proprietà di un albero binario di ricerca (BST), ovvero se u è un nodo in un BST e y è un nodo nel sotto-albero sinistro (destro) di x, allora $y.key \leq x.key$ ($y.key \geq x.key$).
Sfruttando le proprietà di un BST possiamo stampare in ordine gli elementi di un BST con una funzione ricorsiva di visita inorder:

```
INORDER-TREE-WALK(BST T)
  INORDER-TREE-WALK(T, T.root())
INORDER-TREE-WALK(BST T, Node n)
  if (!T.sx_empty(n))
    INORDER-TREE-WALK(T, T.sx(n))
  print n.key
  if (!T.dx_empty(n))
    INORDER-TREE-WALK(T, T.dx(n))

```

La complessità è lineare rispetto al numero dei nodi del BST.
Spesso si ha la necessità di cercare una chiave in un BST. Oltre all'operazione di ricerca, un BST mette a disposizione anche operazioni di `MINIMUN`, `MAXIMUM`, `SUCCESSOR`, `PREDECESSOR` e la complessità è lineare rispetto all'altezza dell'albero.

11.1 Specifica

La specifica sintattica è la seguente:

`create`:	() → BST
`empty`:	(BST) → boolean
`insert`:	(BST, key) → BST
`erase`:	(BST, key) → BST
`search`:	(BST, key) → Node
`minimum`:	(BST) → Node
`maximum`:	(BST) → Node
`predecessor`:	(BST, Node) → Node
`successor`:	(BST, Node) → Node

Di seguito l'implementazione dei metodi:

```
TREE-SEARCH(BST T, key k)
  TREE-SEARCH(T, T.root())

TREE-SEARCH(BST T, Node n, key k)
  if (n.key == k)
    return n
  if (k < n.key && !T.sx_empty(n))
    return TREE-SEARCH(T, T.sx(n), k)
  if (k > n.key && !T.dx_empty(n))
    return TREE-SEARCH(T, T.dx(n), k)
  return NULL

TREE-MINIMUN(BST T)
  TREE-MINIMUM(T, T.root())

TREE-MINIMUM(BST T, Node n)
  if (T.sx_empty(n))
    return n
  return TREE-MINIMUM(T, T.sx(n))

TREE-MAXIMUM(BST T)
  TREE-MAXIMUM(T, T.root())

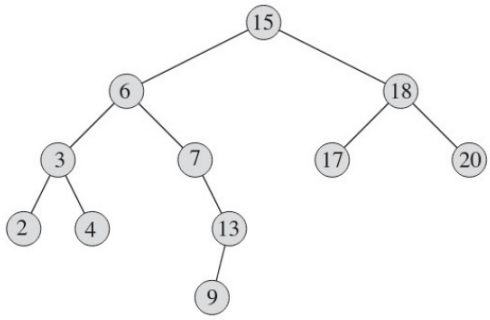
TREE-MAXIMIM(BST T, Node n)
  if (T.dx_empty(n))
    return n
  return TREE-MAXIMUM(T, T.sx(n))

TREE-SUCCESSOR(BST T, Node n)
  if (!T.dx_empty(n))
    return TREE-MINIMUM(T, T.dx(n))
  if (T.root() != n) {
    p = T.parent(n)
    while (T.dx(p) == n) {
      n = p
      p = T.parent(n)
    }
    return p
  }
  return NULL

```

Nel successore se il nodo n ha un figlio destro il successore è il minimo del sotto-albero destro, altrimenti il successore è il più piccolo antenato di n il cui figlio

sinistro è anche antenato di n, ad esempio:



Il successore di 15 è 17 e il successore di 13 è 5.
Analizziamo l'inserimento:

```
TREE-INSERT(BST T, key k)
  if (T.empty())
  T.ins_root()
  n.key = k
  T.write(T.root(), n)
else
  x = T.root()
  found = true
  while (found)
    if (k < x.key && !T.sx_empty())
      x = T.sx(x)
    elif (k > x.key && !T.dx_empty())
      x = T.dx(x)
    else
      found = false
  if (k < x.key)
    T.ins_sx(x)
    n.key = k
    T.write(T.sx(x), n)
  else
    T.ins_dx(x)
    n.key = k
    T.write(T.dx(x), n)

```

Analizziamo la rimozione di una chiave k memorizzata in un nodo z del BST.

- Se z è foglia rimuoviamo il nodo z dal BST;

- Se z ha un solo figlio y allora facciamo occupare da y la posizione di z nel BST;
- Se z ha due figli, allora cerchiamo il successore di y di z, e facciamo occupare ad y la posizione di z nel BST:
  - y si troverà nel sotto-albero destro di z;
  - Si dovrà distinguere tra:
    * y figlio destro di z;
    * y appartenente al sotto-albero destro di z ma non figlio destro di z, si noti che in questo caso y non ha figlio sinistro in quanto è il minimo di un sotto-albero.

11.2 Albero rosso-neri

Abbiamo visto che in un BST le operazioni hanno complessità lineare rispetto all'altezza dell'albero. Gli alberi rosso-neri (RBT) sono uno dei tanti schemi di bilanciamento di BST che garantiscono complessità log-lineare nel caso pessimo. Un albero rosso-nero è un BST in cui ogni nodo memorizza anche il proprio colore, rosso o nero. Imponendo un vincolo sul colore dei nodi su un path dalla radice ad una foglia, i RBT assicurano che nessun path ha una lunghezza doppia di un altro, quindi l'albero è approssimativamente bilanciato.
Un RBT soddisfa le seguenti proprietà:

- Ogni nodo è rosso o nero;
- La raidice è nera;
- Ogni foglia è nera;
- Se un nodo è rosso, entrambi i figli sono neri;
- Per ogni nodo, tutti i path dal nodo ad una foglia contentono lo stesso numero di nodi neri.

Si dimostra che un RBT con n nodi interni ha un altezza al più di $2\log(n+1)$ e quindi la ricerca in un RBT ha complessità log-lineare.
Le operazioni di inserimento e cancellazione per BST potrebbero dare un BST che viola le proprietà di RBT e per ripristinare le proprietà si dovrà cambiare il colore di alcuni nodi dell'albero. Questa si ottiene con delle operazioni di rotazione che preservano le proprietà di un BST.

11.3 AVL Tree

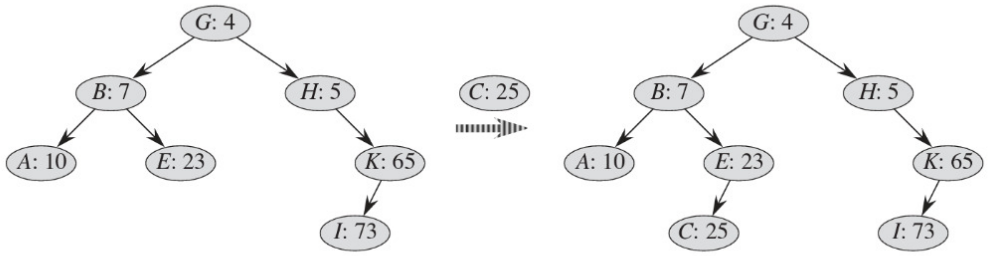
Un AVL Tree è un BST bilanciato in altezza: per ogni nodo x, le altezze del sotto-albero sinistro e destro di x differiscono di al più 1.
Per implementare un AVL si mantiene un attributo extra per ogni nodo, la sua altezza. Per l'inserimento usiamo l'inserimento classico per i BST, ma l'albero potrebbe non essere bilanciato in altezza e in questo caso si usano delle rotazioni che rendono tali alberi bilanciati in altezza.
In alternativa, si può usare una proceudra ricorsiva che inserisce un nodo in un AVL tree conservandole proprietà di un AVL in tempo log-lineare.

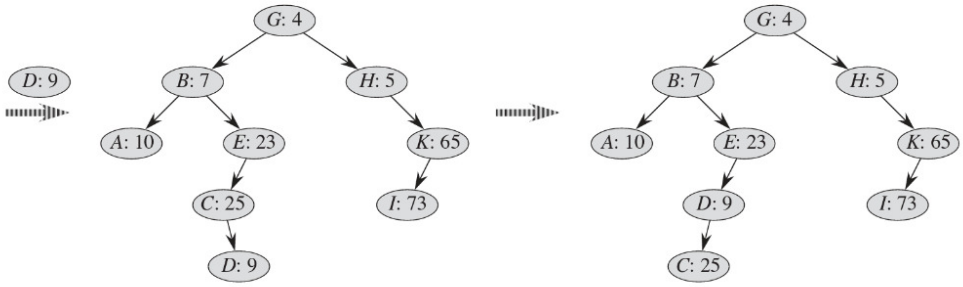
11.4 Treaps

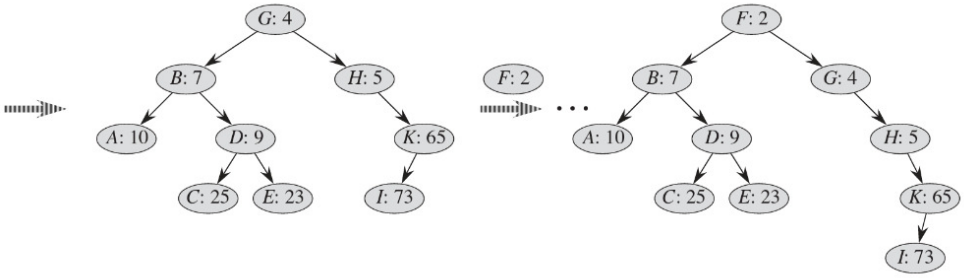
L'inserimento di n chiavi in un BST potrebbe dar luogo ad un albero molto sbilanciato, degradando le prestazioni della ricerca di chiavi. Si può dimostrare che BST costruiti a random tendono ad essere bilanciati: dato un insieme di elementi si inseriscono gli elementi nel BST seguendo l'ordine di una loro permutazione random.
Ma, se gli elementi vengono forniti uno alla volta la soluzione per costruire in modo random il corrispondente BST è mediante l'utilizzo di *treaps*.
Un treap è un BST che memorizza i nodi in modo diverso dal tradizionale. Ad ogni nodo, oltre alla chiave è associata una priorità, un numero random scelto indipendentemente per ogni nodo e **tutte le chiavi e le priorità sono distinte**.
I nodi in un treap sono ordinati in modo tale che le chiavi obbediscono alle proprietà di un BST e le priorità alla proprietà di un min-heap:

- Se v è figlio sinistro di u, allora $v.key < u.key$;
- Se v è figlio destro di u, allora $v.key > u.key$;
- Se v è figlio di u, allora $v.priority > u.priority$.

Quindi, un treap ha le caratteristiche di un BST e di un heap.
Supponiamo di inserire i nodi $x_1, x_2, \ldots, x_n$ in un treap. Il treap risultante è un albero che si sarebbe ottenuto se i nodi fossero stati inseriti in un normale BST nell'ordine dato dalle loro priorità, scelte in modo randomico. Ad esempio:


G: 4
B: 7
H: 5
A: 10
E: 23
K: 65
I: 73
C: 25
G: 4
B: 7
H: 5
A: 10
E: 23
K: 65
C: 25
I: 73


D: 9
G: 4
B: 7
H: 5
A: 10
E: 23
K: 65
C: 25
I: 73
D: 9
G: 4
B: 7
H: 5
A: 10
E: 23
K: 65
D: 9
I: 73
C: 25


G: 4
B: 7
H: 5
A: 10
D: 9
K: 65
C: 25
E: 23
I: 73
F: 2
F: 2
B: 7
G: 4
A: 10
D: 9
H: 5
C: 25
E: 23
K: 65
I: 73

Capitolo 12

Grafi

Un grafo orientato è una coppia $G = (N, A)$, con N insieme finito di elementi, detti nodi, ed A insieme finito di coppie ordinate di nodi, detti archi. I grafi orientati possono essere rappresentati graficamente disegnando ogni nodo con un cerchietto e ogni arco (i, j) con una freccia che esce dal nodo i al nodo j. Il numero di nodi ed il numero di archi sono di solito indicati, rispettivamente, con n ed m. Poichè possono esserci al più $n-1$ archi uscenti da ciascun nodo, risulta $0 \leq m \leq n(n-1)$. In generale, i nodi sono usati per rappresentare oggetti e gli archi per rappresentare relazioni tra coppie di oggetti.
In un grafo orientato G, un cammino è una sequenza di nodi $u_0, u_1, \ldots, u_k$ tali che $(u_i, u_{i+1}) \in A$, per $i = 0, 1, 2, \ldots, k-1$. Il cammino parte dal nodo u_0, attraversa i nodi $u_1, \ldots, u_{k-1}$, arriva al nodo u_k, ed ha lunghezza uguale al numero k di archi. Se non ci sono nodi ripetuti, cioè $u_i \neq u_j$, per $0 \leq i < j \leq k$, allora il cammino è semplice, mentre se $u_0 = u_k$, allora è chiuso. Un cammino sia semplice che chiuso, dove l'unica ripetizione è $u_0 = u_k$, è detto ciclo.
Un grafo orientato è fortemente connesso se per ogni coppia di nodi u e v esiste almeno un cammino da u a v ed almeno un cammino da v ad u.
Talvolta, gli archi di un grafo G sono formato da coppie non ordinate $[i, j]$. In tal caso, G è un grafo non orientato. Mentre in un grafo orientato (i, j) e (j, i) indicano due archi distinti, in uno non orientato $[i, j]$ e $[j, i]$ indicano lo stesso arco, che è detto incidere sui due nodi, i quali sono adiacenti tra loro. Grafi non orientati sono usati per rappresentare relazioni simmetriche tra oggetti.
Anche nei grafi non orientati ci sono nozioni analoghe a quelle di cammino e ciclo. In un grafo non orientato G, una catena è una sequenza di nodi $u_0, u_1, \ldots, u_k$ tali che $[u_i, u_{i+1}] \in A$, per $i = 0, 1, 2, \ldots, k-1$. La catena collega u_0 con u_k, o viceversa, ed ha lunghezza uguale a k. Se non ci sono nodi ripetuti, allora la ca-

tena è semplice, mentre se $u_0 = u_k$, allora è chiusa. Una catena sia semplice che chiusa i cui archi sono tutti distinti è un circuito. Si noti che nei grafi orientati la mancanza di ripetizioni di nodi implica la mancanza di ripetizione degli archi, ma questo non è vero per i grafi non orientati.
Un grafo non orientato è connesso se per ogni coppia di nodi distinti u e v esiste una catena tra u e v.

12.1 Specifica

Poichè ogni grafo non orientato G può essere visto come un grafo orientato D, ottenuto da G sostituendo ogni arco $[i, j]$ con i due archi (i, j) e (j, i), sarà fornita una specifica solo per grafi orientati. Le operazioni definite sui grafi prevedono la possibilità di inserire o cancellare nodi o archi, e di spostarsi da un nodo ad un suo adiacente.

CREAGRAFO :	$() \rightarrow$ grafo
GRAFOVUOTO :	(grafo) $\rightarrow$ booleano
INSNODO :	(nodo, grafo) $\rightarrow$ grafo
INSARCO :	(nodo, nodo, grafo) $\rightarrow$ grafo
CANCNODO :	(nodo, grafo) $\rightarrow$ grafo
CANCARCO :	(nodo, nodo, grafo) $\rightarrow$ grafo
ADIACENTI :	(nodo, grafo) $\rightarrow$ insieme

L'operazione CREAGRAFO restituisce un grafo "vuoto", cioè senza nodi e senza archi, mentre GRAFOVUOTO verifica se il grafo è vuoto oppure no. L'operazione INSNODO aggiunge al grafo un nuovo nodo "isolato", cioè non collegato a nessun altro nodo del grafo, mentre INSARCO aggiunge un nuovo arco tra una coppia di nodi del grafo. CANCARCO permette di cancellare un arco esistente tra una coppia di nodi, mentre CANCNODO cancella un nodo "isolato". Se il nodo da cancellare non è isolato, occorre preventivamente cancellare tutti gli archi che lo collegano ad altri nodi con ripetute applicazioni di CANCARCO. Infine, l'operazione ADIACENTI restituisce l'insieem di tutti i nodi adiacenti ad un certo nodo, cioè tutti i nodi raggiungibili con un arco che esce da tale nodo. Di seguito la specifica semantica:

- CREAGRAFO = G
  Post: $G = (N, A)$, con $N = \emptyset$ ed $A = \emptyset$

- GRAFOVUOTO(G) = b
  Post: $\begin{cases} b = vero & N = \emptyset \wedge A = \emptyset \\ b = falso & altrimenti \end{cases}$

- INSNODO(u, G) = G'
  Pre: $G = (N, A)$, $u \notin N$
  Post: $G' = (N', A)$ con $N' = N \cup \{u\}$

- INSARCO(u, v, G) = G'
  Pre: $G = (N, A)$, $u \in N$, $v \in N$ e $(u, v) \notin A$
  Post: $G' = (N, A')$ con $A' = A \cup \{(u, v)\}$

- CANCNODO(u, G) = G'
  Pre: $G = (N, A)$, $u \in N \wedge \nexists\, v \in N \mid (u, v) \in A \vee (v, u) \in A$
  Post: $G' = (N', A)$ con $N' = N - \{u\}$

- CANCARCO(v, u, G) = G'
  Pre: $G = (N, A)$, $u \in N$, $v \in N$ e $(u, v) \in A$
  Post: $G' = (N, A')$ con $A' = A - \{(u, v)\}$

- ADIACENTI(u, G) = $A(u)$
  Pre: $G = (N, A)$, $u \in N$
  Post: $A(u) = \{v : (u, v) \in A\}$ detto insieme di adiacenza di u.

In molte applicazioni, ai nodi e/o agli archi sono associate delle informazioni di un certo tipo dette etichette o passi. In tal caso, si parla di grafi etichettati o pesati, sui quali è utile introdurre ovvi operatori, per reperire e/o modicare le informazioni associate a nodi e/o archi.

12.2 Realizzazioni con matrici

12.2.1 Matrici di incidenza

La più semplici rappresentazioni di grafi utilizzano matrici. Tra queste le più usate sono la matrice di incidenza nodi-archi e la matrice di adiacenza nodi-nodi. La matrice di incidenza nodi-archi è una matrice rettangolare $B = [b_{ik}]$ di dimensioni $n \times m$, dove ciascuna riga rappresenta un nodo e ciascuna colonna rappresenta

un arco, tale che:

$$b_{ik} = \begin{cases} -1 & \text{se l'arco } k\text{-esimo esce dal nodo } i \\ +1 & \text{se l'arco } k\text{-esimo entra nel nodo } i \\ 0 & \text{altrimenti} \end{cases}$$

Se il grafo non è orientato, invece, si ha:

$$b_{ik} = \begin{cases} 1 & \text{se l'arco } k\text{-esimo è incidente nel nodo } i \\ 0 & \text{altrimenti} \end{cases}$$

La matrice di incidenza è molto utile per rappresentare un grafo in certi problemi di programmazione linare, nei quali si cerca un vettore x di m incognite associato agli archi che soddisfi un sistema di equazioni del tipo $Bx = c$, dove c è un vettore di n termini noti interi. Lo spazio di memoria occupato è sempre $\Theta(nm)$ e dato un nodo non è agevole ricavare l'insieme di adiacenza. Infatti per calcolare un qualsiasi $A(u)$ occure scandire la riga u di B alla ricerca delle colonne k con elementi $b_{uk} = -1$ e per ciascuna colonna k scandirla per individuare l'indice di riga v tale che $b_{vk} = +1$.

12.2.2 Matrici di adiacenza

La matrice di adiacenza nodi-nodi, invece, è una matrice quadrata $E = [e_{ij}]$ di dimensioni $n \times n$, tale che:

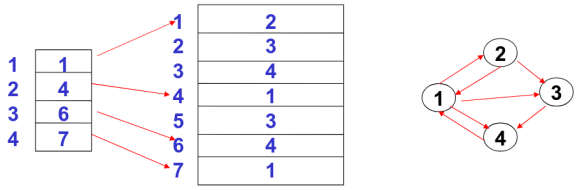
$$e_{ij} = \begin{cases} 1 & (i,j) \in A \\ 0 & (i,j) \notin A \end{cases}$$

Se il grafo non è orientato, allora tale matrice è simmetrica, ovvero $e_{ij} = e_{ji}$ per ogni i e j.
Con questa realizzazione è banale verficare in tempo $O(1)$ se un dato arco è presente nel grafo oppure no. Il tempo per ricavare l'insieme di adiacenza $A(u)$ di un qualsiasi nodo u è sempre $\Theta(n)$, indipendentemente dalla cardinalità di $A(u)$, perchè occorre scandire un'intera riga della matrice alla ricerca degli elementi $b_{uv} = 1$. Se il grafo è pesato, allora nella matrice E si utilizzano i pesi degli archi al posto degli elementi binari. Se p_{ij} è il peso dell'arco (i,j), allora la matrice E diventa:

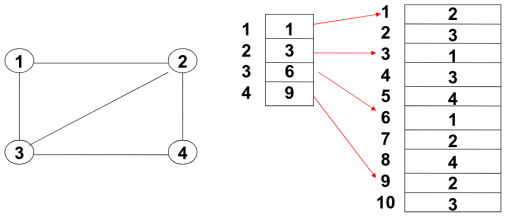
$$e_{ij} = \begin{cases} p_{ij} & (i,j) \in A \\ +\infty \ (-\infty) & (i,j) \notin A \end{cases}$$

12.3 Realizzazione con vettore di adiacenza

È possibile rappresentare il grafo $G = (N, A)$ con due vettori, il vettore NODI e il vettore ARCHI.
Il vettore NODI è formato da N elementi e NODI(i) contiene un cursore alla posizione di ARCHI a partire dalla quale è memorizzato $A(i)$. Per semplicità denotiamo i nodi con interi. Nel caso il grafo sia etichettato possiamo associare al nodo altre informazioni.

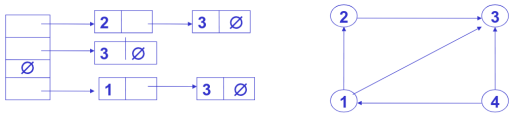


Se il grafo non è orientato è necessario rappresentare ogni arco due volte, se i grafi sono etichettati sui nodi e/o sugli archi i pesi possono essere memorizzati in vettori PESINODI(n) e PESIARCHI(m).



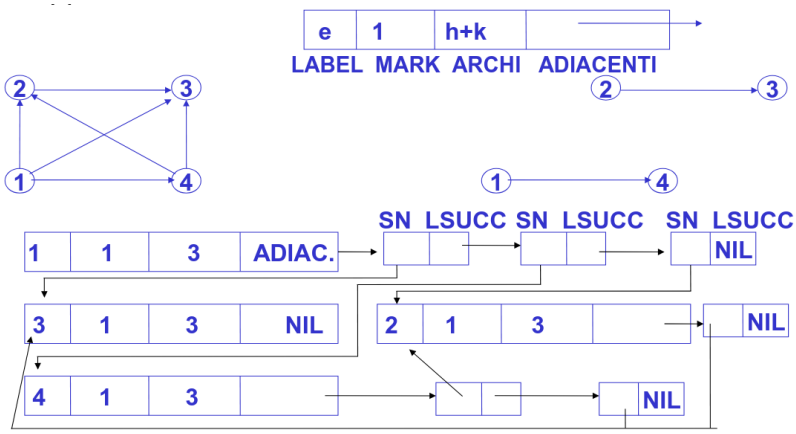
12.4 Rappresentazione con lista di adiancenza

È possibile anche utilizzare un vettore di nodi A ed n liste. Una generica componente $A(i)$ del vettore è il puntatore alla lista $i-$esima in cui sono memorizzati i nodi adiacenti a i.



12.5 Rappresentazione con struttura a puntatori

La rappresentazione con struttura a puntatori è la versione dinamica della matrice di adiacenza estesa. Il nodo v con etichetta e, h archi entranti e k archi uscenti è rappresentato mediante un record.



12.6 Esplorazione di un grafo

Vediamo due metodi sistematici per "visitare" almeno una volta ogni nodo ed ogni arco di un grafo orientanto fortemente connesso (o non orientato e connesso).
Entrambi i metodi effettuano la visita in tempo ottimo $\Theta(n+m)$ a partire da un generico nodo r:

1. *Depth-First-Search* (DFS), ovvero visita in profondità, detta anche a scandaglio;

2. *Breadth-First-Search* (BFS), ovvero visita in ampiezza, detta anche a ventaglio.

Il metodo di visita DFS è una diretta estensione della visita in ordine anticipato di un albero ordinato. Quando viene visitato un nuovo nodo u, ci si allontana da esso il più possibile lungo un cammino (o catena), visitando nodi (e archi) nel cammino fino a giungere in un "vincolo cieco", ovvero in un nodo v i cui nodi adiacenti sono già stati tutti visitati. Quando si è raggiunto v, si torna indietro

lungo l'ultimo arco visitato e si riprende la visita allontanandosi lungo un altro cammino di nodi (e archi) non ancora visitati.
Nella visita BFS, invece, i nodi sono visitati in ordine di distanza crescente dal nodo di partenza r, dove la distanza r ad un generico nodo u è il minimo numero di archi in un cammino (o catena) da r ad u. Tale metodo di visita è un'estensione di una visita per livelli di un albero radicato, in cui i figli di un nodo sono visitati dopo aver visitato tutti gli altri nodi che stanno allo stesso livello del padre.
In entrambi i metodi, occorre mantenere una lista dei nodi che sono già stati visitati, ma i cui nodi adiacenti non sono ancora stati visitati. Poichè la DFS torna indietro dal più recente vicolo cieco, tale lista è di fatto una pila. D'altro lato, poichè la BFS visita i nodi più vicini ad r prima di quelli più lontani, tale lista è una coda. Nel seguito la DFS è descritta con una procedura Pascal ricorsiva, perchè più concisa ed elegante, mentre la BFS è descritta utilizzando gli operatori delle code.

```
1     procedure DFS(var G: grafo; u: nodo);
2       var v: nodo;
3       begin
4         {esamina il nodo u e visitato = true}
5         for each v in A(u) do begin
6           {esamina l'arco (u, v)}
7           if v not visitato then DFS(G, v);
8         end;
9       end;
10
11    procedure BFS(var G: grafo; u: nodo);
12      var v: nodo, Q: coda;
13      begin
14        CREACODA(Q);
15        INCODA(u, Q);
16        while not CODAVUOTA(Q) do begin
17          u := LEGGICODA(Q);
18          FUORICODA(Q);
19          {esamina il nodo u e visitato = true};
20          for each v in A(u) do begin
21            {esamina l'arco (u, v)};
22            if v not visitato and v not in Q then INCODA(v, Q);
23          end;
24        end;
25      end;
26
```

Le procedure DFS e BFS sono metodi di vista sistematica che possono essere usati

per risolvere molti problemi. Ad esempio:

1. Verificare se un grafo non orientato è conneso oppure no (lo è solo se e solo se, al termine della visita, tutti i nodi sono stati marcati visitati);

2. Verificare se un grafo orientato è fortemente connesso oppure no (lo è se e solo se, al termine della visita, tutti i nodi sono stati marcati visitati e, rovesciando successsivamente il vereso di tutti gli archi (i, j) in (j, i) e riapplicando la visita a partire dal medesimo nodo r, tutti i nodi vengono nuovamente marcati visitati);

3. Trovare le componenti connesse di un grafo non orientato G, cioè tutti i sottografi connessi di cui G è composto, tali che nessun sottografo è contenuto in uno più grosso (è sufficiente richiamare iterativamente una DFS modifcata in modo da marcare ciascun nodo con il numero della componente connessa cui il nodo appartiene).

Inoltre, sia la visita DFS che la visita BFS di un grafo, connettendo i nodi marcati ad uno ad uno a partire da un generico nodo i, generano un albero radicato nel nodo iniziale i. Questo risulta interessante quando si voglia risolvere un problema che richiede di selezionare, tra tutti i percorsi che connettono due nodi in un grafo, quello che risulta ottimo, ad esempio in base al criterio di minimizzazione della somma dei pesi associati agli archi, detto ricerca dell'albero minimo di copertura o di attraversamento.
La ricerca del percorso più breve è un problema classico. Sia $G = (N, A)$ un grafo orientato etichettato negli archi con valori interi positivi, i pesi. Partendo dal nodo $r \in N$, per ogni nodo $u \in N$, si calcola il percorso (cammino o catena) che connette r ad u tale che la somma dei pesi associati agli archi che compongono il cammino sia minima. Ad esempio i nodi possono essere delle città e gli archi sono le strade che le collegano e le etichette sugli archi sono le distanze. Il problema consiste nel trovare il percorso più breve tra tutti quelli che connettono la città alle altre città.

Capitolo 13

Complessità degli algoritmi e efficienza dei programmi

Lo studio della computabilità aiuta a stabilire quali problemi ammettono una soluzione algoritmica e quali no. Per i problemi computabili, è interessante conoscere la complessità degli algoritmi che lo risolvono. La *complessità di un algoritmo* è una misura delle risorse di calcolo consumate durante la computazione e l'*efficienza di un algoritmo* è inversamente proporzionale alla sua complessità. La teoria della complessità studia l'uso delle risorse di calcolo necessarie per la computazione di algoritmi e, più in generale, per la soluzione di problemi. Questo studio è importante al fine di:

- poter confrontare algoritmi diversi che risolvono lo stesso problema;
- stabilire se dei problemi ammettono algoritmi risolutivi che sono praticamente computabili.

Intuitivamente un programma è più efficiente di un altro se la sua esecuzione richiede meno risorse di calcolo. Le risorse di calcolo che si considerano sono:

- il tempo di elaborazione richiesto;
- la quantità di memoria necessaria.

Se si potesse valutare il tempo di calcolo in unità standard, come i secondi, la valutazione del costo di un programma sarebbe semplice: basterebbe eseguire il programma misurando il tempo necessario per l'esecuzione. Ma, questo metodo non è attendibile perchè bisgoan considerare le condizioni in cui si effettuano le prove, infatti, bisgona considerare:

- l'elaboratore su cui il programma viene eseguito;
- il compilatore usato per la traduzione;
- la modalità di ingresso dei dati;
- la significatività dei dati sui quali è eseguita la prova.

Utilizzando unità di tempo standard come i secondi non otteniamo una valutazione oggettiva del costo di esecuzione di un programma.
Per risolvere un problema spesso sono disponibili molti algoritmi diversi e, in genere, si valutà la qualità di un algoritmo o si confrontano più algoritmi sulla base del comportamento ch questi presentano al crescere della dimensione del problema.

13.1 Risorse di interesse

Le risorse di interesse sono il *tempo* e lo *spazio di memoria*. Quindi per un algoritmo si parla di:

- *complessità in tempo*;
- *complessità in spazio*.

In ogni caso, per poter valutare la complessità occorre definire un *modelli di costo* che dipende dal particolare modello di macchina astratta a cui si fa riferimento. Le unità di misura che si assuomono in teoria della complessità non sono unità di tempo propriamente intese, per due ragioni:

- non ha senso parlare di tempi di esecuzione quando si considerano modelli di macchine astratte;
- anche se descrivessimo due algoritimi attraverso uno specifico linguaggio di programmazione disponibile per un calcolatore reale, il confronto dei tempi di esecuzione dei due programmi sarebbe lecito solo se i programmi fossero tradotti con il medesimo traduttore e fossero eseguiti con lo stesso elaboratore.

L'analisi dell'efficienza di un programma è basata sull'ipotesi che il costo di ogni istruzione semplice e di ogni operazione di confronto sia pari ad un'unità di costo, indipendentemente dal linguaggio e dal sistema usato.
Ci limiteremo a contare le operazioni eseguite o alcune operazioni chiave o premimenti ammettendo che il tempo compessivo di esecuzione sia proporzionale al numero di tali operazioni.
Tratteremo come non significative le costanti moltiplicative e studieremo le funzioni di complessità nel loro ordine di grandezza.

13.1.1 Dimensione dell'input

Il costo di esecuzione di un programma dipende anche dai particolari dati di ingresso. Ad esempio il costo di esecuzione di un programma di un ordinamente di un insieme di numeri dipende dalle dimensioni dell'insieme che si considera. Per tener conto del numero di dati con cui si esegue il programma assumiamo che la dimensione dell'input rappresenta l'argomento della funzione che esprime il costo di esecuzione di un programma.
Si può caratterizzare tale dimensione mediante un intero n che è precisamente identificato nella macchina di Turing come la lunghezza della porzione di nastro che contiene i dati di ingresso. Impiegando un elaboratore ed un suo linguaggio di programmazione, la dimensione n è lo spazio occupato, nella memoria dell'elaboratore, dai dati relativi al problema da risolvere, o più in generale un numero proporzionale a questo spazio. Ad esempio:

- se si opera su matrici n sarà il numero dei suoi elementi;
- se si opera su un insieme, n sarà il numero dei suoi elementi;
- se si opera su un grafo, n sarà il numero dei nodi o il numero di archi o la somma dei due numeri.

13.2 Complessità in spazio

La complessità in spazio è il massimo spazio invaso dalla memoria durante l'esecuzione dell'algoritmo, il quale può costruire insieme di dati intermedi o di servizio, oltre ad operare sui dati iniziali e finali. Anche in questo caso ci si limita in genere allo studio della complessità asintotica. Poichè abbiamo a disposizione memorie grandissime a basso costo, studieremo la complessità in tempo.

Il costo è unario per i dati di tipo semplice, mentre per array di n elementi, il costo è dato da: $n \cdot$ *dimensione di un elemento*.
Il costo di un record è dato dalla somma dei costi delle singole componenti. Ad esempio, il seguente algoritmo:

```
1       somma = 0
2       for i = 1 to n do somma = somma + A[i]
3
```

ha complessità in spazio pari a 13 se $n = 10$.

13.3 Complessità in tempo e asintotica

Fissata la dimensione n, il tempo che un algoritmo impiega a risolvere il problema è la **complessità in tempo**. L'obiettivo principale sarà esprimere la complessità in tempo come funzione di n e spesso ci limiteremo a studiare il comportamento di tale funzione al crescere di n, detta *complessità asintotica* o semplcimente *complessità*, considerando così i soli termini prevalenti e tralasciando a volte anche le costanti moltiplicative.
Lo studio della complessità asintotica è motivato dal fatto che gli algoritmi sono sempre definiti per n generico: se per valori piccoli di n due algoritmi possono avere efficienza confrontabile, è sempre quello che ha il termine massimo di grado più basso a richiedere minor tempo di esecuzione per un numero illimitato di n superiori ad un opportuno valore n_0.

13.3.1 Modello di costo

Il modello di costo che si utilizza è quello di un linguaggio di programmazione lineare. Assumiamo che il costo di ogni istruzione semplice:

- assegnazione, $\leftarrow$
- aritmentica, $+, -, *, /$
- logica, AND, OR, NOT
- confronto, $\leq, \geq, <, >, =, \neq$
- lettura/scrittura

è unitario.
Il costo di esecuzione di un'istruzione composta è pari alla somma dei costi delle istruzioni che la compongono.
Analizziamo diversi casi. Il costo di un'operazione di selezione: if cond then S1 else S2 è dato da:

- costo test cond + costo S1 se la condizione è verificata;
- costo test cond + costo S2 se la condizione non è verificata.

Il costo di un'istruzione di ciclo: while cond do S1 è dato da:

- costo test cond + (costo test cond + costo S1) * n se il ciclo è ripetuto n volte

Il costo di un ciclo "repeat": repeat S1 until cond è dato da:

- (costo test cond + costo S1) * n se il ciclo è ripetuto n volte

Il costo di un'istruzione for: for i = 1 to n do S1 è equivalente al costo del seguente ciclo:

```
1      i = 1
2      while i <= n do
3        S1;
4        i = i + 1
5
```

ed è quindi dato da: 2 + (costo di S1 + 3) * n.
La complessità di un algoritmo non può essere caratterizzata da una sola funzione di complessità. A parità di dimensione di dati, il tempo di esecuzione può dipendere dalla specifica configurazione dei dati. Si considerano di solito tre differenti tipi di complessità: complessità nel caso medio, ottimo e pessimo. Nella *complessità media* il valore della complessità di un algoritmo è mediato su tutte le possibili occorrenze iniziali dei dati. Nella complessità media si usa spesso la probabilità e il calcolo è spesso difficile.
Il calcolo della *complessità nel caso ottimo* si ottiene considerando, a parità di dimensione dei dati, la configurazione che dà luogo al minimo tempo di esecuzione. Tale complessità è per lo più di interesse secondario anche se è abbastanza facile da determinare.
Per calcolo della *complessità nel caso pessimo* si intende la complessità relativa a quella particolare occorrenza iniziale dei dati per l'algoritmo ha comportamento

pessimo. Tale funzione di complessità fornisce un limite superiore alla complessità, entro cui il funzionamento dell'algoritmo è sempre garantito. Come abbiamo detto, si considerano tre tipi di complessità:

- **Caso pessimo**: si ottiene considerando quella particolare configurazione che, a parità di dimensione dei dati, dà luogo al massimo tempo di calcolo. La corrispondente funzione di complessità è indicata con $f_{pess}(n)$;
- **Caso ottimo**: si ottiene considerando la configurazione che dà luogo al minimo tempo di calcolo. La corrispondente funzione di complessità è indicata con $f_{ott}(n)$;
- **Caso medio**: si riferisce al tempo di calcolo mediato su tutte le possibili configurazione dei dati, sempre per una data dimensione n. Funzione di complessità: $f_{med}(n)$

Esistono particolari algoritmi in cui tutte le possibili configurazioni dei dati sono equivalenti per quanto riguarda la complessità in tempo. In questo caso, la complessità ottima, pessima e media coincidono. Ad esempio:

- $f_{pess} = f_{ott} = f_{med}$ per l'algoritmo del fattoriale;
- $f_{pess} = f_{ott} = f_{med}$ per l'algoritmo del $\min$ di n elementi.

Consideriamo la seguente funzione:

```
1     function ricerca_lineare(var t: array, var k: tipochiave, var trovato:
    boolean): boolean;
2     i := 0;
3     repeat
4       i := i + 1
5     until(t[i].chiave = k) or (i = n)
6     if (t[i].chiave = k) then
7       ricerca_lineare := true
8     else
9       ricerca_lineare := false
10
```

Il costo di eseguzione del programma dipende dalla posizione del particolare elemento che si vuole individuare: se l'elemento cercato è il primo del vettore allora si effettua un solo confronto (caso ottimo); se l'elemento cercato è il secondo allora si effettuano due confronti e così via. Il *caso peggiore* è costituito dalla ricerca dell'ultimo elemento o da una ricerca infruttuosa, perchè in questo caso l'algoritmo esamina tutte le componenti dell'array ed esegue il ciclo n volte.
Valutiamo il costo del programma nel caso di ricerca dell'ultimo elemento:

- la prima istruzione è seguita 1 volta;
- a terza istruzione è eseguita n volte;
- il test dell'istruzione repeat è eseguito n volte;
- il test dell'istruzione if then else è eseguito 1 volta;
- la sesta istruzione è eseguita 1 volta;
- la settima istruzione non viene eseguita.

Pertanto il costo del programma è:

$$1 + n + n + 1 + 1 = 3 + 2n$$

È facile vedere che la stessa funzione esprime anche il costo del programma nel caso di ricerca infruttuosa.
Se si vuole valutare il comportamento del programma nel *caso medio* è necessario distinguere il caso di ricerca con successo da quello di ricerca infruttuosa. Nel caso di ricerca fruttuosa, se si assume che tutti gli elementi dell'array possono essere ricercati con uguale probabilità pari a $1/n$, allora è facile vedere che il programma richiede mediamente $(n+1)/2$. Infatti se ricerchiamo l'i-esimo elemento si effettuano i confronti e, quindi, il numero di confronti medio è dato da:

$$\sum_{1 \leq i \leq n} Prob(E(i))i = \sum_{1 \leq i \leq n} \frac{i}{n} = \frac{(n+1)}{2}$$

ove Prob(E(i)) è la probabilità che l'elemento da cercare nell'array sia quello in posizione i.
Riassumendo, la complessità di un algoritmo è funzione della dimensione dei dati, ovvero della mole dei dati del problema da risolvere. L'individuazione della dimensione dei dati è per lo più immediata. Determinare la complessità in tempo (o in spazio) di un algoritmo significa determinare una funzione di complessità $f(n)$ che fornisca la misura del tempo (o dello spazio), al variare della dimensione dei dati, n.
Le funzioni di complessità sono caratterizzate da due proprietà.

1. Assumono solo valori positivi.
2. Sono crescenti rispetto alla dimensione dei dati.

Per semplificare l'analisi, non si cerca di individuare esattamente la funzione che esprime il costo del programma ma si ritiene sufficiente stabilire il suo comportamento asintotico quando le dimensioni dell'input tendono all'infinito. Si trascurano così costanti moltiplicative e termini additivi di ordine inferiore.
Due programmi i cui costi sono espressi rispettivamente dalle funzioni:

$$(n+3) \qquad (100n+3027)$$

sono caratterizzati dalla stessa complessità asintotica.
$(n+3)$ esprime un costo minore della funzione $(100n+3027)$ e, quindi, ignorare la costante moltiplicativa può essere in alcuni casi una semplificazione eccessiva. Lo studio asintotico offre un criterio chiaro per confrontare due algoritmi. Ad esempio siano:

$$A_1 : f_1(n) = 3n^2 - 4n + 2 \qquad A_2 : f_2(n) = 2n + 3$$

Per $n = 1, 2$ abbiamo $f_1(n) < f_2(n)$, per $n = 3, 4$ otteniamo $f_1(n) > f_2(n)$ quindi l'algoritmo più efficiente è A_2 perchè $\forall n > 2$ vale che $f_1(n) > f_2(n)$. Invece, le funzioni di complessità:

$$\begin{aligned} f_1(n) &= 3n^2 + n + 1 \\ f_2(n) &= 4n^2 \\ f_3(n) &= 4n^2 + 2n \end{aligned}$$

sono asintoticamente equivalenti, in quanto crescono tutte con il quadrato di n.
Il costo di esecuzione di un programma, o di un algoritmo, si esprime come funzione delle dimensioni dell'input in cui si ignorano le costanti moltiplicative. Si definiscono, inoltre, diversi tipi di delimitazioni che formalizzano la complessità di un programma o algoritmo.

13.4 Notazioni

Descriviamo le notazioni usate per descrivere la complessità di un programma o di un algoritmo.

Limite superiore $O(f(n))$ è l'insieme di tutte le funzioni $g(n)$ tali che esistono due costanti positive c e m per cui:

$$g(n) \leq cf(n) \quad \forall n \geq m$$

$g(n) \in O(f(n))$ e tradizionalmente si legge "$g(n)$ è di ordine $f(n)$" e fornisce un limite superiore al comportamento asintotico della funzione g ovvero g non si comporta asintoticamente peggio di f.
Applicata alla funzione di complessità, la notazione O ne delimita superiormente la crescita e fornisce un indicatore di bontà dell'algoritmo. Consideriamo la seguente funzione:

$$f(n) = 3n^2 + 3n - 1$$

è un $O(n^2)$ dal momento che esistono due costanti $c = 4$ e $m = 3$ tali che:

$$\forall n \geq 3 \quad f(n) \leq 4n^2$$

In generale, $f(n)$ è un $O(n^k)$ per $k \geq 2$.

Limite inferiore $\Omega(f(n))$ è l'insieme di tutte le funzioni $g(n)$ tali che esistono due costanti positive c e m per cui:

$$g(n) \geq cf(n) \quad \forall n \geq m$$

$g(n) \in \Omega(f(n))$ tradizionalmente si legge "$g(n)$ è di ordine Ω $f(n)$" e fornisce un limite inferiore al comportamento asintotico della funzione.
$\Theta(f(n))$ è l'insieme di tutte le funzioni $g(n)$ che sono sia $\Omega(f(n))$ sia $O(f(n))$. Formalmente è l'insieme di tutte le funzioni $g(n)$ tali che esistano tre costanti positive c, d e m per cui:

$$df(n) \geq g(n) \geq cf(n)$$

$g(n) \in \Theta(f(n))$ tradizionalmente si legge "$g(n)$ è di ordine $\Theta(f(n))$"; la g si comporta asintoticamente esattamente come f, per cui l'andamento di f caratterizza precisamente quello di g.
La funzione $h(n) = 3n^2 + 3n - 1$ è di ordine n^2 perchè esistono le costanti $c = 4$ e $n = 3$ per cui $h(n) < 4n^2$ per $n \geq 3$.
Notiamo che qualsiasi polinomio di grado k è di ordine n^k e, in accordo con la definizione, è anche di ordine n^j con $j > k$.
La funzione $g(n) = 3n^2 + 2n + 1$ è $O(n^2)$ e anche $O(n^3)$ ma è anche di ordine $\Omega(4n^2), \Omega(n^2), \Omega(n)$; inoltre è di ordine $\Theta(n^2)$, $\Theta(4n^2)$ ma non di ordine $\Theta(n)$ e neppure di ordine $\Theta(n^3)$.

Proprietà Seguono alcune proprietà:

- $g \in O(f)$ implica $(f + g) \in O(f)$
- $f_1 \in O(g_1), f_2 \in O(g_2)$ implica $f_1 + f_2 \in O(g_1 + g_2)$
- O e Ω sono relazioni riflessive e transitive
- Θ è una relazione di equivalenza
- $f \in O(g) \iff g \in \Omega(f)$

13.5 Valutazione della complessità

La valutazione del costo di un programma o di un algoritmo richiede la determinazione di una delimitazione superiore ed una inferiore. Diamo una serie di regole che aiutano a trovare la delimitazione superiore della complessità.

1. Supponiamo che il programma sia composto di due parti P e Q da eseguire sequenzialmente e che i costi di P e Q siano $S(n) = O(f(n))$ e $T(n) = O(g(n))$ rispettivamente. Allora il costo del programma è $O(\max(f(n), g(n)))$.
   Dimostriamo questo risultato.
   In questo caso il costo complessivo del programma è, chiaramente, pari a:

   $$S(n) + T(n)$$

   Inoltre si noti che:

   1.1. $S(n) = O(f(n)) \Rightarrow$ esistono costanti a' e n' tali che $S(n) < a'f(n) \quad \forall n > n'$;

   1.2. $T(n) = O(g(n)) \Rightarrow$ esistono costanti a'' e n'' tali che $S(n) < a''g(n) \quad \forall n > n''$.

   Ne deriva che, per $n > \max(n', n'')$:

   $$S(n) + T(n) < a'f(n) + a''g(n) < (a' + a'') \max(f(n), g(n))$$

   Quindi, $O(S(n) + T(n))$ è proprio $O(\max(f(n), g(n)))$.

2. Questa regola permette di valutare il costo del programma quando esso richiede più volte l'esecuzione di un insieme di istruzioni o l'attivazione di una procedura.
   Supponiamo che un programma richieda per k volte l'esecuzione di una istruzione composta o l'attivazione di una procedura, e sia $f_i(n)$ il costo relativo all'esecuzione i-esima, $i = 1, 2, \ldots, k$. Il costo complessivo del programma è pari a:
$$O\left(\sum_i, f_i(n)\right)$$

13.5.1 Istruzione dominante

Introduciamo ora il concetto di istruzione dominante che permette, in molti casi, di semplificare in modo drastico la valutazione della complessità di un problema.

Definizione 13.1: Operazione dominante
Sia dato un programma o un algoritmo P il cui costo di esecuzione è $t(n)$. Una istruzione di P si dice *istruzione* o *operazione dominante* se, per ogni intero n, essa viene eseguita, nel caso peggiore di input avente dimensione n, un numero di volte $d(n)$ che verifica la seguente condizione:

$$t(n) < ad(n) + b$$

per opportune costanti a e b.

In altre parole, un'istruzione dominante viene eseguita un numero di volte proporzionale al costo di esecuzione di tutto l'algoritmo. È importante osservare che in un programma, più istruzioni possono essere dominanti, ma può anche accadere che il programma non contenga affatto istruzioni dominanti.
Regola 3. Supponiamo che un programma contenga un'istruzione dominante che, nel caso peggiore di input di dimensione n, viene eseguita $d(n)$ volte. La delimitazione superiore alla complessità del programma è $O(d(n))$.
Per individuare un'istruzione dominante è sufficiente, in molti casi, esaminare le operazioni che sono contenute nei cicli più interni del programma.

13.6 Classi di complessità

Grazie all'analisi della complessità asintotica possiamo categorizzare gli algoritmi nelle seguenti classi di complessità:

costante	$O(1)$
logaritmica	$O(\log n)$
lineare	$O(n)$
nlog	$O(n \log n)$
quadratica	$O(n^2)$
cubica	$O(n^3)$
esponenziale	$O(a^n) \quad a > 1$

Gli algoritmi con complessità costante sono più efficienti di quelli con complessità logaritmica che a loro volta sono più efficienti di quelli con complessità lineare e così via.

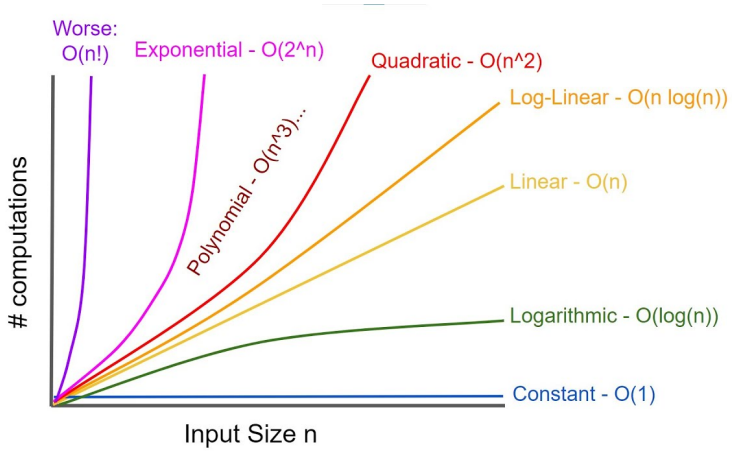


Figura 13.1: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < \cdots < O(a^n)$

La distinzione di maggiore interesse in teoria della complessità è fra:

- algoritmi *polinomiali* con complessità $O(n^k)$ con $k > 0$;
- algoritmi *esponenziali* con complessità $O(a^{g(n)})$ con $a > 1$ e $g(n)$ funzione crescente.

Gli algoritmi di importanza pratica sono quelli polinomiali.
Ad esempio, ordinare un elenco di elementi è un compito molto frequente:

- ordinare numeri in modo crescente;
- ordinare numeri in modo decrescente;
- ordinare stringhe in modo alfabetico.

Vi sono molti modi per ordinare un elenco, ad esempio, il selection sort è uno dei più facili, facili da capire e da programmare, ma non è il più efficiente.
Per ordinare un array di interi in modo crescente, si cerca nell'array il numero più piccolo e lo si scambia col primo elemento dell'array; la parte ordinata dell'array è ora il primo elemento, mentre quella non ancora ordinata sono i rimanenti elementi.
Dopodichè, si cerca nella parte non ordinata il numero più piccolo e lo si cambia con il secondo elemento dell'array e questa ricerca viene ripetuta fino a quando tutti gli elementi sono al posto giusto.

```
1     public void selectionSort(int[] a) {
2       int i,j, indexOfNextSmallest, min, temp;
3       for (i = 0; i < a.length - 1; i++) {
4         min = a[i];
5         indexOfNextSmallest = i;
6         for (j = i + 1; j < a.length; j++) {
7           if (a[j] < min) {
8             min = a[j];
9             indexOfNextSmalles = j;
10          }
11        }
12        temp = a[i];
13        a[i] = a[indexOfNextSmallest];
14        a[indexOfNextSmallest] = temp;
15      }
16    }
17
```

L'ordinamento per selezione ha un ciclo più esterno e uno più interno con proprietà simili, sebbene con scopi differenti. Il ciclo più esterno viene eseguito una volta per ogni valore nell'array e quello più interno confronta il valore scelto dal ciclo più esterno con molti, se non tuti, i valori rimanenti nell'array.
Quindi eseguire n^2 ove n è il valore di elementi dell'array. L'algoritmo di selection sort è quindi di ordine n^2.

Per valutare il costo di un programma a partire dal costo delle sue componenti va affrontato il problema del costo con sottoprogrammi ripetuti.
Sia A un programma la cui esecuzione consiste nell'esecuzione ripetuta di chiamate di funzione il cui costo all'i-esima ripetizione è $f_i(n)$ dove n è la dimensione dell'input di A. Sia $g(n)$ il numero di ripetizione legate alla dimensione dell'input. Il costo del programma $f_A(n)$ è dato da:

$$O\left(\sum_{i=1}^{g(n)} f_i(n)\right)$$

Ad esempio, consideriamo la funzione in grado di determinare se tutti gli elementi di un vettore w siano presenti in un altro vettore v. Per la ricerca si può utilizzare una banale ricerca sequenziale.

```
1       bool cercaInTutti(int v[], int w[]) {
2         for (int i = 0; i < v.length; i++)
3           if (!RicercaSequenziale(v, w[i]))
4             return false;
5         return true;
6       }
7
```

Il numero massimo di chiamate al sottoprogramma *RicercaSequenziale()* è pari a $v.length = n$ e si ha quando tutti gli elementi di w vengono trovati in v. Poichè il costo dell'esecuzione del sottoprogramma è delimitato superiormente da $O(n)$ si può concludere che *CercaInTutti()* è delimitato superiormente da $O(n^2)$.

13.7 Costi per ADT

Da quanto detto si deduce che, poichè gli operatori sulle strutture dati definite da noi sono, di fatto, sottoprogrammi che hanno una loro complessità e vengono applicati più volte, il costo di una realizzazione piuttosto che un'altra incide sicuramente sul costo complessivo del programma che utilizza quegli operatori.
Ad esempio, la complessità degli operatori su lista realizzati con corsori/puntatori è costante $O(1)$ per tutti gli operatori l'occupazione di memoria è $O(maxlung)$ per tutte le liste.
Per la realizzazione con cursori va però considerata la funzione INIZIALIZZALISTALIBERA.
La complessità della procedura di fusione di liste dipende dalla complessità degli operatori su lista utlizzati:

1. CREALISTA;
2. PRIMOLISTA;
3. FINELISTA;
4. LEGGILISTA;
5. INSLISTA;
6. SUCCLISTA.

Nella realizzazione con puntatori e doppi puntatori la complessità di questi operatori è $O(1)$. Nella realizzazione con cursori occorre tenere presente il costo della procedura INIZIALIZZALISTALIBERA che compunque è eseguita fuori dalla procedura di fusione e quindi non va conteggiato ai fini della complessità della procedura.
Consideriamo come operazione dominante il confronto di $elem1$ con $elem2$. Allora la complessità dipende dalla configurazione dei dati di ingresso. In particolare, dette m ed n le lunghezze rispettivamente di $Lista1$ e $Lista2$, possiamo distinguere i seguenti casi:

- Caso ottimo: tutti gli elementi della lista più corta sono minori degli elementi della lista più lunga. Si effettueranno tanti confronti quanti sono gli elementi della lista più corta:

$$f_{ott}(m, n) = \min m, n$$

- Caso pessimo: l'inserimento dell'ultimo elemento di una delle due liste può avvenire solo dopo l'inserimento del penultimo elemento dell'altra lista. Il numero di confroni è pari a:

$$f_{pess}(m, n) = m + n - 1$$

- Caso medio. Senza perdere di generalità supponiamo essere $m \leq n$. Consideriamo allora i seguenti eventi:
  - E_k: l'ultimo elemento di $Lista1$ è inserito prima del k-esimo elemento di $Lista2$ per $k = 1, 2, \ldots, n$.

– Suppniamo che i vari eventi siano tutti equiprobabili (poichè non abbiamo informazioni sufficienti per sostenere il contrario, ipotizziamo una distribuzione uniforme), allora:

$$P(E_k) = \frac{1}{n}$$

poichè i diversi eventi sono n.

Il costo, espresso in termini di numero di confronti, associato ad ogni evento è dato da:

$$costo(E_k) = m + k - 1$$

ove E_1 è il caso ottimo e E_n il caso pessimo.
Allora la complessità nel caso medio sarà data da:

$$\begin{aligned} f_{med}(m,n) &= \sum_{k=1,\dots,n} P(E_k) costo(E_k) \\ &= \frac{1}{n} \sum_{k=1,\dots n} (m + k - 1) \\ &= \frac{1}{n} \left(\sum_{k=1,\dots,n} m + \sum_{k=1,\dots,n} k - \sum_{k=1,\dots,n} 1 \right) \\ &= \frac{1}{n} \left(mn + \frac{n(n+1)}{2} - n \right) \\ &= m + \frac{(n+1)}{2} - 1 \end{aligned}$$

Capitolo 14

Tencniche algoritmiche

14.1 Classificazione dei problemi

I problemi si possono classificare in:

- problemi di ricerca;
- problemi di decisione;
- problemi di ottimizzazione.

Nella fase di concettualizzazione si fornisce una specifica del problema, si stabiliscono le caratteristiche strutturali del problema senza far riferimento ai metodi per la risoluzione.
Nella fase di realizzazione si passa dalla specifica del problema alle scelte per la sua risoluzione, che coinvolgono la definizione dell'*algoritmo risolutivo* e la *realizzazione del programma.*
La specifica del problema si suddivide in diverse fasi:

- **la scelta dell'input**: si stabilisce che i valori di alcune variabili in gioco siano i dati di ingresso del problema. L'insieme di tali valori è lo *spazio di input* del problema;
- **la scelta dello scopo della risoluzione**: si stabilisce che i valori di alcune variabili in gioco rappresentino le soluzioni del problema. L'insieme di tali valori è lo *spazio delle soluzioni*;

- L'identificazione del legame che i **vincoli del problema** impongono tra i valori dello spazio di input e i valori dello spazio delle soluzioni. Ciò conduce alla definizione della *relazione caratteristica* del problema;
- la determinazione di quali **informazioni in uscita** si vuole che il processo risolutivo del problema produca. Queste informazioni stabiliscono lo *spazio di output* e il *quesito* del problema.

Ad esempio, se vogliamo determinare la posizione di un numero interno in un vettore di interi, le variabili in gioco sono: un vettore **v**, un indice **k**, un numero intero **m**.
Lo *spazio di input* è l'insieme delle possibili coppie formate da un vettore v e un numero intero m. Lo *spazio delle soluzioni* è l'insieme degli indici di ogni possibile vettore. La *relazione caratteristica* associa ad ogni coppia (v, m) una posizione k se e solo se il k-esimo elemento di v coincide con m. La relazione la chiamiamo R_{vett}. Lo *spazio di output* contiene lo spazio delle soluzioni più un particolare simbolo che denota l'assenza di soluzioni. Il *quesito* stabilisce che, per ogni manifestazione che ammette soluzione, vogliamo come output un qualunque elemento dello spazio delle soluzioni legato a quella manifestazione della relazione caratteristica.

Definizione 14.1: Specifica del problema
La specifica del problema è una quintupla $< I, S, R, O, Q >$ ove:

- I è lo spazio di input;
- S è lo spazio delle soluzioni;
- $R \subseteq I \times S$ è una reazione su I ed S detta relazione caratteristica;
- O è lo spazio dell'output;
- Q è una regola, detto *quesito*, che sulla base della relazione caratteristica R consente di definire una relazione su I e O, $R_Q \subseteq I \times O$.

Una particolare istanza del problema si ottiene ogni volta che scegliamo un particolare valore dei dati di ingresso. In relazione ad un problema e ad una sua istanza diamo la nozione di **soluzione**.

Definizione 14.2: Soluzione di un problema
Sia $P =< I, S, R, O, Q >$ un problema e sia $i \in I$ una sua istanza, una soluzione S_i di P per i è un elemento di S per cui vale:

$$< i, S_i > \in R$$

Definizione 14.3: Rispostaza ad un problema
Sia $P =< I, S, R, O, Q >$ un problema e sia i una sua istanza, una *risposta* r_i a P per i è un elemento di O per cui vale:

$$< i, r_i > \in R_Q$$

dove R_Q è la relazione su I e O definita in base ad R applicando la regola Q.

14.1.1 Problemi di ricerca

Tra tutte le possibili soluzioni siamo interessati a conoscere una qualunque delle soluzioni del problema per l'istanza che stiamo considerando, oppure, nel caso non esista soluzione, siamo interessati a saperlo. Introduciamo nello spazio di output il simbolo $\perp$ che individua l'*assenza di soluzione*.

Definizione 14.4: Problema di ricerca
Un *problema di ricerca* P è un problema specificato con una quintupla del tipo:

$$< I, S, R, S \cup \{\perp\}, q_{ric} >$$

ove q_{ric} è la regola che definisce, in base ad R, la relazione R_{qric} contenente tutti e soli i seguenti elementi:

- ogni coppia contenuta in R;
- ogni coppia $< i, \perp >$ per ogni istanza i di P per la quale P non ha soluzioni.

Ad esempio, il problema della definizione della posizione di un intero in un vettore di interi è un problema di ricerca:

$$< I, S, R, S \cup \{\perp\}, q_{ric} >$$

ove:

- I è l'insieme delle coppie $< v, m >$;

- S è l'insieme degli interi positivi (indici);
- R è la relazione R_{vett}.

Siamo interessati, per ogni istanza, a calcolare una delle soluzioni oppure a segnalare il fatto che non ne esistono. Si dice anche: "trovare una soluzione ammissibile, cioè che soddisfi la relazione".

Esempio : Problema delle regine

Dato un intero positivo N si determini un posizionamento di N regine in una schiacchiera $N \times N$ tale che nessuna regina minacci qualche altra regina.
La specifica è una quintupla $< \mathbb{N}^+, D, R, D \cup \{\perp\}, q_{ric} >$ ove:

- $\mathbb{N}^+$ è l'insieme di numeri interi positivi;
- D è l'insieme delle disposizioni di regine in scacchiere di ogni dimensione ove il numero di regine è uguale alla dimensione della scacchiera;
- R è la relazione che contiene tutte e sole le coppie $< x, y >$ in cui $x \in \mathbb{N}^+$ e y è una disposizione di x regine nella scacchiera $x * x$.

Esempio : Ordinamento crescente di un vettore di interi

Fornire la specifica in termini di input, spazio delle soluzioni, relazione caratteristica, spazio di output e quesito del problema.
Lo *spazio degli input* è l'insieme di tutti i vettori di interi.
Lo *spazio delle soluzioni* è l'insieme di tutte le possibili sequenze di indici di un vettore (per ogni intero positivo n, l'insieme di tutte le possibili permutazioni da 1 ad n).
La *relazione caratteristica* è l'insieme di tutte e sole le coppie $< V, W >$ tale che se $V = [v_1, \ldots, v_n]$ allora $W = [w_1, \ldots, w_n]$ e dati due indici qualunque q ed m, dove $1 \leq q \leq n$ e $1 \leq m \leq n$ vale:

$$V_{W_q} > V_{W_m} \iff q \geq m$$

Ad esempio, se $V = [7, 1, 3, 4]$ allora $W = [2, 3, 4, 1]$ e $V_{W_2} > V_{W_1}$

14.1.2 Problemi di decisione

La risposta è vero o falso a seconda che il dato di ingresso soddisfi o meno una certa proprietà. Dunque lo spazio di output contiene solo i due valori di verità.

Definizione 14.5: Problema di decisione
Un problema di decisione P è un problema specificato con una quintupla del tipo:

$$< I, S, R, \{TRUE, FALSE\}, q_{dec} >$$

dove il quesito q_{dec} definisce la funzione:

$$R_{qdec} : I \rightarrow \{TRUE, FALSE\}$$

Tale che per ogni istanza i di P, $R_{qdec}(i)$ vale:

- TRUE se $\exists s \in S \mid < i, s > \in R$;
- FALSE altrimenti.

In linea di principio, un problema di decisione può essere risolto mediante due approcci:

1. cercando una risposta al corrispondente problema di ricerca $P' =< I, S, R, S \cup \{\perp\}, q_{ric} >$ detto *problema sottostante*, sfruttando il fatto che dato un $i \in I$, la risposta a P per i è FALSE solo se la risposta a P' coincide con $\perp$, mentre in ogni altro caso la risposta a P è TRUE (*approccio costruttivo*);
2. determinando la risposta apropriata a P senza ottenere una soluzione al corrispondente problema di ricerca sottostante (*approccio non costruttivo*).

Esempio : Problema di partizionamento
Sono dati k numeri interi positivi, $n_1, \ldots, n_k$ la cui somma è $2m$ per un certo intero m. Decidere se i k numeri possano essere ripartiti in due gruppi in modo che la somma dei componenti di ogni gruppo sia m.
Dunque se:

$$n_{a_1}, \ldots, n_{a_p} \text{ e } n_{b_1}, \ldots, n_{b_q}$$

Sono i due gruppi di numeri (cioè $p + q = k$), deve essere:

$$\sum_{i=1}^{p} n_{a_i} = \sum_{j=1}^{q} n_{b_j} = m$$

Lo *spazio di input* è l'insieme I di tutti i possibili insiemi di interi la cui somma è pari.
Lo *spazio delle soluzioni* è l'insieme S di tutte le possibili coppie di insiemi di interi.
La *relazione caratteristica* è l'insieme R di tutte e sole le coppie $< x, y >$ tale che:

- $x \in I, y \in S$;
- $y =< y_1, y_2 >$ è una coppia di insiemi tale che:

$$y_1 \cap y_2 = \emptyset \wedge y_1 \cup y_2 = x$$

e tale che

$$\sum_i y_{1_i} = \sum_j y_{2_j}$$

Ad esempio il vettore $V = [1, 2, 2, 4, 7]$ viene partizionato in: $[1, 2]$ e $[2, 2, 4]$.

14.1.3 Problemi di ottimizzazione

Alle soluzioni ammissibili è associata una misura (costo, obiettivo): risolvere il problema non significa trovare una qualunque soluzione, ma la migliore soluzione seconda la misura o criterio di preferenza fissato.

Definizione 14.6: Problema di ottimizzazione
Un problema di ottimizzazione P è un problema specificato con una quintupla del tipo:

$$< I, S, R, S \cup \{\perp\}, q_{ott}(M, m, \subseteq) >$$

ove:

- M è un insieme qualsiasi;
- m è una funzione del tipo $I \times S \rightarrow M$ detta *funzione obiettivo* di P; per una certa istanza $i \in I$, il valore $m(i, s)$ rappresenta una misura dell'elemento S nello spazio delle soluzioni;
- $\subseteq$ è una relazione di ordinamento su M ($x \subseteq y$ indica "x è migliore di y");
- $q_{ott}(M, m, \subseteq)$ è una regola che definisce, in base ad R, la relazione $R_{qott} \subseteq I \times S$ su I ed S così da individuare:
  - una coppia $(i, s) \forall i \mid \nexists s'$ associata ad i migliore;
  - una coppia $(i, \perp) \forall i$ per la quale P non ha soluzioni.

Esempio : Tre differenti problemi per uno stesso ambito

Dato un insieme W di parole, determinare una parola $A \in W$ tale che non esistano altre parole in W:

- P_1 maggiori di A secondo l'ordine alfabetico;
- P_2 di lunghezza maggiore a quella di A;
- P_3 composte usando tutte le lettere dell'alfabeto che compongono A, più altre lettere.

Lo schema generale per i tre problemi è il seguente:

$$< 2^W, W, \{(X, y) \mid X \in 2^W \wedge y \in X\}, W \cup \{\perp\}, q_{ott}(M, m, \subseteq) >$$

ove W è l'insieme della parole, 2^W è l'insieme di tutti gli insiemi di parole e M è un differente insieme per ciascun problema.
Per il problema P_1 $M = W$ e la funzione obiettivo:

$$m : 2^W \times W \to M \quad \forall i \in I \; m(i, W) = W$$

e la relazione $\subseteq$ è la relazione di ordinamneto alfabetico.
Per il problema P_2 abbiamo che $M = N$ e la funzione obiettivo:

$$m : 2^W \times W \to M \quad \forall i \in I \; m(i, W) \text{ è la lunghezza di } W$$

e la relazione $\subseteq$ è la relazione di ordinamento $\geq$ tra numeri.
Per il problema P_3 M è l'insieme di tutti gli insiemi di lettere dell'alfabeto e la funzione obiettivo:

$m : 2^W \times W \to M \quad \forall i \in I \; m(i, W)$ restituisce l'insieme delle lettere che compongono W

e la relazione è quella di contenmento $\supseteq$ tra insiemi.
Ad esempio sia l'istanza per i tre problemi:

$$\{"TORRE", "ERRORE", "ZERO", "TENORE"\}$$

P_1 ha una soluzione ottima "ZERO".
P_2 ha sue soluzioni ottimali "ERRORE" e "TENORE".
P_3 ha due soluzioni ottimali "TENORE" e "ZERO".

Esempio

Dato un albero i cui nodi sono etichettato con valori interi non negativi, trovare un livello dell'albero il cui peso è massimo, dove per peso di un livello dell'albero si intende la somma delle etichette dei nodi di quel livello.
Lo *spazio degli input* è un insieme I degli alberi i cui nodi sono etichettati con valori interi non negativi.
Lo *spazio delle soluzioni* è un insieme S degli interi non negativi.
La *relazione caratteristica* è l'insieme delle coppie (T, n) dove T è l'albero di profondità p e $n \leq p$.
Lo *spazio di output* è $S \cup \{\perp\}$.
Il *quesito di ottimizzazione* è $q_{ott}(N, m, \geq)$ dove:

$$m : I \times S \to N$$

è la funzione tale che $m(T, n)$ è la somma delle etichette dei nodi in T di livello n.

Esempio : Problema dello zaino

Un ladro durante una rapina in un negozio si trova di fronte a n articoli: l'articolo i-esimo ha un valore di P_i euro e un peso di C_i chilogrammi, dove sia P_i che C_i, per semplicità, sono interi. Il ladro vuole prendere gli articoli di maggior valore ma il suo zaino può supportare un peso massimo di B chilogrammi. Cosa gli conviene fare?
Per formalizzare il problema occorre considerare $2n + 1$ interi positivi:

$$P_1, \ldots, P_n, C_1, \ldots, C_n, B$$

ove $P_1, \ldots, P_n$ sono detti *profitti*, $C_1, \ldots, C_n$ sono i *costi* e B è il *budget*.
Questo problema è un'astrazione di molti prolbemi reali.
Dati $2n + 1$ interi positivi: $P_1, \ldots, P_n, C_1, \ldots, C_n, B$ trovare n valori interi $X_1, \ldots, X_n$ tale che:

$$X_i \in \{0, 1\} \quad 1 \leq i \leq n$$

ove $\sum_{i=1}^{n} P_i X_i$ abbia valore massimo e $\sum_{i=1}^{n} C_i X_i \leq B$.
Lo *spazio di input* I è l'insieme delle coppie (n, Q) dove n è un intero positivo e Q è una sequenza di $2n + 1$ interi positvi. L'elemento di questo spazio è:

$$(n, (P_1, \ldots, P_n, C_1, \ldots, C_n, B))$$

Lo *spazio delle soluzioni* S è l'insieme delle sequenze finite:

$$< X_1, \ldots, X_n >$$

con $n \in N^+$ e $X_i \in \{0, 1\}$.
La *relazione caratteristica* è data dall'insieme delle coppie:

$$(n, (P_1, \ldots, P_n, C_1, \ldots, C_n)), (X_1, \ldots, X_n)$$

tali che:

$$\sum_{i=1}^{n} C_i X_i \leq B$$

Il *quesito* è $q_{ott}(N, m, \geq)$ dove:

$$m = \sum_{i=1}^{n} P_i X_i$$

Dunque, la *specifica di un problema* mira a determinare la struttura del problem, in modo indipendente da considerazioni relative al modo in cui il problema verrà risolto.
Lo *spazio delle soluzioni* è caratteristica generale del problema e non fornisce indicazioni sulle soluzioni per una generica istanza.
È ora necessario uno strumento che aiuti a "caratterizzare" le potenziali soluzioni ad una generica istanza di un problema, per orientare la scelta di un algoritmo risolutivo, il cosidetto **spazio di ricerca**.

14.2 Spazio di ricerca

Affinchè l'analisi di un problema aiuti a formulare un algoritmo risolutivo è necessario disporre di uno strumento concettuale che aiuti a caratterizzare le potenziali soluzioni di una generica istanza del problema.
Determinare lo *spazio di ricerca* per un problema P significa stabilire un metodo che, per ogni istanza i di P, consente di definire un insieme con associate due funzioni:

1. la *funzione di ammissibilità* che permette di verificare se un elemento dello spazio di ricerca corrisponde effettivamente ad una soluzione per i;

2. la *funzione di risposta* che permette di ottenere, dagli elementi dello spazio di ricerca, le corrispondenti risposte per i.

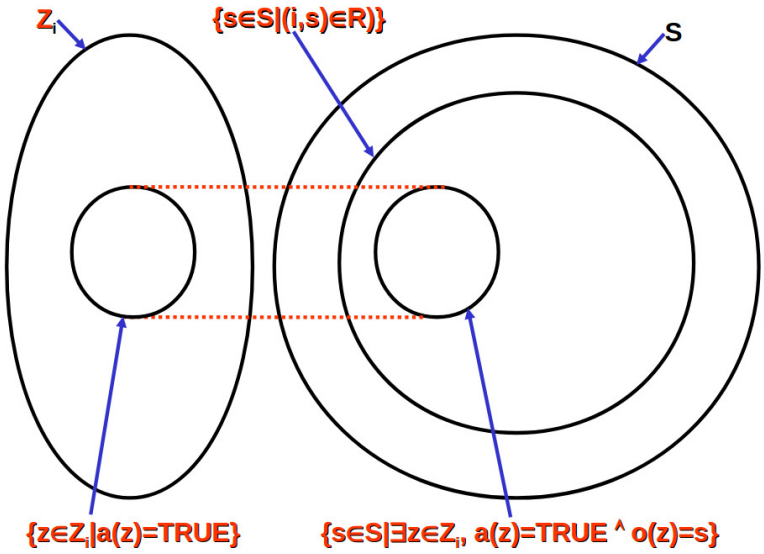
Definizione 14.7: Spazio di ricerca
Sia i un'istanza di un problema $P =< I, S, R, S \cup \{\perp\}, Q >$.
Uno *spazio di ricerca di P per i* è costituito da:

- un insieme Z_i con associate due funzioni:
  - la funzione di ammisibilità:
    $$a : Z_i \to \{TRUE, FALSE\}$$
  - la funzione di risposta:
    $$o : Z_i \to S$$

  che soddisfano le seguenti condizioni:
  1. per ogni elemento z di Z_i, $a(z) = TRUE \iff o(z)$ è soluzione di P per i;
  2. i ha risposta positiva se e solo se vi è almeno un elemento z di Z_i per cui $a(z) = TRUE$ e $o(z)$ è una risposta a P per i.
- un metodo per rappresentare ogni elemento di Z_i mediante una struttura dati e per esprimere la funzione di ammissibilità e di risposta in termini di tale struttura.

Lo spazio di ricerca associato associato ad una istanza i di P caratterizza le soluzioni e le corrispondenti risposte a P per i.

Struttura dello spazio di ricerca per problemi di ricerca con risposta positiva L'insieme $\{s \in S \mid (i, S) \in R\}$ rappresenta le soluzioni di P per l'istanza i (per un problema di ricerca questo insieme coincide con l'insieme delle risposte).
L'insieme $\{s \in S \mid \exists z \in Z_i, a(z) = TRUE \wedge o(z) = s\}$ è costituito dalle soluzioni di P per l'istanza i che sono rappresentate nello spazio di ricerca.
Lo spazio di ricerca non rappresenta necessariamente tutte le soluzioni ad i. Se esistono risposte positive per i basta che una sia rappresentata nello spazio di ricerca.



Struttura dello spazio di ricerca per i problemi di ottimizzazione Per i problemi di ottimizzazione valgono le seguenti osservazioni:

- la definizione di spazio di ricerca stabilisce che la funzione di ammissibilità deve assicurare che:

  $$\forall z \in Z_i \; a(z) = TRUE \iff (i, o(z)) \in R$$

  e queste sono condizioni relative alla relazione caratteristica e non al quesito.

- ottenere una risposta per una certa istanza significa calcolare una soluzione ottimale: lo spazio di ricerca deve includere almeno un elemento dal quale si possa ottenere una soluzione ottimale all'istanza.

Per un problema di ottimizzazione, l'insieme:

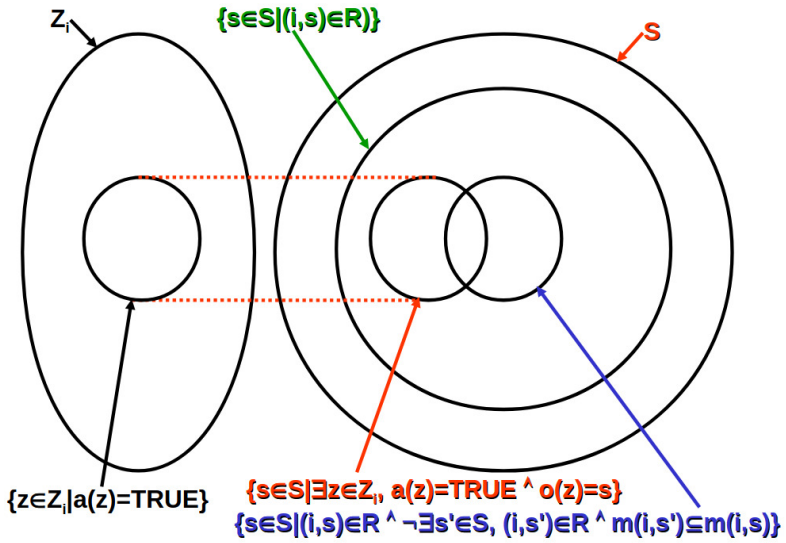
$$\{s \in S \mid (i, s) \in R \wedge \nexists s' \in S, (i, s') \in R' \wedge m(i, s') \subseteq m(i, s)\}$$

rappresenta le soluzioni ottimali di P per l'istanza i, cioè le risposte a P per i. Uno spazio di ricerca per i di P deve contenere la rappresentazione di almeno una tra le soluzioni ottimali se esistono, cioè non deve essere vuota l'intersezione tra l'insieme:

$$\{s \in S \mid (i, s) \in R \wedge \nexists s' \in S, (i, s') \in R' \wedge m(i, s') \subseteq m(i, s)\}$$

e

$$\{s \in S \mid \exists z \in Z_i, a(z) = TRUE \wedge o(z) = s\}$$



Esempio : Ricerca di un elemento in un vettore

Consideriamo una generica istanza del problema con input un vettore V di n elementi e un numero M.
Lo spazio di ricerca associato a tale istanza è l'insieme dei valori che sono indici del vettore, da 1 a n e la funzione di ammissibilità verifica, dato un j, se $V[j] = M$. La funzione di risposta restituisce j.
La struttura dati per rappresentare i singoli elementi dell'insieme è una semplice variabile di tipo intero.

Differenze tra spazio delle soluzioni e spazio di ricerca Lo *spazio delle soluzioni* è una componente della specifica del problema e fa globalmente riferimento a tutte le possibili soluzioni del problema.
Lo *spazio di ricerca* fornisce uno strumento per caratterizzare le soluzioni di ogni singola istanza del problema e determina anche, per ogni istnaza del problema, una struttura di dati e un meccanismo per verificare, tramite la formulazione della funzione di ammissibilità in termini di tale struttura, se una sua configurazione corrisponde o meno ad una soluzione per l'istanza e un metodo per derivare, tramite la formulazione della funzione di risposta, una risposta all'istanza.

Esistono diversi modi per determinare uno spazio di ricerca di un problema. I criteri per valutare la qualità di una scelta rispetto ad un'altra sono i seguenti.

- Lo spazio di ricerca deve caratterizzare le soluzioni ad una istanza in modo non ridondante: è opportuno scegliere una struttura dati che escluda a priori configurazioni che non corrispondono a priori ad alcuna soluzione.
- Lo spazio di ricerca non deve essere una banale riformulazione del problema e della sua relazione caratteristica nè deve nascondere la difficoltà di un problema, ma deve fornire elementi significativi per la comprensione della struttura del problema e delle sue soluzioni.

Esempio : Problema delle n regine

Sia i la dimensione della scacchiera.

- L'insieme che forma lo spazio di ricerca è l'insieme di tutte le possibili scacchiere di dimensione i in cui sono poste i regine.
- La funzione di ammissibilità verifica se la scacchiera soddisfa i vincoli del problema, cioè se non esistono nella scacchiera due regine piazzate nella stessa riga, colonna o diagonale.
- La funzione di risposta è la funzione identità.
- La struttura dati è una matrice $i \times i$ magari di caratteri in cui il carattere Q o $*$ indica la posizione della regina.

Adesso ci chiediamo se questa rappresentazione possa essere migliorata. Per escludere configurazioni con due regine sulla stessa riga rappresentiamo la configurazione di scacchiera mediante un vettore di dimensione i comn elementi interi compresi tra 1 ed i: se l'elemento in posizione j del vettore è h vuol dire che la regina della riga j è in colonna h.
Per escludere configurazioni errate basta escludere vettori che hanno valori uguali in posizioni diverse.
Allora l'insieme degli elementi dello spazio di ricerca coincide con tutti i possibili modi di memorizzare i valori da 1 ad i in un vettore V, ovvero tutte le possibili permutazioni di $(1, \ldots, i)$.
La funzione di ammissibilità verifica se il generico elemento dello spazio di ricerca, cioè il generico vettore V, non contenga due elementi in posizione h e k tali che:

$$|V[h] - V[k]| = |h - k|$$

ovvero due regine nella stessa diagonale.
La funzione di risposta ricostruisce semplicemente, a partire da un generico vettore, la corrispondente configurazione nella scacchiera.

14.3 Teniche algoritmiche

Le tecniche algoritmiche sono basate su diversi paradigmi di utilizzo dello spazio di ricerca e consideriamo:

- la tecnica di enumerazione;
- la tecnica di backtracking;
- la tecnica golosa o greedy;
- la tecnica divide et impera.

I paradigmi sono di tipo *selettivo* o *generativo*.
Dal paradigma *selettivo* sono create tecniche di progetto di algoritmi che, per l'istanza del problema presa in considerazione, visitano lo spazio di ricerca tentando di trovare un elemento ammissibile.
Ogni algoritmo fa riferimento all'intero spazio di ricerca che viene esplorato con sistematicità in una definita modalità. A questo paradigma appartiene la tecnica *enumerativa* e quella di *backtracking*.

Esempio : Ordinamento di un vettore

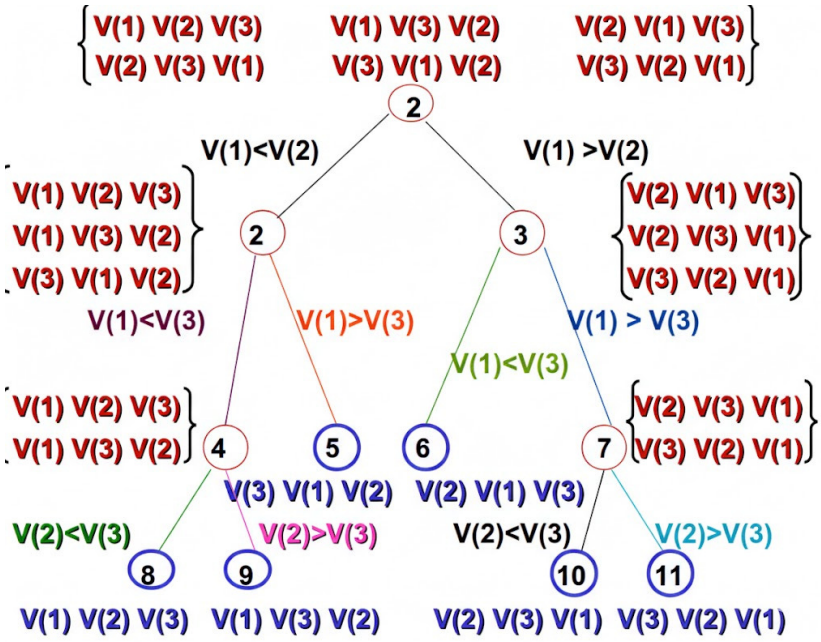
Dato un vettore di interi V, ordinarlo in ordine crescente.
Lo spazio di ricerca è costituito dalle permutazioni di V e la funzione di ammissibilità verifica che non esistano permutazioni tali che se $h > k$, sia:

$$V[h] < V[k]$$

La funzione di risposta è la funzione identità. Se N è la dimensione di V lo spazio di ricerca ha dimensione $N!$.

Un algoritmo *selettivo* effettua la visita dello spazio di ricerca per trovare una permutazione che soddisfi la condizione di ammissibilità. Dal paradigma *generativo* scaturiscono tecniche di progetto di algoritmi che generano direttamtente la soluzione senza selezionarla tra gli elementi dello spazio di ricerca.
In questo paradigma lo spazio di ricerca è considerato esclusivamente in fase di

progetto dell'algoritmo allo scopo di caratterizzare le soluzioni del problema e definire una strategia risolutiva diretta per ogni istanza. A questo paradigma appartengono la tecnica *greedy* e la tecnica *divide-et-impera*.
Un algoritmo *generativo* deriva la soluzione con un procedimento diretto sull'istanza, senza visitare lo spazio di ricerca.
Rappresentiamo il procedimento di esplorazione dello spazio di ricerca mediante un albero, che ha nei nodi i confronti utili a scegliere le permutazioni che sono soluzioni. Lo spazio viene partizionato e ciascuna foglia dell'albero rappresenta un possibile ordinamento. La profondità dell'albero rappresenta il numero di confronti necessario nel caso peggiore.



14.3.1 Tecnica enumerativa

La tecnica enumerativa è basata sulla sistematica ispezione, elemento per elemento, dello spazio di ricerca associato ad un'istanza di un problema, è una tecnica che **garantisce**, se lo spazio di ricerca è finito, una terminazione.
Per un problema di ricerca si termina quando si è individuato un elemento ammissibile o quando è esaurito lo spazio di ricerca.
Per un problema di ottimizzazione si dovrà confrontare, ai fini della selezione, tutti gli elementi ammissibili e la terminazione è comunque data dall'esaurimen-

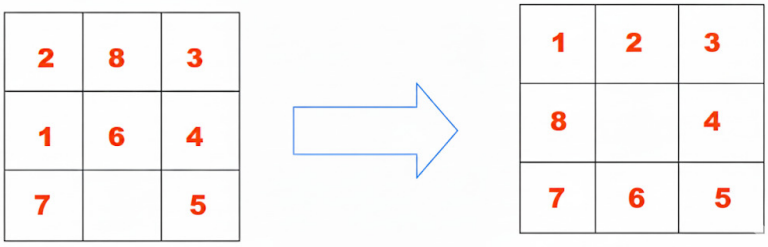
to dello spazio di ricerca.
Per garantire la possibilità di una visita sistematica di uno spazio di ricerca si associa ad esso una relazione di ordinamento totale in modo da definire per lo spazio Z_i associato ad i:

- un metodo per stabilire il primo elemento da considerare;
- un metodo per stabilire l'elemento successivo;
- un metodo per verificare se si sono esaminati tutti gli elementi.

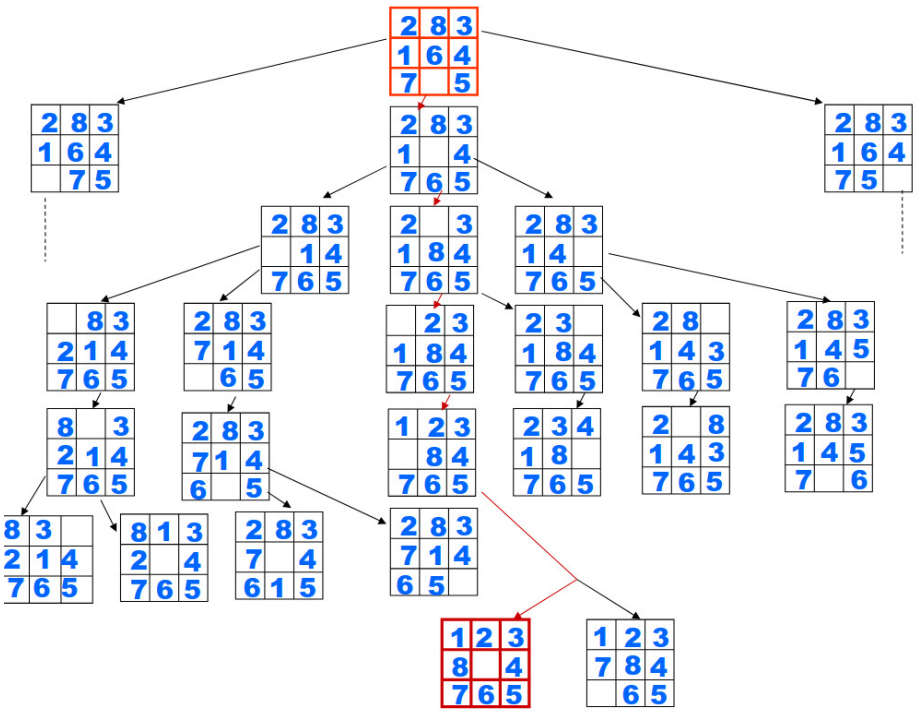
Algoritmo enumerativo per problemi di ricerca Un algoritmo enumerativo per i problemi di ricerca è il seguente:

1. Considera il primo elemento x dello spazio di ricerca;
2. Se $a(x) = TRUE$, allora fornisci $O(x)$ come risultato;
3. Se tutti gli elementi dello spazio di ricerca sono stati esaminati, fornisci $\perp$ come risultato;
4. Altrimenti considera come nuovo x l'elemento successivo dello spazio di ricerca e ripeti dal punto 2.

Esempio : Gioco dell'otto



I tasselli numerati devono essere disposti e ordinati lungo i bordi: la soluzione può essere ricercata esaminando tutte le possibili disposizioni generate attraverso mosse successive. Il procedimento di ricerca è rappresentabile mediante un albero che ha nei nodi le configurazioni possibili generate mossa dopo mossa.



Algoritmo enumerativo per problemi di ottimizzazione Un algoritmo enumerativo per i problemi di ottimizzazione è il seguente:

1. Considera il primo elemento x dello spazio di ricerca;
2. Se $a(x) = TRUE$ e x è la prima soluzione trovata, oppure se $a(x) = TRUE$ e x è la migliore della soluzione ottima corrente, poni la soluzione ottima corrente pari a x;
3. Se tutti gli elementi dello spazio di ricerca sono stati considerati, allora fonisci come risultato $\perp$ se non sono state trovate soluzioni ammissibili, oppure $O(x)$ se x è la soluzione ottima corrente;
4. Altrimenti considera come nuovo x l'elemento successivo dello spazio di ricerca e ripeti dal punto 2.

Esempio : Gioco dell'otto

Se consideriamo come problema il gioco dell'otto, potremmo porci l'obiettivo non solo di trovare una soluzione qualsiasi ma di trovarne una ottima, per esempio, sulla base del criterio "il prima possibile". Questo si traduce nella

possibilità di valutare il numero di mosse che ci porta alla soluzione e nell'esplorare l'intero albero alla ricerca del cammino più breve che ci porta alla soluzione.

14.3.2 La tecnica del backtracking

Nella tecnica di backtracking la generazione degli elementi dello spazio di ricerca da visitare avviene secondo un processo suddiviso in stadi e basato sul fatto che:

- ogni elemento dello spazio di ricerca è considerato costituito da diverse componenti e ad ogni stadio viene scelta una componente;
- se ogni elemento dello spazio di ricerca è strutturato in n componenti, dopo i stadi, ove $i < n$, si è costruita una *soluzione parziale*. La caratteristica della tecnica di backtracking è che in molti problemi è possibile giungere alla conclusione che la soluzione parziale generata è fallimentare. Riconosciuto ciò l'algoritmo interrompe il processo di costruzione e tenta altre vie.

Esempio : String matching

Il problema dello string matching prevede di trovare un'occorrenza di una sequenza P di m caratteri ("pattern") in un'altra sequenza T di n caratteri ("testo"). Le due stringhe P e T sono formate da caratteri tratti dallo stesso alfabeto ed m non supera n.
Un algoritmo banale consiste nel cerca di riconsocere il pattern a partire dalla prima posizione del testo e se il pattern non è intermente riconosciuto, ripetere il procedimento a partire dalla posizione successiva nel testo continuando fino al riconoscimento dell'intero pattern o all'esaurimento del testo.
Se il pattern non è riconosciuto l'algoritmo torna a ripetere il procedimento dalla posizione che segue.
Le stringhe P e T si possono realizzare con due vettori di caratteri e il programma può realizzarsi mediante una funzione che restituisce la posizione di T a partire dalla quale si trova la prima occorrenza di P (se il pattern occorre nel testo) oppure $n + 1$ se P non occorre in T.

```
1      function string1(var P: array; var T: array; var n: integer; var m:
       integer): integer;
2        i := 1;
3        j := 1;
4        k := 1;
5        while (i <= n) and (j <= m) do
6          if T[i] = P[j] then
7            i := i + 1;
```

```
            j := j + 1;
          else
            k := k + 1;
            i := k;
            j := 1;
          if j > m then
            return k
          else
            return i

```

La funzione effettua backtracking sugli indici i e j.
Un altro algoritmo per lo string matching è l'algoritmo di Knuth-Morris-Pratt che trae vantaggio dai confronti già fatti precedentemente sul pattern. Infatti se:

$$T = 110111011001$$

$$P = 110110$$

I primi 5 caratteri di P sono sono uguali ai primi 5 di T, solo il sesto è diverso. Al momento del primo backtrack risulta $i = j = 6$ e ripartire con $i = 2$ e $j = 1$ corrisponde a traslare di una posizione il pattern rispetto al testo.
Vale la pena di verificare che nella sequenza riconosciuta di P: **11011**1 i primi 4 caratteri di P non coincidono con gli ultimi 4 caratteri della sottosequenza riconosciuta di T, nè i primi 3 con gli ultimi 3, ma i primi 2 coincidono con gli ultimi 2.
Quindi, non vale la pena ripartire con $i = 2$ o $i = 3$ ma con $i = 4$ direttamente.
Poichè i due caratteri di P a questo punto coincidono con quelli di T è conveniente ripartire col backtrack con $i = 6$ e $j = 3$ ove 6 è proprio il vecchio valore di i al momento del backtrack, dunque è inutile effettuare backtrack su i. basta farlo su j portandolo dal 6 originario all'attuale 3.
Se si considerano due "copie" dei primi $j - 1$ caratteri del pattern: si dispongano una sotto l'altra in modo che il primo carattere della copia inferiore sia esattamente sotto il secondo carattere della superiore.
Se tutti i caratteri sovrapposti nelle due copie non sono uguali, si traslano di una posizione a destra tutti i caratteri della copia inferiore. Il procedimento si arresta non appena i caratteri sono identici o quando non ci sono più caratteri sovrapposti.
Il nuovo valore di backtrack da assegnare a j $SUCC(j)$ è uguale al numero di caratteri sovrapposti + 1.

```
 procedure CALC_SUCC(var P: array; var SUCC: array; var m: integer);
   j := 1;
   h := 0;
   succ[1] := 0;
   while j <= m do
     if h = 0 then
       j := j + 1;
       h := 1;
       if j <= m then
         if P[j] = P[1] then
           SUCC[j] := 0;
         else
           SUCC[j] := 1;
       else
         if P[j] = P[h] then
           j := j + 1;
           h := h + 1;
           if j <= m then
             if P[j] = P[h] then
               SUCC[j] := SUCC[h];
           else
             SUCC[j] := h;
         else
           h := SUCC[h]

 function string2(var P: array; var T: array, var n: integer, var m:
integer): integer;
   i := 1;
   j := 1;
   CALC_SUCC(p, SUCC, m);
   while (i <= n) and (j <= m) do
     if j = 0 then
       i := i + 1;
       j := 1;
     else
       if T[i] = P[j] then
         i := i +1;
         j := j + 1;
       else
         j := SUCC(j);
     if j > m then
       return i - m;
     else
       return i;
```

Esempio : Problema delle 8 regine con algoritmo di backtracking

Come si è detto possiamo esprimere la soluzione attraverso una permutazione dei numeri da 1 a 8. La soluzione si può esprimere mediante il vettore V definito come segue:

```
type posizione = [1..8];
  vettore = array[1..8] of posizione;
var V: vettore;

procedure otto_regine(var V: vettore; var successo: boolean);
  k := 1;
  V[k] := 1;
  successo := false;
  repeat
    VERIFICA(a); /* Programma che controlla il vettore */
    if (a) then
      repeat
        if (V[k] < 8) then begin
          b := true;
          V[k] := V[k] + 1;
        else /* Backtracking */
          b := false;
          k := k - 1;
      until b or (k = 0);
    else if (k = 8) then
      successo := true;
    else
      k := k + 1;
      V[k] := 1;
  until successo or (k = 0);

```

I vincoli imposti sono:

- le componenti di V devono costituire una permutazione da 1 a 8; per ogni coppia di indici i e j vale $V[i] \neq V[j]$;
- non è possibile avere due elementi sulla stessa diagonale, cioè:

$$(i - j) \neq k - 1$$

$$(i + j) \neq k + 1$$

Il sottoprogramma verifica ha il compito di esaminare il vettore V e se il vettore viola uno dei vincoli rende TRUE la variabile a, altrimenti se i vincoli sono ambedue soddisfatti $a = FALSE$.

Albero di ricerca Sia P un problema e sia dato un metodo per associare ad ogni istanza i di P uno spazio di ricerca Z_i. Sia definito un modo per strutturare ogni elemento di Z_i in un numero finito di componenti.
L'albero di ricerca associato a i tramite Z_i è un albero tale che:

- la radice rappresenta una soluzione parziale fittizia;
- ogni nodo interno a livello j con $j > 0$, rappresenta una soluzione parziale S in cui sono state scelte le prime j componenti, ed ha tanti fili quanti sono i modi possibili di aggiungere ad S la $j+1$-esima componente.
- ogni foglia è un elemento di Z_i.

Per un albero di ricerca associato ad un'istanza i tramite Z_i è importante definire:

- un metodo per rappresentare ogni soluzione parziale;
- un metodo per stabilire se una soluzione parziale viola i vincoli del problema;
- una funzione di controllo del backtracking:

  $$C : \text{NODI} \rightarrow \{\text{TRUE, FALSE}\}$$

  detta *funzione di ammissibilità* tale che $C(N) = TRUE$ per ogni nodo interno N dell'albero, se e solo se la soluzione parziale viola i vincoli, mentre, se N è una foglia, $C(N) = TRUE$ quando il corrispondente elemento dello spazio di ricerca non è ammissibile.

Esempio : Partizionamento di un insieme

Dato un insieme $Y = \{y_1, y_2, \ldots, y_n\}$ di n interi positivi la cui somma è $2m$ vogliamo sapere se esiste un sottoinsieme di Y la cui somma sia pari a M.
Data l'istanza:

$$Y = \{8, 5, 1, 4\}$$

cerchiamo un sottoinsieme X la cui somma sia 9.
La soluzione può essere espressa mediante un vettore:

$$[x_1, \ldots, x_4]$$

ove $x_i \in \{0, 1\}$ e $x_i = 1 \iff y_i \in X$.
L'algoritmo di backtracking invece di generare tutte le quadruple $[x_1, \ldots, x_4]$ costruisce il vettore soluzione partendo dal vettore vuoto e aggiungendo una

componente alla volta.
Il comportamento dell'algoritmo si può descrivere mediante la rappresentazione ad albero dello spazio di ricerca.
La radice rappresenta il vettore vuoto, a livello 1 abbiamo i vettori che è possibile formare con la prima variabile $x_1 = 1$ e $x_1 = 0$ e così via.
Solo nelle foglie avrò la configurazione completa del vettore e solo alcune delle foglie rappresentano soluzioni ammissibili.
L'algoritmo di backtracking è un algoritmo di visita in ordine anticipato dell'albero di ricerca.

L'algoritmo di backtracking si riferisce all'albero di ricerca B_i associato all'istanza i di un problema. Si comporta come se effettuase una visista in profondità.

- Ogni volta che nella visita si analizza un nodo si applica la funzione di controllo del backtracking al nodo. Se la funzione restituisce TRUE, allora quel nodo e tutto il sottoalbero associato al nodo viene abbandonato e la visita continua.
- Se il problema è di ricerca, la visita termina quando si incontra una soluzione (una foglia con funzione di backtracking pari a false) oppure quando non esistono altri nodi da visitare.
- Se il problema è di ottimizzazione, l'algoritmo utilizza una variabile OTTIMO CORRENTE, che memorizza ad ogni passo il miglior elemento ammissibile. L'algoritmo non si interrompre al primo elemento ammissibile trovato, ma continua la visita aggiornando tale variabile.

Esempio : Backtracking sul problema dello zaino

L'albero di ricerca si ottiene considerando che, se n sono gli oggetti, ogni elemento dello spazio di ricerca si può costruire in n stadi dove all'$i-$esimo stadio si decide se includere o no, nel sottoinsieme rappresentato da X, l'oggetto i-esimo. L'albero di ricerca è un albero binario di profondità n.
Ogni soluzione parziale è rappresentata con un vettore e un indice, che indica lo stadio del processo: l'elemento i-esimo del vettore vale 1 se e solo se il i-esimo oggetto è nello zaino.
La funzione di controllo del backtracking su un nodo dell'albero, cui corrisponde la soluzione parziale $< X_1, \ldots, X_i >$, deve restituire il valore TRUE se le scelte fatte portano ad un costo totale già maggiore del budget cioè:

$$\sum_{k=1\ldots i} C_k X_k > B$$

La funzione, inoltre, vale TRUE se il massimo profitto ottenibile con le scelte già fatte è minore del profitto corrispondente all'ottimo corrente.

$$\sum_{I=1\ldots I} P_i X_i + \sum_{i=I+1,\ldots N} P_i < P_{ott}$$

14.4 La tecnica greedy

La tecnica greedy su applica principalmente a problemi di ottimizzazione e richiede che l'algoritmo esegua il processo di costruzione di un elemento dello spazio di ricerca in stadi e si basa sui seguenti principi:

- ad ogni stadio i, per la componente i-esima viene scelto il valore che, tra quelli ammissibili, risulta il migliore rispetto ad un determinato criterio;
- una volta fatta la scelta per la i-esima componente, si passa a considerare le altre, senza più tornare sulla decisione presa.

Lo schema di un algoritmo greedy presuppone che l'algoritmo acquisisca la rappresentazione di un'istanza del problema e disponga di un metodo per organizzare in stadi la costruzione di un elemento dello spazio di ricerca.

1. Poni i a 1 e inizializza z.
2. Determina l'insieme A dei valori ammissibili per la componente i-esima di z e se $A \neq \emptyset$ scegli il migiore in A, rispetto al criterio di preferenza fissato.
3. Se l'i-esimo stadio è l'ultimo allora termina e restituisce $o(z)$ come risultato.
4. Altrimenti incrementa i di 1 e torna al passo 2.

Esempio : Problema dello zaino (knapsack)

Come è noto, si dispone din un budget B e si deve cercare di massimizzare la rendita scegliendo tra n possibili investimenti, ciascuno dei quali caratterizzato da un profitto P_i e da un costo C_i. Dati $2n + 1$ interi positivi:

$$P_1, P_2, \ldots, P_n, C_1, C_2, \ldots, C_n, B$$

si vogliono trovare n valori:

$$X_1, X_2, \ldots, X_n$$

tali che $X_n \in \{0, 1\}$ e $\sum_{i=1}^{n} P_i X_i$ sia massima e $\sum_{i=1}^{n} C_i X_i \leq B$.
Consideriamo l'istanza $B = 5$ e tre elementi disponibili.

$\mathbf{i}$	$\mathbf{P_i}$	$\mathbf{C_i}$	$\mathbf{P_i/C_i}$
1	6	2	3
2	4	1	4
3	7	3	3.5

Il criterio di base per ordinare le variabili è quello di valutare sia il costo che il profitto P_i/C_i. Una volta effettuato l'ordinamento, si possono scegliere le variabili ponendole al massimo valore compatibile con i vincoli.
Il primo metodo consiste nel definire una funzione ZAINOGREEDY che ha come parametri:

- il numero n di oggetti denotati dai numeri da 0 a $n - 1$;
- il vettore P dei profitti;
- il vettore C dei costi;
- il budget B.

Il risultato viene restituito nel vettore X nel quale la i-esima componente vale 1 se l'oggetto i è incluso nella soluzione e 0 altrimenti.
Il secondo metodo consiste nell'ordinare in un vettore ausiliario V gli oggetti secondo l'ordine decrescente del rapporto P_i/C_i. Ogni elemento è in corrispondenza di un oggetto e contiene l'indice dell'oggetto in un apposito campo, oltre al rapporto P_i/C_i ad esso associato.
Dopo l'ordinamento, la funzione scandisce il vettore V, analizzando per ogni i l'oggetto in $V[i]$ e decidendo, in base alla compatibilità con il vincolo sul budget, se includerlo o meno nella soluzione.

Un algoritmo greedy permette di ottenere una soluzione mediante una sequenza di decisioni; in ogni passo viene presa la decisione che al momento appare migliore. Questa strategia euristica non garantisce sempre una soluzione ottima.
Come possiamo determinare se un algoritmo greedy è in grado di trovare la soluzione di un problema di ottimizzazione? I problemi che si prestano ad essere risolti con una strategia greedy presentano alcune caratteristiche:

- la proprietà della scelta greedy;
- la sottostruttura ottima.

La proprietà della scelta greedy assicura che si può ottenere una soluzione ottima globale prendendo decisioni che sono *ottimi locali*.
Un problema presenta una sottostruttura ottima se una soluzione ottima del problema contiene al suo interno una soluzione ottima dei sottoproblemi.
La tecnica greedy è spesso utile per problemi di "scheduling", in cui si hanno programmi da eseguire su un "processore" e si vuole l'ordine di esecuzione "ottimo" in base a un certo criterio.

Esempio : Problema di selezione di attività

Il prolema è quello dell'assegnamento di una risorsa condivisa da un certo numero di attività in competizione tra loro.
Sia $S = \{1, 2, \ldots, n\}$ un insieme di n attività che devono utilizzare una determinata risorsa che non può essere utilizzata contemporaneamente. Una generica attività k è caratterizzata da un tempo di inizio, detta attivazione I_k e un tempo di fine detto conclusione F_k con $I_k \leq F_k$.
Due attività k e j sono dette *compatibili* se gli intervalli $[I_k, F_k]$ e $[I_i, F_j]$ non si sovrappongono.
Il probema chiede di individuare un insieme contenente il massimo numero di attività mutuamente compatibili.
Si assume che le attività in ingresso siano ordinate in modo crescente rispetto al loro tempo di fine: $F_1 \leq F_2 \leq \cdots \leq F_n$.
Lo spazio di ricerca può essere così definito. Se $1, \ldots, n$ sono le attività relative a un'istanza i, allora lo spazio di ricerca è l'insieme di tutti i possibili sottoinsiemi di $\{1, \ldots, n\}$.
Ad ogni stadio l'algortimo greedy sceglie le attività k che, tra quelle ancora disponibili, rilascia per prima la risorsa condivisa, quindi con un F_k minore.
Una volta scelta l'attività k, si aggiorna l'insieme di attività disponibili, eliminando dall'insieme quelle incompatibili con k, cioè quelle che richiedono l'uso della risorsa nel tempo che intercorre tra I_k e F_k.

```
1  procedure greedy(var I: array, var F: array, var n: integer);
2  var A: insieme;
3  CREAINSIEME(A);
4  INSERISCI(1, A);
5  j := 1;
6  for k=2 to n do:
7    if I[k] >= F[j] then:
8      INSERISCI(k, A);
9      j := k;
10
```

Esempio : Percorso più breve in un grafo

Sia $G = (N, A)$ un grafo orientato etichettato negli archi con valori interi positivi. Trovare la lunghezza del percorso più breve che, dato un nodo $r \in N$, per ogni nodo $u \in N$, connetta r ad u.
L'idea è quella di calcolare, in ordine crescente, la lunghezza dei cammini minimi da r a tutti i nodi del grafo. Indichiamo con S l'insieme dei nodi di cui, ad un dato istante, si è già calcolato la lunghezza del cammino minimo da r.
Utilizziamo un vettore $DIST$ con tante componenti quanti sono i nodi del grafo, in modo che $DIST[i]$ rappresenti la lunghezza del cammino minimo tra quelli che vanno da r ad i passando solo per nodi contenuti in S. L'ipotesi di fondo è che le distanze siano interi positivi.
Osserviamo che se il prossimo cammino minimo da generare C è da r al nodo u, tutti i nodi sono in S. Infatti se un nodo k di C non appartenesse a S vi sarebbe un cammino da r a un nodo k non contenuto in S di lunghezza minore a quella di C, contraddicendo l'ipotesi che il prossimo cammino da generare sia C. La lunghezza di C e il nodo u sono facilmente individuabili; basta calcolare il valore minimo di $DIST[i]$ per $i \notin S$. Individuato u si inserisce in S e si aggiorna $DIST$ per i nodi che non sono in S.
In particolare, se per un certo nodo z connesso a u da $< u, z >$ con etichetta E, la somma $DIST[u] + E$ è minore di $DIST[z]$ allora a $DIST[z]$ va assegnato il nuovo valore $DIST[u] + E$.
Viene generato un albero di copertura T, radicato in r, che include un cammino da r ad ogni altro nodo. L'albero radicato T può essere rappresentato con un vettore di padri, inizializzato ad un albero "fittizio" in cui tutti i nodi sono figli di r connessi ad un arco fittizio etichettato con un valore maggiore di tutte le altre etichette.
Una soluzione ammissibile T è ottima se e solo se:

$$DIST[i] + C_{ij} = DIST[j] \quad \forall (i, j) \in T$$

e

$$DIST[i] + C_{ij} \geq DIST[j] \quad \forall \text{ARCO}(i, j) \in A$$

Esiste un altro algoritmo, l'algortimo di Bellman-Ford, che risolve il problema dei cammini minimi nel caso più generale in cui i pesi degli archi possono essere negativi.

```
1   procedure CAMMINIMINIMI(var G: grafo; var r: nodo);
2     T[r] := 0;
3     DIST[r] := 0;
```

```
    for k = 1 to n do
      if k != r then
        T[k] := r;
        DIST[k] := MAXINT;
    INSERISCI(r, S);
    while not INSIEMEVUOTO(S) do
      i := LEGGI(S);
      CANCELLA(i, S);
    for j in A[i] do /* A e' l'insieme di adiacenza */
      if DIST[i] + Cij < DIST[j] then
        T[j] := i;
        DIST[j] := DIST[i] + Cij;
        if not APPARTIENE(j, S) then
          INSERISCI(j, S)
```

Algoritmo di Dijkstra Se la struttura S è una coda con priorità,rappresentata mediante una lista non ordinata, si ottiene un algoritmo noto dal 1959 e attribuito a Dijkstra.
In questo caso le operazioni LEGGI e CANCELLA sono gli operatori basici disponbili nell'algebra della coda con priorità:

MIN:	(priorcoda) $\rightarrow$ tipoelem
CANCELLAMIN:	(priorcoda) $\rightarrow$ priorcoda

Gli elementi della coda con priorità sono i nodi del grafo e le priorità associate altro non sono che le distanze dal nodo origine r. Ad ogni iterazione è estratto da S il noto avente priorità, e quindi distanza, minima.

Algoritmo di Kruskal L'algoritmo di Kruskal permette di risolvere il problema del minimo albero di copertura. Dato un grafo non orientato e connesso $G = (N, A)$, con pesi positivi sugli archi, trovare un albero di copertura per G, cioè un albero avente tutti i nodi in N, ma solo alcuni archi in A, in modo tale che sia minima la somma dei pesi associati agli archi.
Il problema può essere risolto con molti algoritmi, dei quali i più noti si devono a Kruskal (1956) e a Prim (1957).
L'algoritmo di Kruskal usa la tecnica "greedy" e, dopo aver ordinato gli archi secondo i pesi crescenti, li esamina in tale ordine, inserendoli nella soluzione se non formano cicli con altri archi già scelti.
Ad un livello molto generale, l'algoritmo è esprimibile in questi termini:

```
procedure kruskal(var G: grafo);
  T := {};
  for a = 1 to m do
    if (ARCO a = (i, j) non forma ciclo con altri archi di T) then
      T := T + a;

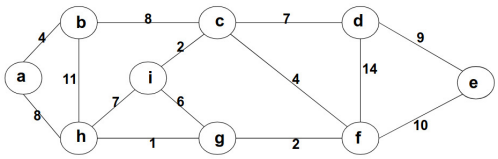
```

Possiamo rappresentare il grafo G come preferiamo. Nella realizzazione data di seguito il grafo è realizzato con un vettore di archi e l'albero T con una lista di archi, realizzata con puntatori.

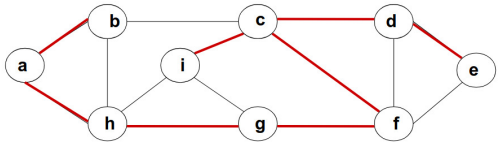
```
ARCO: tipo strutturato con componenti
  - i, j: integer;
  - peso: integer
GRAFO: tipo strutturato con componenti
  - A: array di maxlung elementi di tipo arco;
  - n, m: integer

```

Dato il seguente grafo:



Una volta che si considera un arco, se questo unisce due alberi distinti nella foresta, l'arco viene aggiunto alla foresta e i due alberi fusi in uno.



Nell'algoritmo di Kruskal la costruzione di T avviene per unione di componenti connesse rappresentabili come insiemi disgiunti, due componenti si fondono in una con l'aggiunta di un nuovo arco.
Come è noto, la struttura mfset detta merge-find set è una partizione di un insieme in sottoinsiemi disgiunti detti componenti. Dunque è possibile utilizzare una struttura dati di tipo mfset per costruire T.

```
procedure kruskal(var G: grafo);
  var h: integer;
  var T: lista;
  var S; mfset;
  CREALISTA(T);
  ORDINA(G.A(1), ..., G.A(G.m)) per ordine crescente di G.A(h).PESO;
  CREAMFSET(G.n, S);
  for h = 1 to G.m do begin
    if not TROVA(A(h).i, A(h).j, S) then
      FONDI(A(h).i, A(h).j, S)
      INSLISTA(PRIMOLISTA(T), A(h), T)

```

14.5 La tecnica divide-et-impera

Questa tecnica deriva dall'idea di determinarela strategia di un problema facendo ricorso al principio di decomposizione induttiva. È necessario disporre di:

- una relazione di ordinamento sulle istanze del problema, basata sulla dimensione dell'input;
- un metodo di risoluzione diretto per tutte le istanze del problema che non superano una prefissata dimensione limite;
- un meccanismo per suddividere i dati di ingresso relativi ad un'istanza in diverse parti, ciascuna di dimensione minore di quella originaria e rappresentante l'input di una nuova istanza dello stesso problema;
- un meccanismo per comporre le soluzioni per le istanze individuate dalla suddivisione, per ottenere la soluzione per l'istanza originaria.

L'algortimo divide-et-impera ha il seguente schema:

1. se l'input ha dimensione inferiore a un certo valore k allora utilizza un metodo diretto per ottenere il risultato;
2. altrimenti, dividi l'input in parti, ciascuna di dimensione inferiore all'input originario (*divide*);
3. esegui ricorsivamente l'algoritmo su ciascuno degli input individuati al passo precedente;

4. componi i risultati ottenuti al passo precedenete ottenendo il risultato per l'istanza originaria (*impera*).

L'applicazione più nota di questa tecnica si ha negli algoritmi di ordinamneto (*natural-merge-sort* e *quicksort*).

Esempio : Problema di minimo e massimo simultanei

In alcune applicazioni serve trovare il minimo e il massimo in un insieme di n elementi simulataneamente. Per esempio, un programma grafico può aver bisogno di rappresentare in scala un insieme di dati (x, y): in questo caso va determinato il minimo e il massimo di ogni coordinata.
Cercando il minimo e il massimo in modo indipendente ci vorrà un totale di $2(n-1)$ confronti. Mantenendo gli elementi minimo e massimo via via incontrati e confrontando i due elementi della coppia di input sono sufficienti $3(n/2)$ confronti. L'algoritmo è il seguente:

1. Se la dimensione del vettore non supera 2, allora calcola direttamente, mediante unico confronto, il minimo e il massimo.

2. Altrimenti, dividi il vettore in due sottovettori della stessa dimensione, calcola ricorsivamente il minimo *min1* e il massimo *max1* del primo sottovettore e il secondo *min2* e *max2* del secondo sottovettore.

3. Determina il minimo e il massimo del vettore complessivo confrontando *min1* con *min2* e *max1* con *max2*.

La tecnica divide-et-impera è una tecnica generativa che fa uso della decomposizione induttiva per determinare il metodo solutivo. L'efficacia della tecnica si manifesta attraverso due aspetti:

- consente di progettare algoritmi semplici e intuitivi attraverso l'induzione;

- spesso questi algoritmi hanno prestazioni migliori rispetto ad altri, tipicamente a quelli del paradigma selettivo.

L'efficienza degli algoritmi dipende dal numero a di sottoproblemi generati, dalla dimensione b dei dati di ingresso ai sottoproblemi e dal costo $f(n)$ necessario per la scomposizione del problema e la composizione dei risultati.